PrMS - Product Requirements & Analysis

Productivity Management System - Complete Specification

Product Requirements Analysis Team

January 19, 2026

Chapter 1: Executive Summary & Project Overview

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Foundation Phase

Table of Contents

- 1.1 Project Vision
- 1.2 Target Audience
- 1.3 Core Value Proposition
- 1.4 Key Success Metrics

1.1 Project Vision

1.1.1 Executive Overview

PrMS (Productivity Management System) is a dual-purpose productivity platform designed to consolidate fragmented productivity tools for both individuals and small enterprises. Unlike traditional productivity applications that cater exclusively to either personal users or business entities, PrMS provides a unified platform that adapts to both contexts through intelligent feature allocation and context-aware interfaces.

1.1.2 The Problem Were Solving

For Individuals: Modern professionals and individuals face unprecedented challenges in managing their increasingly complex lives:

- Fragmentation Fatigue: The average person uses 7-12 different apps for productivity (calendar apps, finance trackers, to-do lists, habit trackers, note-taking apps, etc.), leading to:
  - Data silos where information is scattered across platforms
  - Cognitive overhead from context switching between applications
  - Subscription fatigue from managing multiple paid services
  - Inconsistent user experiences and learning curves

  - Inability to see holistic productivity patterns
- Lack of Integration: Current solutions dont communicate with each other:
  - Calendar events dont inform financial planning
  - Task completion doesnt correlate with habit tracking
  - Time tracking exists in isolation from goal achievement
  - No unified dashboard showing complete productivity picture
- Analysis Paralysis: Without centralized analytics, individuals struggle to:
  - Identify productivity patterns and bottlenecks
  - Understand time allocation across different life areas
  - Make data-driven decisions about habit formation
  - Track ROI on personal development initiatives
- Cost Inefficiency: Paying for multiple separate subscriptions:
  - Average cost: $50-120/month across 5-10 productivity apps
  - Many features go unused or underutilized
  - No flexibility to pay only for needed features

For Small Enterprises: Small businesses and startups face operational inefficiencies that hinder growth:

- Tool Sprawl: SMBs typically use 10-20 different software solutions:
  - Inventory management systems
  - Accounting software
  - Project management tools
  - CRM platforms
  - Communication apps
  - Time tracking software
  - Document management systems
  - Result: Annual software costs of $5,000-50,000 for 5-50 employee businesses
- Integration Nightmares:
  - Manual data entry between systems leads to errors
  - No single source of truth for business operations
  - Difficulty generating comprehensive business intelligence
  - API integration costs and technical complexity
- Scalability Issues:
  - Solutions either too basic or enterprise-grade (too complex/expensive)
  - Difficult to add/remove features as business needs evolve
  - Vendor lock-in with rigid pricing models
  - Training overhead when adopting new tools
- Process Inefficiency:
  - Redundant tasks performed manually
  - Lack of automation in routine operations
  - Poor visibility into resource utilization
  - Delayed reporting and decision-making

1.1.3 Our Solution

PrMS addresses these challenges through:

Unified Platform Architecture

- Single application providing 12+ individual productivity features
- 12+ enterprise management tools under one roof
- Seamless data flow between all features
- Consistent UI/UX across all modules
- Cross-platform availability (Web, iOS, Android)

Intelligent Context Switching

```
User Login → Choose Mode
                → Individual Mode → Personal Productivity Suite
                → Enterprise Mode → Business Management Suite
```

- Mode selection at login determines feature set and UI paradigm
- Users can have both individual and enterprise accounts
- Enterprise admins can manage team access to specific features
- Seamless switching between modes for business owners

Modular Feature Economy

- Free Tier: 2-3 core features to demonstrate value
  - Individual: Task Management + Calendar + Notes
  - Enterprise: Project Management + Team Communication + Time Tracking
- Premium Features: Pay only for what you need
  - À la carte feature selection
  - Bundle discounts for feature packages
  - Flexible monthly or annual billing
  - No forced upgrades to access single features

Centralized Intelligence

- Unified data warehouse for all activities
- Cross-feature analytics and insights
- AI-powered productivity recommendations
- Predictive alerts and automation triggers
- Comprehensive reporting dashboard

1.1.4 Vision Statement

To become the singular, indispensable productivity platform that empowers individuals to achieve their personal goals and enables small enterprises to operate with the efficiency of large corporationsall while maintaining simplicity, affordability, and user autonomy.

1.1.5 Mission Statement

We eliminate productivity fragmentation by delivering an integrated, intelligent, and infinitely adaptable platform that centralizes every aspect of personal and professional productivity management.

1.1.6 Core Principles

1. Simplicity Through Integration
   - Complex functionality delivered through intuitive interfaces
   - Reduce cognitive load by centralizing all tools
   - Progressive disclosure of advanced features
2. User Autonomy
   - Freedom to choose only needed features
   - No forced bundling or feature bloat
   - Transparent pricing with no hidden costs
   - Easy data export and platform migration
3. Cross-Platform Consistency
   - Identical feature parity across web and mobile
   - Real-time synchronization across all devices
   - Responsive design adapting to device capabilities
   - Offline-first architecture with conflict resolution
4. Privacy & Security First
   - End-to-end encryption for sensitive data
   - GDPR, CCPA, and SOC 2 compliance
   - User data ownership and control
   - Optional self-hosted deployment for enterprises
5. Continuous Evolution
   - Regular feature updates based on user feedback
   - API-first design for third-party integrations
   - Plugin architecture for custom extensions
   - Community-driven feature roadmap

1.1.7 Strategic Objectives (2026-2028)

Year 1 (2026): Foundation & Market Entry

- Q1-Q2: Complete MVP development with 6 individual + 6 enterprise features
- Q2-Q3: Private beta with 500 selected users (250 individual, 250 enterprise)
- Q3-Q4: Public launch with full 12+12 feature set
- Target: 10,000 registered users, 500 paying customers
- Revenue Target: $250,000 ARR

Year 2 (2027): Growth & Expansion

- Expansion: Add 6 new features per category based on demand
- Platform: Launch iOS and Android native apps
- Integration: Connect with top 20 third-party services (Google, Microsoft, Slack, etc.)
- Target: 100,000 registered users, 10,000 paying customers
- Revenue Target: $5M ARR

Year 3 (2028): Scale & Dominance

- Market Position: Top 5 productivity platform globally
- Enterprise: Launch dedicated enterprise tier with SSO, custom branding
- AI Features: Introduce predictive analytics and automation

- Target: 1M registered users, 100,000 paying customers
- Revenue Target: $50M ARR

1.1.8 Competitive Landscape

Current Market Leaders & Gaps

Competitor	Strengths	Weaknesses	PrMS Advantage
Notion	Flexible workspace, strong note-taking	Steep learning curve, no finance/inventory tools	Dedicated tools for each use case, simpler UX
Monday.com	Enterprise project management	Expensive, poor individual features	Dual-mode with affordable individual tier
Todoist	Simple task management	Limited scope, no enterprise features	Comprehensive feature set in one platform
QuickBooks	Strong accounting	Single-purpose, expensive	Part of broader productivity suite
Asana	Team collaboration	Complex, no finance/inventory	All-in-one with business operations tools
Google Workspace	Integration, reliability	Fragmented experience, limited customization	Unified purpose-built productivity focus
Microsoft 365	Enterprise features	Overwhelming, expensive, complex	Simplified, modular, affordable

Market Differentiation PrMS Unique Advantages:

1. Dual-Mode Architecture: Only platform serving both individuals AND enterprises equally
2. Modular Pricing: Pay-per-feature model vs. forced bundles
3. True Integration: All features share unified data model
4. Simplified Complexity: Enterprise-grade without enterprise-complexity
5. Centralized Analytics: Cross-feature insights unavailable elsewhere

Target Market Size Individual Productivity Market: - TAM (Total Addressable Market): 2.5B knowledge workers globally - SAM (Serviceable Available Market): 500M tech-savvy professionals with productivity pain points - SOM (Serviceable Obtainable Market): 5M users by Year 3 (1% of SAM)

SMB Enterprise Market: - TAM: 200M small businesses globally (1-50 employees) - SAM: 50M businesses actively seeking digital transformation - SOM: 500K businesses by Year 3 (1% of SAM)

Combined Revenue Potential: - Individual ARPU: $120/year 5M users = $600M potential - Enterprise ARPU: $2,400/year 500K businesses = $1.2B potential - Total Market Opportunity: $1.8B by Year 3

1.2 Target Audience

1.2.1 Individual User Personas

Persona 1: Ambitious Alex - The Career Climber Demographics: - Age: 25-35 - Occupation: Mid-level professional (Marketing Manager, Software Engineer, Consultant) - Income: $60,000-$120,000/year - Location: Urban areas, tech hubs - Education: Bachelors degree or higher

Psychographics: - Goals: Career advancement, skill development, work-life balance - Pain Points: - Juggling multiple projects at work and personal life - Tracking professional development goals - Managing side hustles alongside full-time work - Maintaining healthy habits during busy periods - Tech Savviness: High - early adopter, comfortable with new tools - Current Tools: Using 8-12 different apps (Trello, Todoist, Mint, Google Calendar, Evernote, RescueTime, etc.) - Spending Behavior: Willing to pay $15-30/month for tools that save time

What Alex Needs from PrMS: - Unified task and project management - Time tracking to understand productivity patterns - Goal setting with measurable progress - Finance tracking to monitor savings and investments - Calendar integration for time blocking - Daily journal to reflect on progress

Key Trigger: Im tired of switching between 10 different apps just to plan my day

Persona 2: Entrepreneurial Emma - The Solopreneur Demographics: - Age: 28-45 - Occupation: Freelancer, consultant, small business owner (1-3 person operation) - Income: $40,000-$150,000/year (variable) - Location: Mix of urban and suburban - Education: Varied - some college to advanced degrees

Psychographics: - Goals: Business growth, client satisfaction, financial stability - Pain Points: - Wearing multiple hats (marketing, sales, operations, finance) - Tracking billable hours and project profitability - Managing client relationships and invoicing - Separating business and personal finances/tasks - Scaling operations without hiring staff - Tech Savviness: Medium-High - pragmatic adopter - Current Tools: Mix of free and paid tools (Wave, FreshBooks, Trello, Google Suite) - Spending Behavior: Cost-conscious, needs clear ROI, willing to pay $30-75/month if it saves 10+ hours

What Emma Needs from PrMS: - Client/project management (CRM-lite) - Time tracking and invoicing - Finance management (income, expenses, taxes) - Task prioritization across multiple clients - Document organization - Professional dashboard for client presentations

Key Trigger: I need to run my business like a real company without paying for enterprise software

Persona 3: Wellness-Focused Wei - The Holistic Optimizer Demographics: - Age: 22-40 - Occupation: Varied (Teacher, Designer, Healthcare worker, Remote worker) - Income: $35,000-$80,000/year - Location: Diverse geographic distribution - Education: Associates degree to Masters

Psychographics: - Goals: Personal growth, health optimization, mindfulness, skill mastery - Pain Points: - Tracking multiple health and wellness habits - Maintaining consistency with routines - Understanding what habits actually improve wellbeing - Balancing productivity with self-care - Journaling and mood tracking - Tech Savviness: Medium - selective adopter, prefers simple interfaces - Current Tools: Habit tracking apps (Streaks, Habitica), meditation apps, fitness trackers - Spending Behavior: Moderate spender, values quality over features, $10-25/month sweet spot

What Wei Needs from PrMS: - Comprehensive habit tracking with streak management - Mood and

wellness journaling - Health metrics integration - Focus timer for deep work and meditation - Goal visualization and celebration - Gentle reminders and notifications

Key Trigger: I want to see how my habits actually affect my mood and productivity

Persona 4: Student Sarah - The Academic Achiever Demographics: - Age: 18-28 - Occupation: College/Graduate student or recent graduate - Income: $0-$40,000/year (part-time work, internships) - Location: College towns, urban areas - Education: Currently pursuing or recently completed degree

Psychographics: - Goals: Academic success, internship/job placement, skill building - Pain Points: - Managing multiple course schedules and deadlines - Balancing studies with part-time work - Tight budget for productivity tools - Building professional skills and portfolio - Time management with irregular schedules - Tech Savviness: High - digital native - Current Tools: Mostly free tools (Google products, free tiers of apps) - Spending Behavior: Price-sensitive, prefers free tier, might pay $5-10/month

What Sarah Needs from PrMS: - Academic calendar with assignment tracking - Study time tracking and analytics - Note-taking and document organization - Budget tracking (limited income) - Goal setting for career preparation - Simple, clean interface

Key Trigger: I need a free tool that does what Im currently using 5 apps for

1.2.2 Enterprise User Personas

Persona 1: Scaling Sam - The Startup Founder Demographics: - Age: 28-45 - Role: Founder/CEO of startup or small business - Company Size: 5-20 employees - Industry: Tech, Services, E-commerce - Revenue: $500K-$5M ARR

Psychographics: - Goals: Rapid scaling, operational efficiency, team productivity, funding readiness - Pain Points: - Managing costs while growing - Establishing processes without bureaucracy - Visibility into team productivity and resource allocation - Too many tools creating integration headaches - Need analytics for investor reports - Tech Savviness: Very high - technical founder or tech-comfortable - Current Tools: Patchwork of 15-25 tools (Jira, Slack, QuickBooks, Salesforce, etc.) - Budget: $2,000-$10,000/month for all software, looking to consolidate - Decision Factors: ROI, scalability, time-to-implementation

What Sam Needs from PrMS: - Unified operations dashboard - Project and resource management - Basic CRM for sales pipeline - Financial tracking and reporting - Team time and attendance - Inventory/order management (if applicable)

Key Trigger: Were paying for 20 different tools when we only need 70% of their features

Persona 2: Operations Olivia - The Operations Manager Demographics: - Age: 32-50 - Role: COO, Operations Manager, General Manager - Company Size: 10-50 employees - Industry: Retail, Manufacturing, Professional Services - Reporting: Reports to Owner/CEO

Psychographics: - Goals: Process optimization, cost reduction, employee efficiency, scalability - Pain Points: - Manual processes eating up time - Tracking inventory, orders, and fulfillment - Employee scheduling and attendance - Generating reports for leadership - Training employees on multiple systems - Tech Savviness: Medium - practical adopter, not a technologist - Current

Tools: Industry-specific tools + spreadsheets for everything - Budget: $5,000-$20,000/month, cost-conscious - Decision Factors: Ease of use, training requirements, support quality, total cost of ownership

What Olivia Needs from PrMS: - Inventory management with low-stock alerts - Order processing and fulfillment tracking - Employee time tracking and scheduling - Vendor and supplier management - Financial overview and expense tracking - Simple reporting for executive review

Key Trigger: I need something my team can actually learn without me holding their hand

Persona 3: Agency Aaron - The Creative Agency Owner Demographics: - Age: 30-45 - Role: Agency Owner/Creative Director - Company Size: 8-30 employees - Industry: Marketing, Design, Advertising, Consulting - Revenue: $1M-$10M ARR

Psychographics: - Goals: Client satisfaction, profitable projects, team utilization, creative excellence - Pain Points: - Tracking multiple client projects simultaneously - Ensuring profitability per project - Resource allocation across projects - Client communication and expectations - Billing and invoicing complexity - Team collaboration and file sharing - Tech Savviness: High - appreciates good design and UX - Current Tools: Project management (Asana, Monday), Time tracking (Harvest), Invoicing (FreshBooks), CRM (HubSpot) - Budget: $3,000-$15,000/month, values premium solutions - Decision Factors: Design quality, client presentation features, integration capabilities

What Aaron Needs from PrMS: - Multi-project management with client views - Time tracking by project and client - Resource allocation and capacity planning - Invoicing and billing integration - Document management and client portals - Team collaboration tools - Analytics on project profitability

Key Trigger: I need to see real-time profitability on every project while keeping clients happy

Persona 4: Retail Rita - The Small Retail Business Owner Demographics: - Age: 35-55 - Role: Owner/Manager of small retail business - Company Size: 3-15 employees - Industry: Retail (online + physical stores) - Revenue: $250K-$3M annual sales

Psychographics: - Goals: Inventory optimization, sales growth, customer retention, operational efficiency - Pain Points: - Inventory management across multiple channels - Order fulfillment and tracking - Customer relationship management - Employee scheduling and payroll - Managing suppliers and vendors - Financial tracking (COGs, margins, cash flow) - Tech Savviness: Low-Medium - needs simple, purpose-built tools - Current Tools: POS system, spreadsheets, basic accounting software - Budget: $500-$3,000/month, very ROI-focused - Decision Factors: Simplicity, customer support, ROI, reliability

What Rita Needs from PrMS: - Inventory management with automated reordering - Order management and fulfillment - Basic CRM for customer retention - Employee scheduling and time tracking - Vendor management - Financial dashboard with profit margins - Low/no learning curve

Key Trigger: Im losing money because I cant track inventory and orders efficiently

1.2.3 User Segmentation Matrix

Segment	Priority	Market Size	ARPU	Acquisition Cost	LTV/CAC Ratio
Individual - Career Professional	High	100M	$180/yr	$50	3.6x
Individual - Solopreneur	High	50M	$240/yr	$75	3.2x
Individual - Wellness	Medium	200M	$120/yr	$40	3.0x
Individual - Student	Low	150M	$60/yr	$20	3.0x
Enterprise - Startup	High	5M	$3,000/yr	$500	6.0x
Enterprise - SMB Ops	High	10M	$4,500/yr	$750	6.0x
Enterprise - Agency	Medium	2M	$6,000/yr	$1,000	6.0x
Enterprise - Retail	Medium	8M	$2,400/yr	$400	6.0x

1.2.4 Initial Target Focus (MVP Phase)

Primary Target: 1. Ambitious Alex (Career Professionals) - Largest addressable market with high willingness to pay 2. Scaling Sam (Startup Founders) - High ARPU and strong word-of-mouth potential

Secondary Target: 3. Entrepreneurial Emma (Solopreneurs) - Bridge between individual and enterprise features 4. Operations Olivia (SMB Operations) - Clear ROI story, recurring pain points

Future Expansion: - Wellness-focused individuals (add specialized health features) - Students (optimize free tier for viral growth) - Agencies (premium tier with client collaboration) - Retail (add POS integration and e-commerce features)

1.3 Core Value Proposition

1.3.1 Value Proposition Canvas

For Individuals Customer Jobs: - Manage tasks and projects efficiently - Track and improve productivity - Maintain healthy habits and routines - Organize personal finances - Plan and achieve goals - Reduce app fragmentation - Access productivity data anywhere

Pains: - Information scattered across multiple apps - Expensive subscription fatigue ($50-120/month) - Inconsistent interfaces and learning curves - No holistic view of productivity - Manual data entry and duplication - Limited analytics and insights - Time wasted on context switching

Gains: - Centralized productivity hub - Significant cost savings (single subscription) - Consistent,

familiar interface - Comprehensive productivity analytics - Automated data synchronization - AI-powered insights and recommendations - Cross-platform accessibility

PrMS Pain Relievers: - Single Application: All tools in one place, no more app switching
- Unified Pricing: $12-25/month vs. $50-120 across multiple apps
- Seamless Integration: Data flows automatically between features
- Consistent UX: Learn once, use everywhere
- Smart Analytics: Cross-feature insights reveal productivity patterns
- Flexible Features: Pay only for what you need

PrMS Gain Creators: - Productivity Dashboard: See complete productivity picture at a glance
- AI Insights: Predictive recommendations based on your patterns
- Goal Tracking: Connect tasks, habits, and time to goals
- Financial Clarity: Understand spending in context of time and activities
- Health Integration: Correlate productivity with wellness metrics
- True Mobility: Same features on web, iOS, and Android

For Enterprises Customer Jobs: - Manage projects and teams effectively - Track inventory and orders - Monitor finances and profitability - Coordinate employee schedules - Maintain customer relationships - Generate business intelligence - Ensure operational efficiency

Pains: - Using 10-20 disconnected tools ($5K-50K/year) - Manual data entry between systems - No single source of truth - Complex integration and API costs - Difficult to scale tools with business - Training overhead for new tools - Limited cross-functional visibility

Gains: - Operational excellence with fewer tools - Significant cost reduction - Real-time business intelligence - Scalable modular system - Easy employee onboarding - Comprehensive analytics - Process automation

PrMS Pain Relievers: - Unified Platform: Replace 10+ tools with one system
- Cost Efficiency: $200-800/month vs. $5K-50K/year for tool sprawl
- Zero Integration: All features share data automatically
- Easy Scaling: Add/remove features and users as needed
- Quick Training: Consistent interface reduces learning curve
- Single Dashboard: Complete business view in one place

PrMS Gain Creators: - Operations Hub: Inventory, orders, projects, team in one view
- Real-Time Analytics: Live dashboards for decision-making
- Automation Engine: Reduce redundant manual tasks
- Resource Optimization: See team utilization and allocation
- Financial Intelligence: Track profitability by project/product
- Customer Success: Integrated CRM with communication history

1.3.2 Unique Selling Propositions (USPs)

USP #1: True Dual-Mode Platform One Platform, Two Worlds

Unlike competitors that bolt-on features or separate products: - Architected from ground-up for both contexts - Seamless mode switching for users wearing multiple hats - Shared infrastructure ensures feature parity - Solopreneurs graduate to enterprise features as they grow

Competitive Advantage: - Notion: Great for individuals, weak for operations - Monday.com: Strong for teams, poor for personal use - PrMS: Purpose-built for both with equal excellence

USP #2: Intelligent Feature Modularity Pay for What You Use, Not What We Bundle

Revolutionary à la carte model: - Choose exactly the features you need - Add/remove features monthly with no penalty - Bundle discounts when purchasing related features - Never pay for unused functionality

Pricing Example:

```
Base Platform: $5/month (includes core infrastructure)

Individual Features (each $3-8/month):
  Task Management: $3/month
  Calendar & Events: $4/month
  Finance Manager: $6/month
  Habit Tracker: $4/month
  Time Tracking: $5/month
  Premium Analytics: $8/month

Custom Individual Bundle: $5 + (3 free features) + additional at 20% discount

vs. Competitors:
- Notion: $10/month (all or nothing)
- Todoist Premium: $5/month (single feature)
- Mint: Free but limited, paid alternatives $10-15/month
- Total PrMS Value: Same functionality for 40-60% less
```

USP #3: Centralized Intelligence Your Data, Your Insights, Your Advantage

Unique cross-feature analytics: - Time-Money Correlation: See how time spent relates to income/expenses - Productivity-Wellness: Understand impact of habits on work output - Goal-Activity Alignment: Verify daily actions support long-term goals - Prediction Engine: AI forecasts based on historical patterns - Anomaly Detection: Alerts when patterns deviate (spending spike, productivity drop)

No Competitor Offers: - Isolated tools cant correlate data across domains - Manual export/import loses real-time insights - PrMS creates value through integration others cant match

USP #4: Enterprise-Grade Without Enterprise Complexity Big Company Power, Small Company Simplicity

For SMBs that cant afford or dont need full ERP systems: - Full-featured inventory, CRM, finance, project management - But designed for 5-50 person teams, not 5,000 - Self-service setup vs. 6-month implementation projects - Modern UX vs. legacy enterprise interfaces - $200-800/month vs. $5K-50K/year

Sweet Spot: - Too advanced: Spreadsheets and disconnected free tools - PrMS: Perfect fit for SMBs - Too complex: SAP, Salesforce, Oracle (enterprise giants)

USP #5: Platform Consistency True Cross-Platform, Not Cross-Compromise

Full feature parity across all platforms: - Web app (Next.js): Full functionality, responsive design - iOS app (Swift): Native performance, iOS design language - Android app (Kotlin): Material Design, Android conventions - Offline-first: Full functionality without internet - Real-time sync: Changes reflected instantly across devices

Competitive Advantage: - Most competitors: Web-first, mobile is limited - Some: Mobile-first, web is afterthought - PrMS: Equal investment in all platforms

1.3.3 Value Proposition Statements

Individual Tagline:

> Your entire productivity life, unified.

Expanded Statement: > PrMS brings together your tasks, calendar, finances, habits, goals, and time tracking into one intelligent platform that not only organizes your life but helps you understand and improve it. Stop paying for 10 apps. Start living more productively.

Enterprise Tagline:

> Run your business like an enterprise, priced for reality.

Expanded Statement: > PrMS gives small businesses the operational tools of Fortune 500 companiesinventory management, CRM, project tracking, financial analyticswithout the complexity or cost. One platform, complete visibility, real growth.

Combined Brand Tagline:

> Centralize. Simplify. Amplify.

Emotional Appeal: - Centralize: Bring order to chaos, reduce overwhelm - Simplify: Make complex tasks manageable, reduce friction - Amplify: Enhance productivity, multiply impact

1.3.4 Competitive Positioning Map

```
                    High Complexity


                 SAP          Salesforce

              Microsoft      Oracle
              Dynamics



             Monday.com      Asana
Low                                          High
Price            PrMS               Price

        Todoist  Notion     Jira
```

```
      Trello    Google
                Tasks

                     Low Complexity


PrMS Position: Mid-complexity, Mid-price
- More capable than simple tools (Todoist, Trello)
- Simpler than enterprise monsters (SAP, Salesforce)
- More affordable than feature-comparable solutions
- Broader scope than single-purpose tools
```

1.3.5 Value Metrics by User Segment

Individual Users

Metric	Current State (Without PrMS)	With PrMS	Value Created
Monthly Tool Cost	$50-120 (7-12 apps)	$12-25 (one platform)	70-80% savings
Time Spent on Admin	5-7 hours/month managing tools	1-2 hours/month	75% reduction
App Switching	50-100 times/day	0 (single platform)	Eliminated
Data Entry	3-5 hours/month (duplicates)	0 (auto-sync)	100% elimination
Productivity Insights	Limited (siloed data)	Comprehensive	Priceless
Learning Curve	10-15 hours annually (new tools)	2-3 hours (one system)	80% reduction

Annual Value per Individual User: - Direct savings: $300-1,140/year (subscription costs) - Time savings: 48-60 hours/year Œ $50/hour opportunity cost = $2,400-3,000 - Total Annual Value: $2,700-4,140 - PrMS Cost: $144-300/year - ROI: 9-14x

Enterprise Users

Metric	Current State	With PrMS	Value Created
Annual Software Cost	$10K-50K (10-20 tools)	$2,400-9,600	50-80% savings
Integration Costs	$5K-20K/year (APIs, middleware)	$0 (built-in)	100% elimination

Metric	Current State	With PrMS	Value Created
Training Time	40-80 hours/year (multiple systems)	10-15 hours/year	70-80% reduction
Admin Overhead	1-2 FTE managing tools	0.25 FTE	75% reduction
Report Generation	20-40 hours/month (manual)	2-4 hours/month	90% reduction
Data Accuracy	85-90% (manual entry errors)	98-99% (auto-sync)	Significant quality improvement

Annual Value per SMB (20 employees): - Software consolidation: $7,600-40,400/year - Integration elimination: $5,000-20,000/year - Admin time savings: $40,000-80,000/year (1 FTE) - Training reduction: $4,000-8,000/year - Total Annual Value: $56,600-148,400 - PrMS Cost: $4,800-9,600/year - ROI: 6-15x

---

1.4 Key Success Metrics

1.4.1 North Star Metric

Primary North Star: > Weekly Active Engaged Users (WAEU)

Definition: Users who perform at least 3 meaningful actions across 2+ features in a week

Why This Metric: - Measures true engagement, not just logins - Validates cross-feature value proposition - Predicts retention and conversion - Aligns team on user value delivery

Target Trajectory: - MVP Launch (Q3 2026): 500 WAEU - End of Year 1 (Q4 2026): 2,000 WAEU - End of Year 2 (Q4 2027): 25,000 WAEU - End of Year 3 (Q4 2028): 200,000 WAEU

1.4.2 Product Metrics

Acquisition Metrics

Metric	Definition	Year 1 Target	Year 2 Target	Year 3 Target
Total Registered Users	All signups	10,000	100,000	1,000,000
Individual Accounts	Personal mode signups	7,500	75,000	800,000
Enterprise Accounts	Business mode signups	2,500	25,000	200,000
Organic Sign-up Rate	Non-paid acquisition %	40%	55%	70%
Viral Coefficient	New users per existing user	0.3	0.5	0.8

Metric	Definition	Year 1 Target	Year 2 Target	Year 3 Target
Sign-up Conversion Rate	Landing page account	15%	22%	30%

Activation Metrics

Metric	Definition	Target
Time to First Value	Sign-up first meaningful action	<5 minutes
Onboarding Completion	% completing setup wizard	>80%
Feature Adoption (Free)	% using free features within 7 days	>90%
Feature Exploration	% trying 1 premium feature (trial)	>60%
Aha Moment Rate	% reaching activation milestone	>70%

Activation Milestones: - Individual: Create 3 tasks + 1 calendar event + add 1 financial transaction - Enterprise: Create 1 project + invite 1 team member + add 1 inventory item

Engagement Metrics

Metric	Definition	Target
Daily Active Users (DAU)	Users with 1 action/day	35% of registered
Weekly Active Users (WAU)	Users with 1 action/week	65% of registered
Monthly Active Users (MAU)	Users with 1 action/month	80% of registered
DAU/MAU Ratio	Stickiness indicator	>40%
Session Frequency	Average sessions per user/week	8-12 sessions
Session Duration	Average time per session	8-15 minutes
Features Used per Session	Cross-feature engagement	2.5 average
Power User %	Using 5+ features regularly	>25%

Retention Metrics

Metric	Definition	Target
D1 Retention	Return next day	>50%
D7 Retention	Return within 7 days	>40%
D30 Retention	Return within 30 days	>30%
D90 Retention	Return within 90 days	>25%
12-Month Retention	Still active after 1 year	>60%
Churn Rate (Monthly)	% canceling per month	<5%
Net Retention Rate	Revenue retention + expansion	>110%

Monetization Metrics

Metric	Definition	Year 1	Year 2	Year 3
Free to Paid Conversion	% upgrading from free	5%	10%	15%
Trial to Paid Conversion	% converting after premium trial	25%	35%	45%
Average Revenue Per User (ARPU)	Total revenue / total users	$25	$50	$65
ARPU - Individual	Individual tier average	$15	$30	$40
ARPU - Enterprise	Enterprise tier average	$100	$180	$240
Customer Lifetime Value (LTV)	Total revenue per customer	$300	$800	$1,500
Customer Acquisition Cost (CAC)	Cost to acquire paying customer	$60	$120	$180
LTV/CAC Ratio	Unit economics health	5x	6.7x	8.3x
Payback Period	Months to recover CAC	3 months	2.5 months	2 months
Monthly Recurring Revenue (MRR)	Predictable monthly revenue	$10K	$400K	$5M
Annual Recurring Revenue (ARR)	Annualized revenue	$120K	$4.8M	$60M

Product Quality Metrics

Metric	Definition	Target
App Crashes	Crashes per user session	<0.1%
API Error Rate	Failed API calls	<0.5%
Page Load Time (Web)	Time to interactive	<2 seconds
App Launch Time (Mobile)	Cold start to usable	<3 seconds
Sync Latency	Cross-device sync delay	<2 seconds
Uptime	Service availability	>99.9%
Bug Resolution Time	Critical bugs fixed	<24 hours
Customer Satisfaction (CSAT)	Post-interaction survey	>4.5/5
Net Promoter Score (NPS)	Would recommend?	>50

1.4.3 Business Metrics

Financial Health

Metric	Year 1	Year 2	Year 3
Total Revenue	$250K	$5M	$60M
Gross Margin	85%	87%	90%
Operating Expenses	$800K	$3M	$25M
EBITDA	-$550K	$2M	$35M
Cash Burn Rate	$50K/month	$0 (profitable)	N/A
Runway	18 months	Infinite	Infinite
Capital Raised	$1M (seed)	$5M (Series A)	$0 (cash flow positive)

Growth Metrics

Metric	Year 1	Year 2	Year 3
MRR Growth Rate	15%/month	10%/month	8%/month
User Growth Rate	20%/month	12%/month	10%/month
Revenue Growth (YoY)	N/A (new)	1,900%	1,100%
Market Share	<0.1%	0.5%	3%

Team Metrics

Metric	Year 1	Year 2	Year 3
Team Size	8	25	80
Engineers	5	15	40
Product/Design	1	3	8
Sales/Marketing	1	4	18
Support/Ops	1	3	14
Revenue per Employee	$31K	$200K	$750K

1.4.4 Feature-Specific Success Metrics

Individual Features Performance Each feature tracked against: - Adoption Rate: % of individual users activating feature - Usage Frequency: Actions per active user per week - Retention Impact: Difference in retention for users of this feature - Conversion Impact: Influence on free-to-paid conversion - NPS Impact: Feature-specific satisfaction

Target Distribution:

```
Tier 1 Features (Must Use - 80%+ adoption):
  Task Management: 85% adoption
  Calendar & Events: 80% adoption
  Notes: 75% adoption

Tier 2 Features (High Value - 40-60% adoption):
```

```
  Finance Manager: 55% adoption
  Time Tracking: 50% adoption
  Habit Tracker: 45% adoption
  Goals: 40% adoption

Tier 3 Features (Specialized - 15-30% adoption):
  Health Tracker: 30% adoption
  Focus Timer: 25% adoption
  Journal: 25% adoption
  File Manager: 20% adoption
```

Enterprise Features Performance Target Distribution:

```
Tier 1 Features (Core Operations - 70%+ adoption):
  Project Management: 90% adoption
  Team Communication: 85% adoption
  Finance Suite: 75% adoption
  Time & Attendance: 70% adoption

Tier 2 Features (Growth Enablers - 40-60% adoption):
  CRM: 60% adoption
  Order Management: 55% adoption
  Analytics Dashboard: 50% adoption
  Document Management: 45% adoption

Tier 3 Features (Specialized - 20-40% adoption):
  Inventory Management: 40% adoption
  Invoice & Billing: 35% adoption
  Resource Planning: 30% adoption
  Vendor Management: 25% adoption
```

1.4.5 Milestone-Based Success Criteria

MVP Success (Q3 2026) Criteria: - 500 total users (350 individual, 150 enterprise) - 50 paying customers - 70% onboarding completion - >4.0 NPS from early adopters - <0.5% crash rate - All 6+6 core features functional

Public Launch Success (Q4 2026) Criteria: - 10,000 total users - 500 paying customers - $10K MRR - 40% D30 retention - All 12+12 features live - iOS and Android apps in stores - <5% monthly churn

Product-Market Fit Validation (Q2 2027) Criteria: - >40% of users would be very disappointed if product disappeared (Sean Ellis test) - >50 NPS - >10% organic growth rate - >5x LTV/CAC ratio - >30% users using 3+ features regularly - Customer testimonials and case studies

Scale Achievement (Q4 2028) Criteria: - 1M+ total users - 100K+ paying customers - $5M MRR ($60M ARR) - >60% 12-month retention - Profitable unit economics - Top 10 productivity app rankings - International expansion (3+ countries)

1.4.6 Leading vs. Lagging Indicators

Leading Indicators (Predictive): - Onboarding completion rate Predicts long-term retention - Time to first value Predicts activation success - Feature exploration rate Predicts conversion - NPS score Predicts organic growth - Cross-feature usage Predicts stickiness

Lagging Indicators (Outcome): - Monthly Recurring Revenue Result of all efforts - Customer Lifetime Value Result of retention + expansion - Market Share Result of competitive positioning - Profitability Result of efficient operations

1.4.7 Measurement & Analytics Stack

Tools & Infrastructure:

```
User Analytics:
  Amplitude: Product analytics and cohort analysis
  Mixpanel: Event tracking and funnel optimization
  Google Analytics 4: Traffic and acquisition

Product Intelligence:
  Pendo: In-app guidance and feature adoption
  FullStory: Session replay and user experience
  Hotjar: Heatmaps and user feedback

Business Metrics:
  ChartMogul: SaaS metrics and MRR tracking
  Stripe: Payment and revenue analytics
  Metabase: Custom business intelligence dashboards

Performance Monitoring:
  Datadog: Infrastructure and APM
  Sentry: Error tracking and alerting
  Pingdom: Uptime and performance monitoring
```

Reporting Cadence: - Daily: Core metrics dashboard (DAU, MRR, errors) - Weekly: Product review (features, engagement, retention) - Monthly: Business review (revenue, growth, unit economics) - Quarterly: Board metrics (ARR, growth rate, runway)

1.5 Summary & Next Steps

1.5.1 Chapter Summary

This executive summary has established:

1. Clear Vision: PrMS as the unified productivity platform serving both individuals and enterprises
2. Defined Audience: 4 individual personas and 4 enterprise personas with distinct needs
3. Strong Value Proposition: Centralization, cost savings, and intelligence unavailable in fragmented solutions

4. Measurable Success: Comprehensive metrics framework from acquisition to retention to revenue

1.5.2 Key Takeaways

- Market Opportunity: $1.8B potential by Year 3 in fragmented productivity market
- Unique Position: Only platform equally serving individual and enterprise needs
- Economic Model: 6-14x ROI for customers, 5-8x LTV/CAC for business
- Success Path: MVP PMF Scale with clear metrics at each stage

1.5.3 Foundation for Remaining Chapters

This overview provides context for: - Chapter 2-3: Detailed feature specifications for identified personas - Chapter 4: Technical architecture to deliver on platform promises - Chapter 5-6: Data models and APIs supporting integration vision - Chapter 7: UI/UX design serving distinct user modes - Chapter 8: Business model monetizing value proposition - Chapter 9: Roadmap executing vision with resource allocation - Chapter 10: Security and quality ensuring trust and reliability

---

Next Chapter: Chapter 2: Individual User Features

This document is part of the PrMS Product Requirements & Analysis documentation suite. For navigation, refer to the Main Index.

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Product Team - Status: Draft - Foundation Phase - Next Review: February 1, 2026

Chapter 2: Individual User Features

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Feature Specification Phase

Table of Contents

- 2.1 Event Tracker & Calendar Management
- 2.2 Finance Manager
- 2.3 Task Management & To-Do Lists
- 2.4 Habit Tracker
- 2.5 Goal Setting & Progress Monitoring
- 2.6 Time Tracking & Analytics
- 2.7 Note Taking & Documentation
- 2.8 Health & Wellness Tracker
- 2.9 Focus Timer (Pomodoro)
- 2.10 Personal Dashboard & Analytics
- 2.11 File Manager & Cloud Storage

- 2.12 Daily Journal & Mood Tracker

Feature Priority Matrix

Feature	Tier	Free/Premium	Priority	MVP
Task Management	Core	Free	P0	
Calendar & Events	Core	Free	P0	
Note Taking	Core	Free	P0	
Finance Manager	High Value	Premium	P1	
Time Tracking	High Value	Premium	P1	
Habit Tracker	High Value	Premium	P1	
Goal Setting	High Value	Premium	P2	
Personal Dashboard	High Value	Premium	P2	
Health Tracker	Specialized	Premium	P3	
Focus Timer	Specialized	Premium	P3	
Journal & Mood	Specialized	Premium	P3	
File Manager	Specialized	Premium	P3	

2.1 Event Tracker & Calendar Management

2.1.1 Overview

Purpose: Centralized calendar system for scheduling, event management, and time blocking to help users organize their time effectively.

Target Users: All individual users (90% expected adoption)

Tier: FREE (Core Feature)

Key Differentiators: - Smart time blocking with AI suggestions - Integration with tasks and goals - Multiple calendar views (day, week, month, agenda) - Color-coded categories with custom labels - Recurring events with complex patterns - Time zone support for global users

2.1.2 Core Features

Calendar Views Month View: - Standard grid calendar showing month at a glance - Color-coded events by category - Multi-day event spanning - Quick event creation by clicking date - Mini calendar widget for navigation

Week View: - Time-slot based view (hourly breakdown) - Drag-and-drop event rescheduling - Time blocking visualization - Conflicts and overlaps highlighted - Current time indicator

Day View: - Detailed hourly schedule - Event details visible inline - Quick add from any time slot - Schedule density meter - Focus mode highlighting current/next event

Agenda View: - List-based chronological display - Grouped by date - Completed/upcoming separation - Search and filter capabilities - Print-friendly format

Custom Views: - Premium: Custom date ranges (2-week, 3-day, etc.) - Premium: Timeline view for project planning - Premium: Year overview with major events

Event Management Event Properties:

```
{
  id: "uuid",
  title: "Event Title",
  description: "Detailed description with rich text",
  startDateTime: "2026-01-20T09:00:00Z",
  endDateTime: "2026-01-20T10:00:00Z",
  allDay: false,
  category: "work", // work, personal, health, social, etc.
  color: "#3B82F6",
  location: {
    type: "physical|virtual",
    address: "123 Main St",
    coordinates: { lat: 40.7128, lng: -74.0060 },
    virtualLink: "https://zoom.us/j/123456789"
  },
  recurrence: {
    frequency: "daily|weekly|monthly|yearly",
    interval: 1,
    daysOfWeek: [1, 3, 5], // Mon, Wed, Fri
    endType: "after|until|never",
    endCount: 10,
    endDate: "2026-12-31"
  },
  reminders: [
    { type: "notification", minutes: 30 },
    { type: "email", minutes: 1440 } // 24 hours
  ],
  attachments: ["fileId1", "fileId2"],
  linkedTasks: ["taskId1", "taskId2"],
  linkedGoals: ["goalId1"],
  attendees: [], // Future: collaboration feature
  status: "scheduled|completed|cancelled",
  visibility: "private|public",
  timeZone: "America/New_York"
}
```

Event Types: - Standard Events: Regular calendar entries - All-Day Events: No specific time slots - Multi-Day Events: Spanning multiple consecutive days - Recurring Events: Repeating patterns - Blocked Time: Reserved slots (no specific event) - Travel Time: Auto-calculated buffer between events

Smart Features Time Blocking Assistant: - AI suggests optimal time blocks based on: - Historical productivity patterns - Task priorities and deadlines - Energy levels (morning person vs. night owl)

- Calendar density and gaps - Drag tasks from task list to calendar for scheduling - Visual feedback on overcommitment

Conflict Detection: - Automatic overlap identification - Warning before creating conflicting events - Suggested alternative time slots - Option to override for intentional overlaps

Smart Suggestions: - Best time to schedule based on availability - Travel time recommendations between locations - Buffer time suggestions between meetings - Focus time block recommendations (2-4 hour chunks)

Natural Language Input: - Team meeting tomorrow at 2pm for 1 hour - Lunch with Sarah next Tuesday - Gym every Monday, Wednesday, Friday at 6am - Parses and creates events automatically

Integrations External Calendar Sync (Premium): - Google Calendar (bidirectional sync) - Apple Calendar (iCloud) - Microsoft Outlook/Office 365 - Real-time updates - Conflict resolution for dual edits

Internal Integrations: - Tasks: Convert tasks to scheduled events - Goals: Link events to goal progress - Time Tracking: Auto-start timers from events - Finance: Link events to expenses (meals, travel) - Habits: Schedule recurring habit reminders

2.1.3 User Flows

Create Event Flow:

```
1. User clicks time slot or "+ New Event"
2. Quick add modal appears
3. Enter title (required)
4. Select date/time (pre-filled if clicked slot)
5. Optional: Add details (location, description, category)
6. Optional: Set recurrence pattern
7. Optional: Add reminders
8. Click "Save"
9. Event appears on calendar with confirmation
10. Smart suggestions appear if conflicts detected
```

Time Blocking Flow:

```
1. User opens task list sidebar
2. Drags task onto calendar view
3. System suggests optimal time slot (highlighted)
4. User drops task in preferred slot
5. Event auto-created with task details
6. Task marked as "scheduled"
7. Link established between task and event
```

2.1.4 UI/UX Specifications

Color System: - 12 pre-defined category colors - Custom color picker for advanced users - Accessibility-compliant contrast ratios - Color-blind friendly palette option

Responsive Design: - Desktop: Side-by-side month + week view - Tablet: Single view with swipe navigation - Mobile: Optimized day/agenda views, simplified month view

Interactions: - Click: Select/view event details - Double-click: Edit event - Drag: Move event to different time - Drag edge: Adjust duration - Right-click: Context menu (duplicate, delete, share) - Hover: Quick preview tooltip

Performance: - Load 3 months of events instantly (<500ms) - Smooth scrolling through years of data - Efficient rendering for 1000+ events - Background sync without UI blocking

2.1.5 Technical Requirements

Data Storage: - Local caching for offline access - Sync queue for offline changes - Optimistic UI updates - Conflict resolution algorithm

Notifications: - In-app notifications - Browser push notifications (web) - Mobile push notifications (iOS/Android) - Email reminders (optional) - SMS reminders (Premium)

Search & Filters: - Full-text search across all events - Filter by category, date range, status - Search by location or attendee - Saved filter presets

---

2.2 Finance Manager

2.2.1 Overview

Purpose: Personal finance tracking with notification-based transaction logging, budget management, and spending analytics.

Target Users: Career professionals, solopreneurs, wellness-focused users (65% expected adoption)

Tier: PREMIUM ($6/month standalone or included in bundles)

Key Differentiators: - Quick-entry notification system - Visual spending analytics with charts - Budget alerts and overspending warnings - Category-based expense tracking - Income tracking with multiple sources - Financial goal integration - Manual entry + optional bank sync

2.2.2 Core Features

Transaction Management Transaction Entry Methods:

1. Quick Add (Notification-Based): - Floating notification appears at scheduled times (e.g., 8pm daily) - Did you spend money today? - Quick amount entry: $25 - Category quick-select buttons - Optional note field - Save in <10 seconds - Dismissable if no transactions

2. Manual Entry:

```
{
  id: "uuid",
  type: "expense|income|transfer",
  amount: 45.99,
  currency: "USD",
  category: "groceries",
  subcategory: "food",
```

```
  date: "2026-01-19",
  time: "14:30",
  merchant: "Whole Foods",
  paymentMethod: "credit_card_visa",
  description: "Weekly grocery shopping",
  tags: ["essential", "weekly"],
  receipt: "fileId",
  recurring: false,
  linkedEvent: "calendarEventId", // e.g., business lunch
  linkedBudget: "budgetId",
  location: { lat: 40.7128, lng: -74.0060 },
  verified: true
}
```

3. Bank Sync (Premium Plus): - Connect bank accounts via Plaid/Yodlee - Automatic transaction import - Category auto-suggestion based on ML - Merchant name normalization - Duplicate detection - Manual review and categorization

Transaction Types: - Expenses: Money out - Income: Money in - Transfers: Between accounts (not counted in budget) - Refunds: Reversal of expense - Adjustments: Corrections

Categories & Budgets Default Categories:

```
Essential:
  Housing (Rent/Mortgage, Utilities, Insurance)
  Food (Groceries, Dining Out, Coffee)
  Transportation (Gas, Public Transit, Parking, Car Payments)
  Healthcare (Insurance, Medications, Doctor Visits)
  Debt Payments (Credit Cards, Loans)

Lifestyle:
  Shopping (Clothing, Electronics, Home Goods)
  Entertainment (Streaming, Movies, Events, Hobbies)
  Travel (Flights, Hotels, Vacation)
  Personal Care (Gym, Salon, Spa)
  Education (Courses, Books, Subscriptions)

Financial:
  Savings (Emergency Fund, Retirement, Investments)
  Gifts & Donations
  Miscellaneous
```

Budget Management: - Set monthly budgets per category - Track spending vs. budget in real-time - Visual progress bars (green/yellow/red) - Alerts at 50%, 75%, 90%, 100% thresholds - Rollover option for unused budget - Auto-adjust suggestions based on patterns

Budget Types: - Fixed Budget: Same amount monthly (e.g., $500 groceries) - Percentage Budget: % of income (e.g., 30% housing) - Goal-Based: Save toward specific amount - Variable Budget: Seasonal adjustments

Financial Analytics Dashboard Widgets: - Monthly Summary: - Total income - Total expenses - Net savings - Savings rate % - Comparison to previous month

- Spending Breakdown:
  - Pie chart by category
  - Top 5 expenses
  - Spending trends (up/down arrows)
  - Unusual spending alerts
- Budget Status:
  - All budgets with progress bars
  - Over/under budget highlights
  - Projected end-of-month status
- Cash Flow:
  - Line graph showing income vs. expenses over time
  - Rolling 3/6/12 month view
  - Identify spending patterns

Advanced Analytics (Premium): - Spending Trends: Year-over-year comparisons - Category Deep Dive: Merchant-level analysis per category - Time Analysis: Day of week/time of day spending patterns - Comparative Metrics: Spending vs. similar users (anonymized) - Predictive Forecast: Projected expenses based on history - Tax Estimates: Income tax projection for freelancers

Reports: - Monthly expense report (PDF export) - Category breakdown report - Income summary report - Tax preparation report (income/deductions) - Custom date range reports

Account Management Account Types: - Cash: Physical cash tracking - Checking: Bank checking account - Savings: Savings accounts - Credit Card: Track balances and payments - Investment: Portfolio value (read-only or manual) - Loan: Track principal and payments

Account Features: - Multiple accounts support - Starting balance - Current balance (auto-calculated) - Manual adjustments - Account transfers - Interest tracking - Credit limit monitoring

2.2.3 Notification System

Daily Reminder: - Appears at user-specified time - Log todays transactions - Quick-add interface - Smart suggestions based on recurring patterns - Snooze option (remind in 1 hour)

Budget Alerts: - Near limit warnings (75%, 90%) - Over-budget notifications - Large transaction alerts (user-defined threshold) - Unusual spending pattern detection

Smart Notifications: - You havent logged transactions in 3 days - Recurring subscription charged: Netflix $15.99 - Projected to exceed dining budget by $150 this month - Great job! 25% under budget in Shopping

2.2.4 User Flows

Quick Transaction Entry (Primary Flow):

```
1. Notification appears: "Log today's spending?"
2. User taps notification
3. Quick-add modal opens
4. Enter amount: "25.50"
```

```
5. Tap category: "Groceries"
6. Optional: Add note
7. Tap "Save"
8. Confirmation: "Transaction saved"
9. Dashboard updates in real-time
```

Budget Setup Flow:

```
1. Navigate to Finance > Budgets
2. Click "+ Create Budget"
3. Select category
4. Enter monthly amount
5. Choose budget type (fixed/percentage)
6. Set alert thresholds
7. Save
8. Budget tracking begins immediately
```

2.2.5 UI/UX Specifications

Color Coding: - Green: Under budget, income, positive trends - Yellow: Approaching limit (75-90%) - Red: Over budget, overdue payments, negative trends - Blue: Neutral, informational - Purple: Savings, investments

Charts & Visualizations: - Pie charts for category distribution - Line graphs for trends over time - Bar charts for month-to-month comparison - Progress bars for budgets - Heatmaps for spending intensity by day

Mobile Optimization: - Large touch targets for quick entry - Swipe gestures for category selection - Bottom sheet for quick add - Offline-capable with sync queue

2.2.6 Data Security & Privacy

Security Measures: - End-to-end encryption for financial data - Biometric authentication option - Auto-lock after inactivity - Secure bank connection via Plaid (PCI-DSS compliant) - No storage of bank credentials - Optional data export/backup

Privacy: - Financial data never used for ads - Aggregated anonymized analytics only with consent - GDPR/CCPA compliant - One-click data deletion

---

2.3 Task Management & To-Do Lists

2.3.1 Overview

Purpose: Comprehensive task and project management for personal and professional productivity.

Target Users: All users (95% expected adoption)

Tier: FREE (Core Feature)

Key Differentiators: - Hierarchical task organization (tasks, subtasks, checklists) - Multiple views (list, board, calendar) - Priority and deadline management - Quick capture from anywhere - Powerful filtering and search - Recurring tasks - Integration with calendar and goals

2.3.2 Core Features

Task Structure Task Properties:

```
{
  id: "uuid",
  title: "Complete project proposal",
  description: "Draft and finalize Q2 project proposal for client meeting",
  status: "not_started|in_progress|completed|cancelled",
  priority: "low|medium|high|urgent",
  dueDate: "2026-01-25",
  dueTime: "17:00", // optional
  createdDate: "2026-01-19",
  completedDate: null,
  estimatedDuration: 120, // minutes
  actualDuration: null,

  // Organization
  project: "projectId", // group related tasks
  tags: ["work", "client", "Q2"],
  category: "work",

  // Hierarchy
  parentTask: null, // null if top-level
  subtasks: ["subtaskId1", "subtaskId2"],
  checklist: [
    { id: "1", text: "Research competitors", completed: true },
    { id: "2", text: "Draft outline", completed: false }
  ],

  // Relationships
  linkedGoal: "goalId",
  linkedEvent: "calendarEventId",
  dependencies: ["taskId3"], // can't start until these complete

  // Recurrence
  recurring: {
    frequency: "daily|weekly|monthly",
    interval: 1,
    daysOfWeek: [1, 5], // Mon, Fri
    endDate: "2026-12-31"
  },

  // Reminders
  reminders: [
    { type: "notification", triggerAt: "2026-01-25T09:00:00Z" }
  ],
```

```
  // Metadata
  attachments: ["fileId1"],
  notes: "Additional context",
  effortLevel: "easy|moderate|hard",
  context: "home|office|phone|errands",
  energyRequired: "low|medium|high"
}
```

Task Views List View (Default): - Grouped by: Today, Upcoming, Someday, Completed - Sort by: Priority, Due date, Created date, Manual - Inline editing - Drag to reorder - Expand/collapse subtasks - Quick actions (complete, edit, delete)

Board View (Kanban): - Columns: To Do, In Progress, Completed - Custom columns (Premium) - Drag-and-drop between columns - Card-based interface - Swim lanes by project/priority

Calendar View: - Tasks plotted on calendar by due date - Drag to reschedule - Color-coded by priority/project - Integration with events

Project View: - Group tasks by project - Project progress percentage - Collapse/expand projects - Project-level metadata

Context View (Premium): - Group by context tags (@home, @office, @phone) - Location-based suggestions - Energy level filtering

Smart Task Management Quick Capture: - Global keyboard shortcut (Ctrl/Cmd + K) - Floating + button always visible - Voice input (mobile) - Natural language parsing: Call dentist tomorrow at 2pm #health !high - Email-to-task (Premium)

Smart Scheduling: - AI suggests optimal time to work on task - Considers: deadline, duration estimate, calendar availability, energy patterns - One-click schedule to calendar - Auto-reschedule if not completed

Prioritization Engine: - Eisenhower Matrix view (Urgent/Important grid) - Auto-priority suggestion based on: - Deadline proximity - Linked goal importance - Dependencies - User override always possible

Batch Operations: - Multi-select tasks - Bulk edit (change project, priority, due date) - Bulk complete/delete - Bulk reschedule

Recurring Tasks Patterns Supported: - Daily (every day, every X days, weekdays only) - Weekly (specific days, every X weeks) - Monthly (specific date, last day, first Monday, etc.) - Yearly (anniversary, birthday) - Custom (advanced patterns)

Behavior: - New instance created on completion - Or auto-created on due date - Skip weekends option - Pause/resume recurring series

2.3.3 Productivity Features

Focus Mode: - Hide completed tasks - Show only todays tasks - Distraction-free interface - Timer integration for current task

Task Templates (Premium): - Pre-defined task sets - Examples: Morning Routine, Weekly Review, Project Kickoff - Checklist templates - One-click instantiation

Task Insights: - Completion rate trends - Average time to complete tasks - Most productive hours/days - Procrastination patterns (tasks repeatedly rescheduled)

2.3.4 User Flows

Quick Task Creation:

```
1. Press Ctrl/Cmd + K anywhere
2. Type task title: "Buy groceries"
3. Optional: Add due date with natural language "tomorrow"
4. Optional: Add priority "!high"
5. Press Enter
6. Task created and appears in list
```

Task Completion:

```
1. Click checkbox next to task
2. Task marked complete with animation
3. Moved to "Completed" section
4. Linked time tracking stops (if running)
5. Goal progress updates
6. If recurring, next instance created
```

2.4 Habit Tracker

2.4.1 Overview

Purpose: Build and maintain positive habits through systematic tracking, streaks, and behavioral insights.

Target Users: Wellness-focused users, career professionals (55% expected adoption)

Tier: PREMIUM ($4/month)

Key Differentiators: - Visual streak tracking with motivational design - Flexible frequency (daily, weekly, custom) - Habit stacking suggestions - Reminder system - Analytics on habit formation - Integration with health tracker and journal

2.4.2 Core Features

Habit Configuration Habit Structure:

```
{
  id: "uuid",
  name: "Morning meditation",
  description: "10 minutes of mindfulness",
  icon: "meditation",
  color: "#8B5CF6",
```

```
  // Frequency
  frequency: {
    type: "daily|weekly|monthly|custom",
    target: 7, // times per week for weekly, or specific days for daily
    specificDays: [1, 2, 3, 4, 5], // Mon-Fri
    timeOfDay: "morning|afternoon|evening|anytime",
    preferredTime: "07:00" // suggested time
  },

  // Tracking
  trackingType: "checkbox|counter|duration|value",
  unit: "minutes", // for value tracking
  targetValue: 10, // for measurable habits

  // History
  completions: [
    { date: "2026-01-19", completed: true, value: 12, note: "Felt great" }
  ],

  // Streaks
  currentStreak: 15,
  longestStreak: 30,
  totalCompletions: 145,

  // Reminders
  reminders: [
    { time: "07:00", days: [1,2,3,4,5] }
  ],

  // Organization
  category: "health|productivity|learning|social|other",
  linkedGoal: "goalId",
  habitStack: "habitStackId", // chain habits together

  // Metadata
  startDate: "2025-10-01",
  archived: false,
  difficulty: "easy|medium|hard"
}
```

Tracking Types 1. Checkbox (Yes/No): - Simple binary tracking - Did it or didnt - Examples: Exercise, Read, Meditate

2. Counter: - Count occurrences - Examples: Glasses of water (8/day), Push-ups (3 sets/day) - Target number to achieve

3. Duration: - Track time spent - Examples: Study (2 hours), Practice instrument (30 min) - Integration with time tracker

4. Value: - Measure specific metrics - Examples: Miles run, Pages read, Money saved - Custom units (km, pages, $, etc.)

Habit Views Daily View: - List of todays habits - Quick check-off interface - Progress circles for each habit - Motivational quotes - Streak indicators

Calendar View: - Heat map showing completion patterns - Color intensity = consistency - Click date to see that days habits - Identify gaps and trends

Analytics Dashboard: - Overall completion rate - Individual habit success rates - Streak visualization - Best/worst days of week - Monthly progress charts - Year-in-review summary

Habit Library: - All habits organized by category - Active vs. Archived - Quick access to edit/delete - Statistics per habit

2.4.3 Motivation & Gamification

Streaks: - Current streak prominently displayed - Longest streak record - Near-miss protection: Youre about to lose a 30-day streak! - Freeze days (Premium): 1 per month, skip without breaking streak

Achievements: - Milestone badges: 7-day, 30-day, 100-day, 365-day streaks - Perfect week/month badges - Multiple habit maintenance - Comeback badges (restart after break)

Progress Visualization: - Habit strength meter (0-100%) - Visual indicators of habit formation stages: - Seedling (0-7 days) - Growing (8-21 days) - Rooted (22-66 days) - Mastered (67+ days)

Social Features (Future): - Accountability partners - Habit challenges - Community support - Leaderboards (opt-in)

2.4.4 Smart Features

Habit Stacking: - Chain habits together: After morning coffee, do 10 push-ups - Visual workflow of stacked habits - All-or-nothing completion option - Template stacks for common routines

Smart Reminders: - Time-based notifications - Location-based (arrive home habit reminder) - Context-aware (phone idle for 10min meditation reminder) - Adaptive timing (learns best response times)

Insights & Recommendations: - You complete Exercise 85% more when done before 9am - Your meditation streak correlates with better sleep scores - Consider adding Reading to your Evening routine - Suggest new habits based on goals

Habit Pairing: - Identify existing habits - Suggest complementary new habits - Example: You exercise regularly. Try adding post-workout stretching

2.4.5 User Flows

Create Habit:

```
1. Click "+ New Habit"
2. Enter habit name: "Morning run"
3. Choose icon and color
4. Set frequency: "Daily, 5 days/week"
```

```
5. Select days: Mon-Fri
6. Set reminder: 6:30 AM
7. Choose tracking type: Duration
8. Set target: 30 minutes
9. Save
10. Habit added to today's list
```

Daily Check-In:

```
1. Morning reminder appears: "Time for your habits!"
2. Open habit tracker
3. See today's habits list
4. Tap checkbox for completed habits
5. For duration habits, tap to start timer
6. After completion, log value
7. See updated streak count
8. Motivational message: "15 day streak!  "
```

2.5 Goal Setting & Progress Monitoring

2.5.1 Overview

Purpose: Define, track, and achieve personal and professional goals with structured planning and progress visualization.

Target Users: Career professionals, solopreneurs (60% expected adoption)

Tier: PREMIUM ($5/month)

Key Differentiators: - SMART goal framework - Hierarchical goal structure (yearly quarterly monthly) - Multi-dimensional progress tracking - Visual progress indicators - Integration with tasks, habits, calendar, and time tracking - Milestone celebrations

2.5.2 Core Features

Goal Structure Goal Properties:

```
{
  id: "uuid",
  title: "Launch side business",
  description: "Build and launch online course platform",

  // SMART Framework
  specific: "Create and sell online courses about web development",
  measurable: {
    metric: "revenue",
    target: 10000,
    current: 2500,
    unit: "USD"
  },
```

```
achievable: "Leverage existing skills and audience",
relevant: "Aligns with career growth and financial independence",
timeBound: {
  startDate: "2026-01-01",
  targetDate: "2026-12-31",
  deadline: "2026-12-31"
},

// Organization
category: "career|financial|health|learning|personal|social",
priority: "high",
status: "not_started|in_progress|completed|on_hold|cancelled",

// Hierarchy
parentGoal: "goalId", // long-term goal this contributes to
subGoals: ["subGoalId1", "subGoalId2"],

// Milestones
milestones: [
  {
    id: "m1",
    title: "Complete course curriculum",
    targetDate: "2026-03-31",
    completed: true,
    completedDate: "2026-03-15"
  },
  {
    id: "m2",
    title: "Record first course",
    targetDate: "2026-06-30",
    completed: false
  }
],

// Progress Tracking
progress: {
  percentage: 25,
  lastUpdated: "2026-01-19",
  updateFrequency: "weekly",
  calculationType: "manual|automatic", // auto from linked items
},

// Connections
linkedTasks: ["taskId1", "taskId2"],
linkedHabits: ["habitId1"],
linkedEvents: ["eventId1"],
requiredResources: ["$500 equipment", "20 hours/week time"],
```

```
  // Tracking
  checkIns: [
    {
      date: "2026-01-15",
      progress: 20,
      note: "Completed curriculum outline",
      mood: "motivated"
    }
  ],

  // Metadata
  visibility: "private|shared",
  tags: ["entrepreneurship", "passive income"],
  motivationNotes: "Why this matters: Financial freedom and helping others",
  obstaclesAnticipated: ["Time management", "Marketing skills gap"]
}
```

Goal Types By Timeframe: - Long-term Goals: 1-5 years (e.g., Retire by 50) - Annual Goals: Yearly objectives (e.g., Earn $100K in 2026) - Quarterly Goals: 3-month targets (e.g., Launch MVP by Q2) - Monthly Goals: Short-term objectives (e.g., Lose 5 lbs this month)

By Type: - Outcome Goals: Specific end result (lose 20 lbs) - Process Goals: Habits to develop (exercise 4x/week) - Performance Goals: Measurable improvements (run 5K in <25 min)

By Category: - Career & Professional - Financial & Money - Health & Fitness - Learning & Education - Relationships & Social - Personal Development - Creative & Hobbies

Progress Tracking Automatic Progress Calculation: - Based on completed linked tasks - Based on habit consistency - Based on milestone completion - Formula: (completedItems / totalItems) Œ 100

Manual Progress Updates: - Weekly/monthly check-ins - Slider to adjust percentage - Required notes explaining change - Photo/file uploads as evidence

Progress Visualization: - Circular progress rings - Linear progress bars - Line graph over time - Comparison to projected progress - Behind/ahead indicators

2.5.3 Goal Planning Features

Goal Wizard (Setup Assistant):

```
Step 1: What do you want to achieve?
  → Text input with examples

Step 2: Make it SMART
  → Specific: [refine statement]
  → Measurable: [define metric and target]
  → Achievable: [assess feasibility]
  → Relevant: [connect to values]
  → Time-bound: [set deadline]
```

```
Step 3: Break it down
  → Suggest 3-5 milestones
  → Create timeline
  → Identify required tasks

Step 4: Set up tracking
  → Choose update frequency
  → Enable linked task auto-progress
  → Set reminder schedule

Step 5: Stay motivated
  → Add motivational note
  → Set celebration rewards
  → Identify accountability partner
```

Goal Templates: - Financial Goals (Save $X, Pay off debt, Earn income) - Fitness Goals (Lose weight, Run distance, Build strength) - Career Goals (Promotion, Skill certification, Side business) - Learning Goals (Read X books, Complete course, Learn language)

Dependency Mapping: - Visualize goal relationships - Parent-child goal hierarchy - Task Milestone Goal chain - Critical path identification

2.5.4 Motivation & Accountability

Check-In System: - Scheduled reminders (weekly, bi-weekly, monthly) - Structured check-in form: - Progress update (percentage or metric) - What went well? - What challenges did you face? - What will you do next? - Mood/motivation level

Milestone Celebrations: - Pop-up animations on milestone completion - Achievement notifications - Progress share cards (social media ready) - Reflection prompts

Accountability Features: - Commitment contracts (self or with partner) - Public goal sharing (opt-in) - Progress reports (weekly email summaries) - Streak tracking for consistency

Obstacle Management: - Log obstacles as they occur - Solutions repository - Pattern recognition (recurring obstacles) - Pivot recommendations when stuck

2.5.5 Analytics & Insights

Goal Dashboard: - All goals overview - Progress at a glance - Upcoming milestones - Goals needing attention (no recent updates) - Completion rate stats

Time Analysis: - Time invested toward goal (from time tracking) - Projected completion date based on current pace - Burndown chart - Effort vs. progress correlation

Success Patterns: - Goals completed vs. abandoned - Optimal goal size (completion rates by scope) - Best categories (where you succeed most) - Seasonality trends

Predictive Insights: - At current pace, youll reach this goal by [date] - Youre 2 weeks behind schedule. Increase effort by 30% to get back on track - Similar goals took you average 3 months. This is on pace

2.5.6 User Flows

Create Goal:

```
1. Navigate to Goals
2. Click "+ New Goal"
3. Enter goal title
4. Launch Goal Wizard
5. Complete SMART framework
6. Break into milestones
7. Link existing tasks/habits
8. Set check-in frequency
9. Save goal
10. Goal appears on dashboard with 0% progress
```

Weekly Check-In:

```
1. Reminder: "Time for your weekly goal check-in"
2. Open check-in form
3. See current progress: 35%
4. Review linked completed tasks (auto-calculated progress bump)
5. Manual adjustment if needed
6. Answer reflection questions
7. Submit
8. See updated progress visualization
9. Motivational message based on progress
```

2.6 Time Tracking & Analytics

2.6.1 Overview

Purpose: Track time spent on activities, projects, and tasks to understand productivity patterns and optimize time allocation.

Target Users: Career professionals, solopreneurs, students (70% expected adoption)

Tier: PREMIUM ($5/month)

Key Differentiators: - One-click start/stop timers - Automatic tracking with AI categorization - Manual time entry for retrospective logging - Detailed analytics and reports - Billable hours tracking for freelancers - Integration with tasks, projects, and calendar - Idle time detection

2.6.2 Core Features

Timer Functionality Timer Types:

1. Active Timer (Stopwatch):

```
{
  id: "uuid",
  type: "active",
  activity: "Website redesign",
```

```
  project: "projectId",
  task: "taskId",
  startTime: "2026-01-19T14:30:00Z",
  endTime: null, // null while running
  duration: 3600, // seconds (updated live)

  billable: true,
  hourlyRate: 75.00,
  tags: ["client work", "design"],

  pauses: [
    { pausedAt: "2026-01-19T15:00:00Z", resumedAt: "2026-01-19T15:15:00Z" }
  ],

  notes: "Working on homepage mockup",
  automaticallyStopped: false,
  idleTime: 180 // seconds of detected idle time
}
```

2. Manual Entry: - Retrospective time logging - Specify: activity, start time, end time, duration - Bulk entry for past day/week - Import from calendar events

3. Automatic Tracking (Premium Plus): - Desktop app monitors active application - Categorizes time automatically - Runs in background - Privacy controls (pause tracking, exclude apps)

Time Entry Management Quick Actions: - Start timer with one click - Floating widget showing current timer - Global keyboard shortcut to start/stop - Quick-switch between recent activities - Timer continues in background (web, mobile)

Bulk Operations: - Duplicate time entries - Split entries (break one into multiple) - Merge entries (combine similar activities) - Delete/archive old entries

Smart Features: - Pomodoro integration (25min work, 5min break) - Idle time detection with prompt: Keep 10min idle time or discard? - Suggested activities based on calendar - Auto-stop reminder if timer runs >8 hours

2.6.3 Analytics & Reporting

Dashboard Widgets:

Today Summary: - Total time tracked today - Breakdown by project/activity - Comparison to daily average - Productivity score (focused vs. distracted time)

This Week: - Total hours tracked - Daily breakdown (bar chart) - Most time-consuming projects - Billable vs. non-billable ratio

Trends: - Weekly/monthly time trends - Category distribution (pie chart) - Peak productivity hours (heat map) - Day-of-week patterns

Detailed Reports:

Time Sheet Report: - Date range selector - Grouped by: Day, Project, Task, Activity - Columns: Start, End, Duration, Notes, Billable - Export: CSV, PDF, Excel - Print-friendly format

Project Report: - All time entries for a project - Total time invested - Estimated vs. actual time - Team member breakdown (future: collaboration) - Profitability (if billable)

Activity Report: - Time spent per activity type - Trends over time - Efficiency metrics - Recommendations for optimization

Productivity Report: - Focused work time vs. distractions - Productive hours per day - Context switching frequency - Meeting time overhead - Deep work blocks identified

2.6.4 Billable Hours & Invoicing

Billable Configuration: - Mark projects/activities as billable - Set hourly rates (per project or global) - Different rates for different task types - Automatic calculation of earnings

Invoicing Integration (Premium): - Generate invoices from time entries - Group by client/project/date range - Customizable invoice templates - Include detailed time breakdown - Tax calculation - Send directly to client - Payment tracking

Client Management: - Multiple clients - Client-specific rates - Retainer tracking (hours purchased) - Budget alerts (approaching client budget limit)

2.6.5 Integrations

Internal: - Tasks: Start timer from task, auto-link time to task - Calendar: Import calendar events as time entries - Goals: Track time invested toward goals - Projects: Aggregate time at project level - Finance: Billable time income transactions

External (Premium): - Calendar Sync: Import events from Google/Outlook - Project Tools: Export to Jira, Asana, Trello - Accounting: Export to QuickBooks, FreshBooks - Toggl/Harvest Import: Migrate existing data

2.6.6 Productivity Insights

AI-Powered Recommendations: - Youre most productive 9-11am. Schedule deep work then. - Meetings take up 40% of your time. Consider consolidating. - You context-switch every 15min on average. Try time-blocking. - Your average focus session is 35min. Ideal for Pomodoro technique.

Pattern Recognition: - Identify procrastination (tasks repeatedly started/stopped) - Optimal work session length - Best days/times for specific activity types - Correlation with mood, energy levels (if using health tracker)

Goals Integration: - Youve invested 15 hours toward Learn Python goal this month - Progress visualization: time spent goal achievement - Pace tracking: Need 5 more hours this week to stay on track

2.6.7 User Flows

Start Timer (Primary Flow):

```
1. Click floating "Start Timer" button
2. Modal appears with recent activities
```

```
3. Select activity or type new one
4. Optional: Link to task/project
5. Optional: Mark as billable
6. Click "Start"
7. Timer begins, widget shows elapsed time
8. Minimized widget in corner
9. Click "Stop" when done
10. Entry saved, added to today's total
```

Review Days Time:

```
1. Navigate to Time Tracking
2. See today's timeline visualization
3. Hover over blocks to see details
4. Identify gaps or anomalies
5. Click gap to add manual entry
6. See summary stats at top
7. Compare to previous days
```

2.7 Note Taking & Documentation

2.7.1 Overview

Purpose: Capture thoughts, ideas, and information with a flexible, organized note-taking system.

Target Users: All users (80% expected adoption)

Tier: FREE (Core Feature)

Key Differentiators: - Rich text editor with markdown support - Organizational hierarchy (notebooks notes) - Quick capture from anywhere - Search and tagging - Media embedding - Templates - Version history (Premium)

2.7.2 Core Features

Note Structure:

```
{
  id: "uuid",
  title: "Meeting Notes - Q1 Planning",
  content: "<rich text HTML or markdown>",
  contentType: "richtext|markdown|checklist",

  // Organization
  notebook: "notebookId",
  tags: ["meeting", "planning", "Q1"],
  category: "work",

  // Metadata
  createdDate: "2026-01-19T10:30:00Z",
```

```
  modifiedDate: "2026-01-19T14:45:00Z",
  author: "userId",

  // Features
  pinned: true,
  starred: false,
  archived: false,
  encrypted: false, // Premium

  // Attachments
  attachments: ["fileId1", "fileId2"],
  images: ["imageId1"],

  // Connections
  linkedTasks: ["taskId1"],
  linkedEvents: ["eventId1"],
  linkedNotes: ["noteId2"], // backlinks

  // Sharing (future)
  shared: false,
  permissions: [],

  // Version control
  versionHistory: [] // Premium
}
```

Editor Features: - Formatting: Bold, italic, underline, strikethrough - Lists: Bulleted, numbered, checklists - Headings: H1-H6 - Links: URLs, internal note links - Code: Inline code, code blocks with syntax highlighting - Tables: Create and edit tables - Images: Drag-drop, paste, upload - Embeds: YouTube videos, tweets, maps - Mentions: @task, @event references (Premium)

Organization: - Notebooks: Top-level containers - Sections: Within notebooks (Premium) - Tags: Cross-cutting categorization - Favorites: Pin important notes - Recently Viewed: Quick access to recent notes

2.7.3 Note Templates

Built-in Templates: - Meeting Notes (agenda, attendees, action items) - Daily Journal (date, mood, highlights, reflections) - Project Plan (goals, timeline, resources, risks) - Book Summary (title, author, key points, quotes) - Decision Log (context, options, decision, reasoning) - Learning Notes (concept, examples, practice questions)

Custom Templates (Premium): - Create your own - Reusable structure - Variables and placeholders

2.7.4 Search & Discovery

Full-Text Search: - Search across all note content - Filter by notebook, tag, date - Search within attachments (Premium) - Keyboard shortcut (Cmd/Ctrl + P)

Smart Suggestions: - Related notes based on content - Suggested tags - Duplicate detection -

Orphaned notes (no tags, no notebook)

2.7.5 Collaboration (Future/Premium)

- Share notes with read/write permissions
- Real-time collaborative editing
- Comments and discussion threads
- Change tracking

2.7.6 User Flow

Quick Note Creation:

```
1. Press Cmd/Ctrl + N anywhere
2. New note window opens
3. Start typing title
4. Press Tab, begin content
5. Auto-saves as you type
6. Click outside or Cmd+S to finalize
7. Choose notebook (or default "Quick Notes")
8. Add tags if desired
9. Note saved
```

2.8 Health & Wellness Tracker

2.8.1 Overview

Purpose: Monitor health metrics and wellness indicators to understand mind-body connections with productivity.

Target Users: Wellness-focused users (40% expected adoption)

Tier: PREMIUM ($4/month)

Key Differentiators: - Multiple health metric tracking - Integration with fitness devices (Apple Health, Google Fit) - Correlation with productivity metrics - Sleep tracking and analysis - Mood and energy logging - Insights on health-productivity relationships

2.8.2 Trackable Metrics

Physical Health: - Weight and body composition - Sleep duration and quality - Exercise (type, duration, intensity) - Water intake - Nutrition (calories, macros) - Heart rate, blood pressure - Steps and movement

Mental Wellness: - Mood (scale or emoji) - Energy levels - Stress levels - Anxiety/depression indicators - Mindfulness minutes

Habits: - Caffeine consumption - Alcohol intake - Screen time - Time outdoors

2.8.3 Data Entry

Methods: - Manual daily logging - Import from Apple Health - Import from Google Fit - Import from Fitbit, Garmin, etc. - Smart scale integration - Photo journaling

Quick Log Interface: - Daily check-in form (2-3 minutes) - Most important metrics prioritized - Historical data visible for context - Streaks and trends shown

2.8.4 Analytics

Health Dashboard: - Key metrics overview - Trends over time - Averages and ranges - Correlation insights

Productivity Correlation: - On days you sleep >7hrs, your productivity is 25% higher - Exercise correlates with better mood scores - High caffeine days show productivity spike then crash

Reports: - Weekly health summary - Monthly trends - Export data for doctor visits

---

2.9 Focus Timer (Pomodoro)

2.9.1 Overview

Purpose: Structured focus sessions using Pomodoro Technique and custom intervals to enhance concentration.

Target Users: Students, career professionals (45% expected adoption)

Tier: PREMIUM ($3/month)

Features: - Customizable work/break intervals - Task integration (what are you working on?) - Distraction logging - Session statistics - Ambient sound options - Do Not Disturb mode integration

Pomodoro Workflow:

```
1. Choose task to work on
2. Start 25-minute focus timer
3. Work without interruptions
4. Timer ends, 5-minute break starts
5. After 4 pomodoros, 15-30 minute long break
6. Repeat cycle
7. Track completed pomodoros per task
```

Customization: - Custom intervals (25/5 is default, adjust to preference) - Auto-start next phase or manual - Skip break option - Extended break after X sessions - Different profiles (deep work, light work, study)

Integration: - Links to time tracking - Counts toward task time estimates - Productivity analytics

---

2.10 Personal Dashboard & Analytics

2.10.1 Overview

Purpose: Unified view of all productivity metrics with AI-powered insights and recommendations.

Target Users: Power users (35% expected adoption)

Tier: PREMIUM ($8/month)

Features:

Dashboard Widgets: - Todays summary (tasks, events, time, habits) - Week at a glance - Goal progress overview - Finance snapshot - Health metrics - Productivity score (calculated from all features)

Customization: - Drag-drop widget arrangement - Show/hide widgets - Custom date ranges - Dark/light theme

AI Insights: - Daily briefing: What to focus on today - Productivity patterns identified - Recommendations for improvement - Anomaly detection (unusual patterns) - Predictive scheduling

Reports: - Weekly productivity report (email) - Monthly achievements summary - Quarterly goal review - Annual retrospective

---

2.11 File Manager & Cloud Storage

2.11.1 Overview

Purpose: Centralized file storage and organization for productivity-related documents.

Target Users: Solopreneurs, students (30% expected adoption)

Tier: PREMIUM ($5/month includes 10GB, additional storage available)

Features: - Cloud file storage - Folder organization - File linking to tasks, notes, events - Version history - Sharing and permissions - Search within documents - Preview for common formats

Storage Tiers: - Free: 1GB - Premium: 10GB - Premium Plus: 100GB - Business: 1TB

2.12 Daily Journal & Mood Tracker

2.12.1 Overview

Purpose: Reflective journaling with mood tracking for mental wellness and self-awareness.

Target Users: Wellness-focused users (40% expected adoption)

Tier: PREMIUM ($4/month)

Features:

Journal Entries: - Daily prompts (What went well? What could be better?) - Gratitude logging (3 things youre grateful for) - Wins and lessons - Tomorrows intentions - Free-form writing - Photo attachments - Weather and location auto-logged

Mood Tracking: - Multiple moods per day - Intensity scale (1-10) - Mood categories: Happy, Sad, Anxious, Energetic, Calm, Stressed, etc. - Context tagging (what influenced mood) - Visual mood calendar (heat map)

Insights: - Mood patterns over time - Correlation with activities, sleep, habits - Trigger identification - Sentiment analysis of journal text - Gratitude trends

Prompts & Exercises: - Daily reflection questions - Mindfulness exercises - CBT-inspired thought records - Goal alignment check-ins

---

Summary & Feature Matrix

Free vs. Premium Feature Allocation

FREE Tier (3 Features): 1. Task Management - Essential for all users 2. Calendar & Events - Core productivity need 3. Note Taking - Universal utility

Premium Features (Choose à la carte): 4. Finance Manager - $6/month 5. Habit Tracker - $4/month 6. Goal Setting - $5/month 7. Time Tracking - $5/month 8. Health Tracker - $4/month 9. Focus Timer - $3/month 10. Personal Dashboard - $8/month 11. File Manager - $5/month 12. Journal & Mood - $4/month

Bundle Options: - Productivity Bundle: Tasks, Calendar, Notes, Goals, Time Tracking - $15/month (save 35%) - Wellness Bundle: Habits, Health, Journal, Focus Timer - $12/month (save 30%) - Complete Individual: All 12 features - $25/month (save 50%)

Next Chapter: Chapter 3: Enterprise Features

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Product Team - Status: Draft - Feature Specification Phase

Chapter 3: Enterprise Features

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Feature Specification Phase

Table of Contents

- 3.1 Inventory Management System
- 3.2 Order Management System
- 3.3 Finance & Accounting Suite
- 3.4 Employee Time & Attendance Tracking
- 3.5 Project Management & Collaboration
- 3.6 Customer Relationship Management (CRM)

- 3.7 Invoice & Billing Management
- 3.8 Team Communication Hub
- 3.9 Document Management System
- 3.10 Analytics & Reporting Dashboard
- 3.11 Resource Allocation & Planning
- 3.12 Vendor & Supplier Management

Enterprise Feature Priority Matrix

Feature	Tier	Free/Premium	Priority	MVP
Project Management	Core	Free	P0	
Team Communication	Core	Free	P0	
Time & Attendance	Core	Free	P0	
Finance & Accounting	High Value	Premium	P1	
Order Management	High Value	Premium	P1	
CRM	High Value	Premium	P1	
Invoice & Billing	High Value	Premium	P2	
Analytics Dashboard	High Value	Premium	P2	
Inventory Management	Specialized	Premium	P3	
Document Management	Specialized	Premium	P3	
Resource Planning	Specialized	Premium	P3	
Vendor Management	Specialized	Premium	P3	

3.1 Inventory Management System

3.1.1 Overview

Purpose: Track inventory levels, manage stock across multiple locations, automate reordering, and prevent stockouts for product-based businesses.

Target Users: Retail businesses, e-commerce, manufacturing, warehouses (45% of enterprise users)

Tier: PREMIUM ($75/month base + $2/user)

Key Differentiators: - Multi-location inventory tracking - Real-time stock level monitoring - Automated low-stock alerts - Barcode/QR code scanning - Batch and lot tracking - FIFO/LIFO/FEFO cost methods - Integration with order management and accounting

3.1.2 Core Features

Product Management Product Schema:

```
{
  id: "uuid",
  sku: "PROD-001",
  name: "Wireless Mouse",
```

```
description: "Ergonomic wireless mouse with USB receiver",

// Classification
category: "Electronics",
subcategory: "Computer Accessories",
brand: "TechBrand",
tags: ["wireless", "ergonomic", "office"],

// Variants
hasVariants: true,
variants: [
  {
    id: "var1",
    sku: "PROD-001-BLK",
    attributes: { color: "Black", size: "Standard" },
    barcode: "123456789012",
    stockLevel: 150
  },
  {
    id: "var2",
    sku: "PROD-001-WHT",
    attributes: { color: "White", size: "Standard" },
    barcode: "123456789013",
    stockLevel: 80
  }
],

// Inventory Tracking
trackInventory: true,
inventoryMethod: "FIFO|LIFO|FEFO|Average",
stockLevel: 230, // total across all variants/locations
reservedStock: 15, // allocated to pending orders
availableStock: 215,
reorderPoint: 50,
reorderQuantity: 100,
safetyStock: 25,

// Multi-location
stockByLocation: [
  { locationId: "warehouse1", quantity: 150 },
  { locationId: "store1", quantity: 80 }
],

// Batches/Lots
batchTracking: true,
batches: [
  {
    batchNumber: "BATCH-2026-001",
```

```
      quantity: 100,
      manufacturingDate: "2026-01-01",
      expiryDate: "2028-01-01",
      location: "warehouse1",
      cost: 8.50
    }
  ],

  // Pricing
  costPrice: 8.50,
  sellingPrice: 24.99,
  wholesalePrice: 18.99,
  margin: 194%, // calculated
  currency: "USD",
  taxable: true,
  taxRate: 0.08,

  // Physical Properties
  weight: 0.15, // kg
  dimensions: { length: 10, width: 6, height: 4 }, // cm

  // Supplier Info
  primarySupplier: "supplierId",
  supplierSKU: "SUP-WM-001",
  leadTime: 14, // days
  moq: 50, // minimum order quantity

  // Images & Media
  images: ["imageId1", "imageId2"],
  documents: ["specSheet.pdf"],

  // Status
  status: "active|discontinued|out_of_stock",
  createdDate: "2025-06-01",
  lastRestocked: "2026-01-15",

  // Audit
  createdBy: "userId",
  lastModifiedBy: "userId",
  lastModifiedDate: "2026-01-19"
}
```

Stock Operations Stock Movements:

```
{
  id: "uuid",
  type: "adjustment|received|sold|transferred|returned|damaged|lost",
  productId: "productId",
```

```
  variantId: "variantId",
  batchNumber: "BATCH-2026-001",

  quantity: 50,
  quantityBefore: 180,
  quantityAfter: 230,

  fromLocation: "supplierId", // for received
  toLocation: "warehouse1",

  date: "2026-01-19",
  reason: "Purchase order #PO-1234 received",
  notes: "Checked and verified",

  relatedDocument: {
    type: "purchase_order|sale|transfer|adjustment",
    id: "documentId"
  },

  costPerUnit: 8.50,
  totalCost: 425.00,

  performedBy: "userId",
  approvedBy: "managerId",

  attachments: ["deliveryNote.pdf"]
}
```

Operation Types:

1. Stock Adjustment: Manual correction of inventory levels
2. Stock Receipt: Receiving inventory from suppliers
3. Stock Sale: Inventory sold to customers (auto from orders)
4. Stock Transfer: Moving between locations
5. Stock Return: Customer returns or supplier returns
6. Stock Damage/Loss: Write-offs and losses

Alerts & Automation Automated Alerts: - Low Stock Alert: When inventory drops below reorder point - Out of Stock: When quantity reaches zero - Overstock Alert: When inventory exceeds maximum threshold - Expiry Warning: Products approaching expiration (30/60/90 days) - Slow-Moving Stock: Items not sold in X days - Dead Stock: No movement in 180+ days

Auto-Reordering (Premium Plus): - Automatic purchase order generation when stock hits reorder point - Smart reorder quantity calculation based on: - Historical sales velocity - Lead time - Safety stock requirements - Seasonal trends - Send to supplier or require approval

Stock Forecasting: - Predict future stock needs based on: - Historical sales data - Seasonality patterns - Growth trends - Upcoming promotions - Recommend optimal stock levels

Multi-Location Management Locations: - Warehouses: Primary storage facilities - Retail Stores: Physical store locations - Consignment: Stock at partner locations - In-Transit: Stock being transferred

Location Features: - Real-time stock visibility across all locations - Transfer requests between locations - Location-specific reorder points - Stock allocation rules (which location fulfills orders) - Location performance analytics

Barcode & Scanning Supported Formats: - UPC/EAN barcodes - QR codes - Custom SKU codes

Scanning Functions: - Stock receiving (scan to add) - Stock counting (physical inventory) - Order fulfillment (scan to ship) - Product lookup - Mobile app barcode scanner

Physical Inventory: - Scheduled or ad-hoc stock counts - Count sheets generation - Variance reports (expected vs. actual) - Adjustment recommendations - Count history and accuracy tracking

3.1.3 Reports & Analytics

Inventory Reports: - Current Stock Report: Real-time inventory levels - Stock Valuation: Total inventory value (at cost) - Stock Movement Report: All transactions in date range - Reorder Report: Items needing reorder - Dead Stock Report: Non-moving inventory - ABC Analysis: Categorize inventory by value/velocity - Stock Aging Report: How long items have been in stock

Performance Metrics: - Inventory turnover ratio - Days inventory outstanding - Stock accuracy rate - Fill rate (demand met from stock) - Carrying cost of inventory

---

3.2 Order Management System

3.2.1 Overview

Purpose: End-to-end order processing from creation to fulfillment, integrated with inventory and customer management.

Target Users: E-commerce, retail, wholesale, B2B (70% of enterprise users)

Tier: PREMIUM ($60/month base + $1.50/user)

Key Differentiators: - Multi-channel order consolidation - Automated fulfillment workflows - Real-time order tracking - Integration with shipping carriers - Returns and refunds management - B2B and wholesale order support

3.2.2 Core Features

Order Schema

```
{
  id: "uuid",
  orderNumber: "ORD-2026-00123",
  type: "sales_order|purchase_order|transfer_order",

  // Customer Info
```

```
customer: {
  id: "customerId",
  name: "John Doe",
  email: "john@example.com",
  phone: "+1-555-0123",
  type: "retail|wholesale|b2b"
},

// Order Details
items: [
  {
    productId: "productId",
    variantId: "variantId",
    sku: "PROD-001-BLK",
    name: "Wireless Mouse - Black",
    quantity: 2,
    unitPrice: 24.99,
    discount: 0,
    tax: 4.00,
    total: 53.98
  }
],

// Pricing
subtotal: 49.98,
discountTotal: 0,
taxTotal: 4.00,
shippingCost: 8.99,
totalAmount: 62.97,
currency: "USD",

// Addresses
billingAddress: {
  street: "123 Main St",
  city: "New York",
  state: "NY",
  zipCode: "10001",
  country: "USA"
},
shippingAddress: { /* same structure */ },

// Fulfillment
fulfillmentStatus: "pending|processing|packed|shipped|delivered|cancelled",
paymentStatus: "pending|authorized|paid|refunded|failed",

warehouse: "warehouse1",
shippingMethod: "standard|express|overnight",
carrier: "UPS|FedEx|USPS",
```

```
  trackingNumber: "1Z999AA10123456784",

  // Dates
  orderDate: "2026-01-19T10:30:00Z",
  expectedShipDate: "2026-01-20",
  expectedDeliveryDate: "2026-01-25",
  actualShipDate: null,
  actualDeliveryDate: null,

  // Payment
  paymentMethod: "credit_card|debit_card|paypal|bank_transfer|cash",
  paymentReference: "ch_1234567890",

  // Source
  channel: "website|mobile_app|in_store|phone|marketplace",
  salesPerson: "userId",

  // Notes
  customerNotes: "Please gift wrap",
  internalNotes: "VIP customer",

  // Related
  invoiceId: "invoiceId",
  shipmentId: "shipmentId",

  // Status History
  statusHistory: [
    { status: "pending", timestamp: "2026-01-19T10:30:00Z", user: "system" },
    { status: "processing", timestamp: "2026-01-19T11:00:00Z", user: "userId" }
  ]
}
```

Order Workflow Order Lifecycle:

```
1. Order Created
   ↓
2. Payment Authorized/Received
   ↓
3. Inventory Reserved
   ↓
4. Order Processing (picking items)
   ↓
5. Order Packed
   ↓
6. Shipping Label Generated
   ↓
7. Order Shipped
   ↓
```

```
8. In Transit
   ↓
9. Delivered
   ↓
10. Order Completed
```

Automated Actions: - On Order Creation: - Reserve inventory - Send order confirmation email to customer - Notify fulfillment team - Check fraud risk (high-value orders)

- On Payment Received:
  - Update payment status
  - Trigger fulfillment workflow
  - Update accounting records
- On Shipment:
  - Send tracking email to customer
  - Update inventory (deduct stock)
  - Create invoice
- On Delivery:
  - Request review/feedback
  - Close order
  - Archive

Multi-Channel Support Sales Channels: - Direct Website: Native e-commerce integration - Mobile App: In-app purchases - In-Store POS: Physical store sales - Phone Orders: Manual entry by staff - Marketplaces: Amazon, eBay, Etsy (Premium Plus) - Wholesale Portal: B2B customer orders

Channel Features: - Unified order view across all channels - Channel-specific pricing rules - Inventory sync across channels - Channel performance analytics

Fulfillment Management Picking & Packing: - Pick lists generation (optimized by warehouse layout) - Batch picking for multiple orders - Packing slips printing - Quality check workflow - Gift wrapping options

Shipping Integration (Premium): - Real-time rate calculation from carriers - Label generation (UPS, FedEx, USPS, DHL) - Tracking number auto-import - Shipment notifications - Batch shipping for multiple orders - International customs forms

Drop Shipping: - Forward orders to suppliers - Supplier fulfillment tracking - Automated supplier notifications

Returns & Refunds Return Management:

```
{
  id: "uuid",
  returnNumber: "RET-2026-00045",
  orderId: "orderId",
  customer: "customerId",

  items: [
    {
```

```
      orderItemId: "itemId",
      productId: "productId",
      quantityReturned: 1,
      reason: "defective|wrong_item|not_as_described|changed_mind",
      condition: "unopened|opened|damaged",
      restockable: true
    }
  ],

  refundAmount: 24.99,
  refundShipping: true,
  restockingFee: 0,

  status: "requested|approved|received|refunded|rejected",

  returnLabel: "labelId",
  trackingNumber: "9876543210",

  notes: "Product arrived damaged in shipping",
  photos: ["photo1.jpg", "photo2.jpg"],

  requestDate: "2026-01-25",
  approvedDate: "2026-01-25",
  receivedDate: null,
  refundedDate: null
}
```

Return Workflow: 1. Customer requests return 2. Admin reviews and approves/rejects 3. Return label generated and emailed 4. Customer ships item back 5. Warehouse receives and inspects 6. If approved: Restock inventory + issue refund 7. If rejected: Return to customer with explanation

Refund Processing: - Full refunds - Partial refunds - Store credit option - Restocking fees (configurable) - Automatic or manual approval

3.2.3 Order Analytics

Key Metrics: - Orders per day/week/month - Average order value (AOV) - Order fulfillment time - Shipping time - Return rate - Order accuracy rate - Revenue by channel

Reports: - Sales by product - Sales by customer segment - Geographic sales distribution - Peak ordering times - Abandoned orders (started but not completed)

3.3 Finance & Accounting Suite

3.3.1 Overview

Purpose: Complete financial management including accounts, general ledger, expenses, revenue tracking, and financial reporting for small businesses.

Target Users: All enterprise users (85% adoption)

Tier: PREMIUM ($80/month base + $3/user)

Key Differentiators: - Double-entry accounting system - Multi-currency support - Automated reconciliation - Financial statement generation - Tax preparation support - Integration with banking and payments - Audit trail and compliance

3.3.2 Core Features

Chart of Accounts Account Structure:

```
{
  id: "uuid",
  accountNumber: "1000",
  accountName: "Cash - Operating Account",
  accountType: "asset|liability|equity|revenue|expense",
  accountSubtype: "current_asset|fixed_asset|current_liability|equity|operating_revenue|cogs|o

  parentAccount: null, // for sub-accounts

  balance: 15250.50,
  currency: "USD",

  description: "Primary business checking account",
  taxRelevant: true,

  status: "active|inactive",

  bankAccountLinked: true,
  bankAccountId: "bankId",

  lastReconciled: "2026-01-15"
}
```

Standard Chart of Accounts:

```
Assets (1000-1999)
  Current Assets (1000-1299)
     Cash & Cash Equivalents (1000-1099)
     Accounts Receivable (1100-1199)
     Inventory (1200-1299)
  Fixed Assets (1300-1999)
      Property & Equipment (1300-1399)
      Accumulated Depreciation (1400-1499)

Liabilities (2000-2999)
  Current Liabilities (2000-2299)
     Accounts Payable (2000-2099)
     Credit Cards (2100-2199)
```

```
    Short-term Loans (2200-2299)
  Long-term Liabilities (2300-2999)

Equity (3000-3999)
  Owner's Equity (3000-3099)
  Retained Earnings (3100-3199)
  Draws (3200-3299)

Revenue (4000-4999)
  Operating Revenue (4000-4499)
  Other Income (4500-4999)

Expenses (5000-9999)
  Cost of Goods Sold (5000-5999)
  Operating Expenses (6000-8999)
  Other Expenses (9000-9999)
```

Journal Entries Transaction Entry:

```
{
  id: "uuid",
  journalEntryNumber: "JE-2026-00123",
  date: "2026-01-19",
  type: "standard|adjusting|closing|reversing",

  description: "Payment received from customer",
  reference: "INV-2026-00045",

  lines: [
    {
      account: "1000", // Cash
      debit: 1000.00,
      credit: 0,
      description: "Payment from ABC Corp"
    },
    {
      account: "1100", // Accounts Receivable
      debit: 0,
      credit: 1000.00,
      description: "Invoice #45 settled"
    }
  ],

  totalDebit: 1000.00,
  totalCredit: 1000.00,
  balanced: true,

  source: "manual|automatic|import",
```

```
  sourceDocument: {
    type: "invoice|bill|order|payment",
    id: "documentId"
  },

  posted: true,
  postedDate: "2026-01-19T14:30:00Z",
  postedBy: "userId",

  attachments: ["receipt.pdf"],

  tags: ["customer_payment", "revenue"],

  approvalRequired: false,
  approvedBy: null,

  reversed: false,
  reversingEntry: null
}
```

Automated Journal Entries: - Sales orders Revenue recognition - Purchase orders Expense recognition - Payroll Salary expense - Inventory movements COGS adjustments - Depreciation schedules - Recurring entries (rent, subscriptions)

Accounts Receivable (AR) Customer Invoices: - Create and send professional invoices - Recurring invoices for subscriptions - Payment terms (Net 30, Net 60, etc.) - Partial payments tracking - Overdue notices automation - Aging reports (current, 30, 60, 90+ days)

Payment Collection: - Record payments against invoices - Multiple payment methods - Payment plans for large amounts - Bad debt write-offs - Collection reminders

Accounts Payable (AP) Vendor Bills: - Record bills from suppliers - Payment scheduling - Batch payments - Early payment discounts - 1099 tracking (US)

Expense Management: - Employee expense claims - Receipt scanning and OCR - Approval workflows - Reimbursement processing - Expense categorization

Banking & Reconciliation Bank Feeds (Premium Plus): - Auto-import transactions from banks - Smart categorization with ML - Duplicate detection - Match transactions to invoices/bills

Bank Reconciliation: - Monthly reconciliation workflow - Statement upload - Match transactions - Identify discrepancies - Reconciliation reports - Audit trail

Financial Statements Balance Sheet: - Assets = Liabilities + Equity - Current vs. non-current classification - Comparative periods - Export to PDF/Excel

Profit & Loss Statement (Income Statement): - Revenue - Expenses = Net Income - Operating vs. non-operating income/expenses - Gross profit, operating profit, net profit - Month/quarter/year views - YoY comparisons

Cash Flow Statement: - Operating activities - Investing activities - Financing activities - Beginning and ending cash - Direct or indirect method

Other Reports: - Trial Balance - General Ledger - Account Activity - Budget vs. Actual - Tax Summary

Budgeting & Forecasting Budget Management: - Create annual/monthly budgets by account - Department/project budgets - Budget vs. actual variance analysis - Budget alerts and notifications - Rolling forecasts

Financial Planning: - Revenue projections - Expense forecasting - Cash flow projections - Scenario modeling (best/worst/likely)

Multi-Currency Features: - Record transactions in any currency - Automatic exchange rate updates - Realized and unrealized gains/losses - Multi-currency reporting - Base currency conversion

Tax Management Tax Tracking: - Sales tax calculation and tracking - Tax liability accounts - Tax payment scheduling - Form preparation assistance (1099, W-2, etc.) - Tax reports for CPA

Compliance: - Audit trail for all transactions - User activity logs - Read-only archive periods - Data export for auditors

---

3.4 Employee Time & Attendance Tracking

3.4.1 Overview

Purpose: Track employee work hours, attendance, time-off, and integrate with payroll processing.

Target Users: All enterprise users with employees (90% adoption)

Tier: FREE (Core Feature, advanced features premium)

Key Differentiators: - Multiple clock-in methods (web, mobile, kiosk) - GPS and geofencing - Shift scheduling - Time-off management - Overtime calculation - Payroll integration - Compliance reporting

3.4.2 Core Features

Time Tracking Clock In/Out:

```
{
  id: "uuid",
  employee: {
    id: "employeeId",
    name: "Jane Smith",
    employeeNumber: "EMP-001",
    department: "Sales",
    position: "Sales Associate"
  },
```

```
  clockInTime: "2026-01-19T09:00:00Z",
  clockOutTime: "2026-01-19T17:30:00Z",

  clockInMethod: "web|mobile|kiosk|manual",
  clockInLocation: {
    lat: 40.7128,
    lng: -74.0060,
    address: "123 Office St, New York, NY"
  },

  breakStart: "2026-01-19T12:00:00Z",
  breakEnd: "2026-01-19T12:30:00Z",

  totalHours: 8.0,
  regularHours: 8.0,
  overtimeHours: 0,
  breakTime: 0.5,

  scheduledShift: "shiftId",
  late: false,
  earlyLeave: false,

  workLocation: "office|remote|client_site",
  project: "projectId", // if working on specific project

  notes: "Worked on client presentation",

  approved: true,
  approvedBy: "managerId",
  approvedDate: "2026-01-20"
}
```

Clock-In Methods:

1. Web App: Desktop browser clock in/out
2. Mobile App: Smartphone with GPS verification
3. Kiosk Mode: Tablet at office entrance with PIN/photo
4. Biometric: Fingerprint/face recognition (hardware integration)
5. Manual Entry: Manager enters time for employees
6. Auto Clock-In: Based on location/device (Premium)

Geofencing & GPS Location Verification: - Define geofence around office/job sites - Require employees within geofence to clock in - GPS coordinates logged with each clock-in - Prevent buddy punching - Remote work tracking

Shift Scheduling Schedule Management:

```
{
  id: "uuid",
```

```
  employee: "employeeId",

  shiftDate: "2026-01-20",
  startTime: "09:00",
  endTime: "17:00",

  shiftType: "regular|overtime|on_call|remote",
  department: "Sales",
  location: "Main Office",

  breakDuration: 30, // minutes

  role: "Sales Associate",
  supervisor: "managerId",

  status: "scheduled|confirmed|missed|worked",

  notes: "Bring laptop for remote setup",

  createdBy: "managerId",
  createdDate: "2026-01-15"
}
```

Scheduling Features: - Drag-and-drop calendar interface - Recurring shifts - Shift templates - Auto-scheduling based on availability - Shift swapping (employee-initiated, manager-approved) - Coverage requirements monitoring - Conflict detection (double-booking, overtime limits)

Time-Off Management Leave Request:

```
{
  id: "uuid",
  employee: "employeeId",

  leaveType: "vacation|sick|personal|parental|unpaid|bereavement",

  startDate: "2026-02-01",
  endDate: "2026-02-05",

  totalDays: 5,
  totalHours: 40,

  reason: "Family vacation",

  status: "pending|approved|denied|cancelled",

  requestedDate: "2026-01-19",
  reviewedBy: "managerId",
  reviewedDate: "2026-01-20",
```

```
  reviewNotes: "Approved - enjoy your trip!",

  affectsPayroll: true,
  paid: true
}
```

Leave Policies: - Accrual rules (X days per year) - Carry-over policies - Blackout dates (no time-off periods) - Minimum notice requirements - Maximum consecutive days - Balance tracking - Automatic accrual calculations

Attendance Reports Reports: - Timesheet Report: Individual or team timesheets - Attendance Summary: Whos in/out today - Late/Absence Report: Tardiness and absences - Overtime Report: OT hours by employee - Time-Off Report: Leave taken and balances - Labor Cost Report: Hours Œ wages - Department Hours: Total hours by department - Payroll Export: Ready for payroll processing

Analytics: - Average hours per employee - Absenteeism rate - Overtime trends - Peak staffing hours - Schedule adherence

Payroll Integration Payroll Data Export: - Regular hours - Overtime hours - Holiday pay - Sick/vacation hours used - Custom pay codes - Format: CSV, Excel, or direct integration with payroll software

Supported Payroll Systems: - QuickBooks Payroll - Gusto - ADP - Paychex - Custom exports

---

3.5 Project Management & Collaboration

3.5.1 Overview

Purpose: Plan, execute, and track projects with team collaboration, task assignment, and progress monitoring.

Target Users: All enterprise users (95% adoption)

Tier: FREE (Core Feature, advanced features premium)

Key Differentiators: - Multiple project views (list, board, Gantt, calendar) - Task dependencies and critical path - Resource allocation - Team collaboration features - Time tracking integration - File sharing and version control - Agile and waterfall methodologies

3.5.2 Core Features

Project Structure Project Schema:

```
{
  id: "uuid",
  projectCode: "PROJ-2026-001",
  name: "Website Redesign",
  description: "Complete redesign of company website",

  // Organization
```

```
portfolio: "portfolioId", // group of related projects
client: "clientId",

// Methodology
methodology: "agile|waterfall|hybrid",
framework: "scrum|kanban|custom",

// Team
projectManager: "userId",
team: [
  {
    userId: "userId1",
    role: "Developer",
    allocation: 100, // percentage
    hourlyRate: 75.00 // for cost tracking
  }
],

// Timeline
startDate: "2026-01-15",
endDate: "2026-04-15",
status: "planning|active|on_hold|completed|cancelled",

// Budget
budgetType: "fixed|time_and_materials",
estimatedHours: 400,
actualHours: 120,
budgetAmount: 30000,
actualCost: 9000,

// Progress
progress: 30, // percentage
tasksTotal: 45,
tasksCompleted: 14,

// Milestones
milestones: [
  {
    id: "m1",
    name: "Design Approval",
    dueDate: "2026-02-15",
    completed: true,
    completedDate: "2026-02-10"
  }
],

// Settings
visibility: "team|company|client",
```

```
  archived: false,

  // Metadata
  tags: ["marketing", "digital", "high_priority"],
  customFields: {
    "client_budget": 35000,
    "contract_number": "CNT-2026-045"
  }
}
```

Task Management (Enhanced from Individual) Team Task Schema:

```
{
  // All individual task properties plus:

  projectId: "projectId",

  assignedTo: "userId",
  assignedBy: "managerId",
  assignedDate: "2026-01-19",

  watchers: ["userId2", "userId3"], // notified of changes

  dependencies: [
    {
      taskId: "taskId2",
      type: "finish_to_start|start_to_start|finish_to_finish|start_to_finish",
      lag: 2 // days delay after dependency completes
    }
  ],

  estimatedHours: 8,
  actualHours: 6.5,

  billable: true,
  billingStatus: "not_invoiced|invoiced|paid",

  sprint: "sprintId", // for agile projects
  epicId: "epicId",
  storyPoints: 5, // agile estimation

  customStatus: "todo|in_review|qa|done", // customizable workflow

  comments: [
    {
      id: "commentId",
      user: "userId",
      text: "Design mockup attached",
```

```
      timestamp: "2026-01-19T15:30:00Z",
      attachments: ["mockup.png"]
    }
  ],

  timeEntries: ["timeEntryId1", "timeEntryId2"]
}
```

Project Views 1. List View: - Hierarchical task list - Group by: Status, Assignee, Priority, Milestone - Filters and saved views - Inline editing - Bulk actions

2. Board View (Kanban): - Customizable columns (To Do, In Progress, Review, Done) - Drag-and-drop task movement - WIP (work in progress) limits - Swim lanes (by assignee, priority) - Board for each project or cross-project boards

3. Gantt Chart (Premium): - Timeline visualization - Task dependencies shown - Critical path highlighting - Drag to adjust dates - Milestone markers - Resource allocation bars - Baseline comparison (planned vs. actual)

4. Calendar View: - Tasks plotted by due date - Color-coded by project/priority - Team calendar (whos working on what when) - Drag to reschedule

5. Timeline View: - High-level project roadmap - Milestone-focused - Multiple projects on one timeline - Dependency arrows

Agile Features (Premium) Sprints:

```
{
  id: "uuid",
  sprintNumber: 5,
  name: "Sprint 5 - Authentication Module",
  project: "projectId",

  startDate: "2026-01-20",
  endDate: "2026-02-02",
  duration: 14, // days

  goal: "Complete user authentication and authorization",

  tasks: ["taskId1", "taskId2"],
  totalStoryPoints: 34,
  completedStoryPoints: 0,

  status: "planned|active|completed",

  dailyStandups: [
    {
      date: "2026-01-20",
      notes: "Team synced on sprint goals",
      attendees: ["userId1", "userId2"]
```

```
    }
  ],

  retrospective: {
    whatWentWell: "Good collaboration",
    whatToImprove: "Better estimation",
    actionItems: ["Use planning poker"]
  }
}
```

Backlog Management: - Product backlog - Sprint backlog - Backlog grooming/refinement - Priority ordering - Story point estimation - Epics and user stories

Scrum Ceremonies: - Sprint planning - Daily standups - Sprint review - Sprint retrospective - Backlog refinement

Burn Charts: - Burndown chart (work remaining) - Burnup chart (work completed) - Velocity tracking (story points per sprint)

Team Collaboration Comments & Mentions: - Comment on tasks, projects, documents - @mention team members (notifications) - Rich text comments - File attachments - Emoji reactions

Activity Feed: - Real-time updates on project activity - Filter by: All, Mentions, Assignments, Comments - Mark as read/unread - Email digests (daily/weekly)

Notifications: - Task assigned to you - Task deadline approaching - Project milestone completed - Mentioned in comment - Task status changed - New file uploaded

File Management Project Files: - Folder structure within projects - Drag-drop upload - File versioning - Preview for common formats (PDF, images, Office docs) - Attach files to tasks - Shared drive per project

Version Control: - Track file versions - Restore previous versions - See who changed what when - Compare versions

Templates Project Templates: - Save projects as templates - Pre-configured tasks, milestones, workflows - Templates for common project types: - Software development - Marketing campaigns - Product launches - Event planning

Task Templates: - Recurring task sets - Checklists - Standard operating procedures

3.5.3 Resource Management

Resource Allocation: - View team capacity - Assign resources to projects/tasks - Prevent overallocation - Balance workload across team - Resource utilization reports

Capacity Planning: - Available hours per person - Allocated hours - Remaining capacity - Forecast future resource needs

---

3.6 Customer Relationship Management (CRM)

3.6.1 Overview

Purpose: Manage customer relationships, track sales pipeline, and improve customer engagement.

Target Users: Sales-focused businesses (60% of enterprise users)

Tier: PREMIUM ($65/month base + $2/user)

Key Differentiators: - 360ř customer view - Sales pipeline visualization - Lead scoring and qualification - Email integration and tracking - Activity timeline - Reporting and forecasting

3.6.2 Core Features

Contact Management Contact Schema:

```
{
  id: "uuid",
  type: "lead|contact|customer",

  // Personal Info
  firstName: "John",
  lastName: "Doe",
  email: "john.doe@example.com",
  phone: "+1-555-0123",
  mobile: "+1-555-0124",

  // Company (for B2B)
  company: {
    id: "companyId",
    name: "Acme Corp",
    industry: "Technology",
    size: "50-200",
    website: "acmecorp.com",
    address: { /* full address */ }
  },

  position: "CTO",
  department: "Technology",

  // Lead Source
  source: "website|referral|cold_call|event|advertising|partner",
  sourceDetails: "Google Ads - Product Campaign",

  // Status
  status: "new|contacted|qualified|unqualified|customer|lost",
  leadScore: 85, // 0-100

  // Assignment
  owner: "salesRepId",
```

```
  team: "teamId",

  // Relationships
  deals: ["dealId1", "dealId2"],
  orders: ["orderId1"],
  invoices: ["invoiceId1"],

  // Communication
  activities: ["activityId1"], // calls, emails, meetings
  lastContactedDate: "2026-01-15",
  nextFollowUpDate: "2026-01-25",

  // Preferences
  communicationPreference: "email|phone|sms",
  doNotContact: false,
  emailOptIn: true,

  // Custom Fields
  customFields: {
    "budget": 50000,
    "decision_timeline": "Q1 2026",
    "pain_points": "Manual processes"
  },

  // Social
  linkedin: "linkedin.com/in/johndoe",
  twitter: "@johndoe",

  // Tags
  tags: ["enterprise", "hot_lead", "decision_maker"],

  // Notes
  notes: "Very interested in enterprise plan",

  // Metadata
  createdDate: "2026-01-10",
  createdBy: "userId",
  lastModifiedDate: "2026-01-19"
}
```

Sales Pipeline Deal/Opportunity Schema:

```
{
  id: "uuid",
  dealName: "Acme Corp - Enterprise Plan",

  contact: "contactId",
  company: "companyId",
```

```
// Pipeline
pipeline: "pipelineId",
stage: "stageId",
stageHistory: [
  { stage: "Prospecting", enteredDate: "2026-01-10", exitDate: "2026-01-12" },
  { stage: "Qualification", enteredDate: "2026-01-12", exitDate: null }
],

// Value
dealValue: 50000,
currency: "USD",

// Probability
probability: 60, // percentage (auto or manual)
expectedRevenue: 30000, // dealValue × probability

// Timeline
expectedCloseDate: "2026-03-31",
actualCloseDate: null,

// Status
status: "open|won|lost",
lostReason: null,

// Assignment
owner: "salesRepId",

// Products
products: [
  {
    productId: "productId",
    quantity: 10,
    unitPrice: 5000,
    discount: 0,
    total: 50000
  }
],

// Activities
activities: ["activityId1"],
nextActivity: {
  type: "call",
  dueDate: "2026-01-22",
  description: "Product demo"
},

// Competitors
```

```
  competitors: ["Competitor A", "Competitor B"],

  // Notes
  notes: "Decision maker is CTO. Budget approved for Q1.",

  // Source
  source: "Inbound lead from website",

  createdDate: "2026-01-10",
  lastActivityDate: "2026-01-19"
}
```

Pipeline Stages:

```
Default Sales Pipeline:
1. Lead Generation (10% probability)
2. Qualification (20%)
3. Needs Analysis (40%)
4. Proposal (60%)
5. Negotiation (80%)
6. Closed Won (100%) or Closed Lost (0%)
```

Custom Pipelines: - Create multiple pipelines (e.g., Enterprise Sales, SMB Sales, Partnership) - Custom stages per pipeline - Automation rules (e.g., auto-move after 30 days)

Activity Tracking Activity Types:

```
{
  id: "uuid",
  type: "call|email|meeting|task|note|demo",

  subject: "Product demo with Acme Corp",
  description: "Demonstrated enterprise features",

  relatedTo: {
    contact: "contactId",
    company: "companyId",
    deal: "dealId"
  },

  participants: ["contactId", "userId"],

  dueDate: "2026-01-19T14:00:00Z",
  completedDate: "2026-01-19T15:30:00Z",

  duration: 90, // minutes

  outcome: "positive|neutral|negative",
  nextSteps: "Send proposal by Friday",
```

```
  owner: "salesRepId",

  status: "scheduled|completed|cancelled"
}
```

Email Integration (Premium): - Sync emails with contacts - Email tracking (opens, clicks) - Email templates - Bulk email campaigns - Auto-log emails to CRM

Lead Management Lead Capture: - Web forms integration - Import from CSV - Manual entry - API integration - Email parsing

Lead Scoring:

```
{
  criteria: [
    { field: "jobTitle", value: "C-Level", points: 20 },
    { field: "companySize", value: "100+", points: 15 },
    { field: "industry", value: "Technology", points: 10 },
    { field: "engagement", value: "Visited pricing page", points: 25 }
  ],

  totalScore: 70,
  grade: "A|B|C|D" // based on score ranges
}
```

Lead Qualification: - BANT framework (Budget, Authority, Need, Timeline) - Custom qualification criteria - Auto-convert to opportunity when qualified

Lead Assignment: - Round-robin distribution - Territory-based assignment - Lead pool for unassigned leads

Customer 360 View Unified Interface: - Contact information - Communication history (all calls, emails, meetings) - Deal history - Order history - Support tickets (if support module exists) - Files and documents - Notes and comments - Activity timeline

3.6.3 Sales Analytics

Reports: - Sales Forecast: Projected revenue by close date - Pipeline Report: Deals by stage, value, probability - Win/Loss Analysis: Why deals won or lost - Sales by Rep: Individual performance - Lead Conversion Rate: Lead Opportunity Customer - Sales Cycle Length: Average time to close - Activity Report: Calls, emails, meetings logged

Dashboards: - Revenue YTD vs. target - Pipeline value by stage - Top deals closing this month - Leaderboard (top sales reps)

3.7 Invoice & Billing Management

3.7.1 Overview

Purpose: Create, send, and track invoices with automated billing and payment processing.

Target Users: Service businesses, B2B (65% of enterprise users)

Tier: PREMIUM ($55/month base + $1.50/user)

Key Features: - Professional invoice creation - Recurring billing - Online payment acceptance - Payment reminders - Late fees - Multi-currency - Integration with accounting

Invoice Schema:

```
{
  id: "uuid",
  invoiceNumber: "INV-2026-00123",

  customer: "customerId",

  invoiceDate: "2026-01-19",
  dueDate: "2026-02-18", // Net 30

  lineItems: [
    {
      description: "Website Development",
      quantity: 40,
      unitPrice: 75.00,
      amount: 3000.00,
      taxable: true
    }
  ],

  subtotal: 3000.00,
  taxRate: 0.08,
  taxAmount: 240.00,
  discount: 0,
  total: 3240.00,

  currency: "USD",

  paymentTerms: "net_30",

  status: "draft|sent|viewed|partial|paid|overdue|cancelled",

  paymentsMade: [
    {
      date: "2026-01-25",
      amount: 1240.00,
      method: "credit_card",
```

```
      reference: "ch_abc123"
    }
  ],

  amountPaid: 1240.00,
  amountDue: 2000.00,

  sentDate: "2026-01-19",
  viewedDate: "2026-01-20",
  paidDate: null,

  reminders: [
    { sentDate: "2026-02-05", type: "upcoming" },
    { sentDate: "2026-02-19", type: "overdue" }
  ],

  linkedOrder: "orderId",
  linkedProject: "projectId",

  notes: "Payment due within 30 days",
  terms: "Standard payment terms apply",

  attachments: ["contract.pdf"]
}
```

Features: - Invoice templates (customizable) - Logo and branding - PDF generation - Email delivery - Online payment portal - Partial payments - Recurring invoices (subscriptions) - Credit notes/refunds - Payment reminders (automatic) - Late fees (automatic or manual)

3.8 Team Communication Hub

3.8.1 Overview

Purpose: Centralized team communication with channels, direct messaging, and integrations.

Target Users: All enterprise users (95% adoption)

Tier: FREE (Core Feature, advanced features premium)

Key Features: - Channels and direct messages - File sharing - Voice/video calls (Premium) - Screen sharing - Threaded conversations - @mentions and notifications - Search and archives - Integration with other modules

Channel Structure:

```
{
  id: "uuid",
  name: "general",
  displayName: "#general",
```

```
  type: "public|private|direct_message",

  description: "General company announcements",

  members: ["userId1", "userId2"],

  createdBy: "userId",
  createdDate: "2026-01-01",

  // Settings
  archived: false,
  notifications: "all|mentions|nothing",

  // Related
  project: "projectId", // optional project channel

  pinnedMessages: ["messageId1"]
}
```

Features: - Emoji reactions - Message editing and deletion - Code snippets - Polls - Reminders - Status updates (available, away, in meeting)

3.9 Document Management System

3.9.1 Overview

Purpose: Centralized document storage, organization, version control, and collaboration.

Target Users: All enterprise users (75% adoption)

Tier: PREMIUM ($50/month base + 100GB storage)

Key Features: - Folder hierarchy - Version control - Access permissions - Document approval workflows - Full-text search - Document templates - E-signatures (Premium Plus) - Audit trail

Document Schema:

```
{
  id: "uuid",
  name: "Client Proposal.docx",
  path: "/projects/website-redesign/proposals/",

  type: "file|folder",
  mimeType: "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
  size: 524288, // bytes

  version: 3,
  versions: [
    { version: 1, uploadedBy: "userId1", uploadedDate: "2026-01-10", size: 450000 },
    { version: 2, uploadedBy: "userId1", uploadedDate: "2026-01-15", size: 500000 },
```

```
    { version: 3, uploadedBy: "userId2", uploadedDate: "2026-01-19", size: 524288 }
  ],

  uploadedBy: "userId",
  uploadedDate: "2026-01-19T14:30:00Z",

  lastModifiedBy: "userId2",
  lastModifiedDate: "2026-01-19T16:45:00Z",

  permissions: [
    { userId: "userId1", role: "owner" },
    { userId: "userId2", role: "editor" },
    { teamId: "teamId1", role: "viewer" }
  ],

  tags: ["proposal", "client", "2026"],

  relatedTo: {
    project: "projectId",
    customer: "customerId"
  },

  checkoutStatus: {
    checkedOut: true,
    checkedOutBy: "userId2",
    checkedOutDate: "2026-01-19T16:30:00Z"
  },

  approval: {
    required: true,
    status: "pending|approved|rejected",
    approvers: ["managerId"],
    approvedBy: null,
    approvedDate: null
  }
}
```

3.10 Analytics & Reporting Dashboard

3.10.1 Overview

Purpose: Business intelligence with customizable dashboards and comprehensive reporting.

Target Users: Management, decision-makers (50% of enterprise users)

Tier: PREMIUM ($70/month)

Key Features: - Real-time dashboards - Custom reports - Data visualization (charts, graphs) -

Scheduled reports - Export capabilities - Cross-module analytics - KPI tracking - Predictive analytics (AI)

Dashboard Widgets: - Revenue metrics - Sales pipeline - Order fulfillment - Inventory levels - Team productivity - Project status - Customer metrics - Financial summaries

---

3.11 Resource Allocation & Planning

3.11.1 Overview

Purpose: Optimize resource utilization across projects and operations.

Target Users: Project managers, operations (40% of enterprise users)

Tier: PREMIUM ($60/month)

Key Features: - Resource capacity planning - Workload balancing - Skill matrix - Resource forecasting - Utilization reports - Scenario planning - Budget allocation

---

3.12 Vendor & Supplier Management

3.12.1 Overview

Purpose: Manage relationships with vendors and suppliers, track purchases and performance.

Target Users: Procurement, operations (35% of enterprise users)

Tier: PREMIUM ($45/month)

Key Features: - Vendor database - Purchase order management - Vendor performance tracking - Contract management - Price comparisons - Vendor approval workflows - Payment terms tracking

Vendor Schema:

```
{
  id: "uuid",
  vendorNumber: "VEN-001",
  name: "Office Supplies Inc",

  contactInfo: {
    email: "orders@officesupplies.com",
    phone: "+1-555-9999",
    address: { /* full address */ }
  },

  category: "Office Supplies",

  paymentTerms: "net_30",
  preferredPaymentMethod: "bank_transfer",

  rating: 4.5,
```

```
  contracts: ["contractId1"],

  products: ["productId1", "productId2"],

  performance: {
    onTimeDelivery: 95,
    qualityScore: 4.8,
    responseTime: "2 hours"
  },

  totalPurchases: 45000,

  status: "active|inactive|blocked"
}
```

Summary & Enterprise Feature Matrix

Free vs. Premium Allocation

FREE Tier (3 Features): 1. Project Management - Essential for team coordination 2. Team Communication - Core collaboration need 3. Time & Attendance - Basic employee tracking

Premium Features (Choose à la carte): 4. Finance & Accounting - $80/month 5. CRM - $65/month 6. Order Management - $60/month 7. Invoice & Billing - $55/month 8. Inventory Management - $75/month 9. Analytics Dashboard - $70/month 10. Resource Planning - $60/month 11. Document Management - $50/month 12. Vendor Management - $45/month

Bundle Options: - Operations Bundle: Inventory, Orders, Vendors - $150/month (save 25%) - Financial Bundle: Finance, Invoicing, Analytics - $180/month (save 20%) - Complete Enterprise: All 12 features - $400/month (save 45%)

Pricing Notes: - Prices are base monthly fees - Per-user fees apply ($1.50-$3/user depending on feature) - Annual billing offers 20% discount - Volume discounts for 50+ users

---

Next Chapter: Chapter 4: Technical Architecture

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Product Team - Status: Draft - Feature Specification Phase

Chapter 4: Technical Architecture

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Technical Design Phase

Table of Contents

- 4.1 System Architecture Overview
- 4.2 Monorepo Structure
- 4.3 Frontend Architecture (Next.js)
- 4.4 Mobile Architecture (Kotlin & Swift)
- 4.5 Backend Architecture (Express.js)
- 4.6 Database Design (MongoDB)
- 4.7 API Gateway & Microservices
- 4.8 Authentication & Authorization

4.1 System Architecture Overview

4.1.1 High-Level Architecture

```
                        CLIENT LAYER

  Web App         iOS App (Swift)         Android App (Kotlin)
  (Next.js)       Native UI               Material Design
  React           SwiftUI                 Jetpack Compose




                      CDN
                   (Cloudflare)



             API GATEWAY LAYER
         - Rate Limiting
         - Authentication
         - Request Routing
         - Load Balancing



                 BACKEND SERVICES LAYER

         Core API             Feature Services
         (Express.js)         - Individual Features
         - User Management    - Enterprise Features
         - Auth Service       - Analytics Service
         - Sync Service       - Notification Service
```

```
Primary DB            Services Layer
(MongoDB)             - Redis Cache
- User Data           - Message Queue
- App Data            - File Storage (S3)
- Transactions        - Search (Elastic)



External APIs
- Payment (Stripe)
- Email (SendGrid)
- SMS (Twilio)
- Analytics
```

4.1.2 Architecture Principles

1. Microservices-Oriented (Modular Monolith to Start) - Begin with modular monolith for faster development - Clear service boundaries for future microservices split - Feature modules are independently deployable units - Shared core services (auth, sync, notifications)

2. API-First Design - RESTful APIs with GraphQL for complex queries - Comprehensive API documentation (OpenAPI/Swagger) - Versioned APIs (v1, v2) for backward compatibility - Consistent response formats and error handling

3. Mobile-First, Cloud-Native - Responsive web design for all devices - Native mobile apps for optimal UX - Serverless functions for scaling - Container-based deployment (Docker/Kubernetes)

4. Real-Time & Offline-First - WebSocket connections for real-time updates - Offline data persistence with sync queue - Conflict resolution for concurrent edits - Progressive Web App (PWA) capabilities

5. Security by Design - Zero-trust architecture - Encryption at rest and in transit - Regular security audits - OWASP Top 10 compliance - SOC 2 Type II certification path

6. Scalability & Performance - Horizontal scaling capabilities - Caching strategies (Redis, CDN) - Database indexing and optimization - Lazy loading and code splitting - Performance monitoring and optimization

4.1.3 Technology Stack Summary

Layer	Technology	Purpose
Frontend Web	Next.js 14+ (App Router)	React framework with SSR/SSG
	TypeScript	Type safety
	Tailwind CSS	Utility-first styling
	Zustand/Redux	State management
	React Query	Server state management

Layer	Technology	Purpose
Mobile iOS	Swift 5.9+	Native iOS development
	SwiftUI	Declarative UI framework
	Combine	Reactive programming
	Core Data	Local persistence
Mobile Android	Kotlin 1.9+	Native Android development
	Jetpack Compose	Modern UI toolkit
	Coroutines/Flow	Async programming
	Room	Local database
Backend	Node.js 20+ LTS	Runtime environment
	Express.js 4.x	Web framework
	TypeScript	Type safety
	Socket.IO	WebSocket communication
Database	MongoDB 7.0+	Primary database
	Redis 7.x	Caching & sessions
	Elasticsearch 8.x	Full-text search
Infrastructure	Docker	Containerization
	Kubernetes	Container orchestration
	AWS/GCP	Cloud provider
	Terraform	Infrastructure as Code
DevOps	GitHub Actions	CI/CD pipeline
	Jest/Vitest	Testing
	Sentry	Error tracking
	Datadog	Monitoring & APM

4.1.4 Deployment Architecture

```
Production Environment:


                    AWS/GCP CLOUD



     Region 1 (Primary - US-East)

        Availability Zone A
        - Web Servers (Auto-scaling)
        - API Servers (Auto-scaling)
        - Redis Cluster (Master)


        Availability Zone B
        - Web Servers (Auto-scaling)
        - API Servers (Auto-scaling)
        - Redis Cluster (Replica)
```

```
MongoDB Atlas Cluster
- Multi-AZ Replica Set
- Automated Backups
```

```
Region 2 (DR - US-West)
- Hot Standby
- Database Read Replicas
- Disaster Recovery
```

```
Global Services
- CloudFront CDN
- S3 Object Storage
- Route 53 DNS
- CloudWatch Monitoring
```

Environments: - Development: Local Docker Compose setup - Staging: Mirrors production, smaller scale - Production: Multi-region, auto-scaling, HA - Testing: Isolated environment for QA

4.2 Monorepo Structure

4.2.1 Repository Organization

```
prms-monorepo/
  apps/
     web/                      # Next.js web application
        src/
           app/              # Next.js 14 App Router
              (auth)/       # Auth route group
              (individual)/ # Individual mode routes
              (enterprise)/ # Enterprise mode routes
              api/          # API routes
              layout.tsx
           components/       # React components
              ui/           # Base UI components
              features/     # Feature-specific components
              layouts/      # Layout components
           lib/              # Utilities and helpers
           hooks/            # Custom React hooks
           store/            # State management (Zustand)
```

```
        styles/           # Global styles
        types/            # TypeScript types
    public/               # Static assets
    next.config.js
    tailwind.config.js
    package.json

mobile-ios/               # Swift iOS application
    PrMS/
        App/              # App entry point
        Core/             # Core utilities
            Network/      # API client
            Storage/      # CoreData/UserDefaults
            Auth/         # Authentication
            Sync/         # Offline sync
        Features/         # Feature modules
            Tasks/
            Calendar/
            Finance/
            ...
        Shared/           # Shared components
            UI/           # SwiftUI components
            Models/       # Data models
            Extensions/
        Resources/        # Assets, localizations
    PrMSTests/
    PrMS.xcodeproj

mobile-android/           # Kotlin Android application
    app/
        src/
            main/
                java/com/prms/
                    core/          # Core functionality
                        network/   # Retrofit API client
                        database/  # Room database
                        auth/      # Authentication
                        sync/      # Offline sync
                    features/      # Feature modules
                        tasks/
                        calendar/
                        finance/
                        ...
                    ui/            # Compose UI components
                    MainActivity.kt
                res/               # Resources
                AndroidManifest.xml
            test/                  # Unit tests
```

```
            build.gradle.kts
        build.gradle.kts

    api/                        # Express.js backend
        src/
            app.ts              # Express app setup
            server.ts           # Server entry point
            config/             # Configuration
                database.ts
                redis.ts
                environment.ts
            middleware/         # Express middleware
                auth.ts
                validation.ts
                errorHandler.ts
                rateLimit.ts
            routes/             # API routes
                auth.routes.ts
                users.routes.ts
                individual/   # Individual feature routes
                enterprise/   # Enterprise feature routes
            controllers/        # Route controllers
            services/           # Business logic
                auth.service.ts
                sync.service.ts
                individual/
                enterprise/
            models/             # Mongoose models
            validators/         # Request validation schemas
            utils/              # Utility functions
            types/              # TypeScript types
        tests/                  # Test files
        package.json

packages/                       # Shared packages
    shared-types/               # Shared TypeScript types
        src/
            individual/         # Individual feature types
            enterprise/         # Enterprise feature types
            common/             # Common types
            index.ts
        package.json

    ui-components/              # Shared UI components
        src/
            Button/
            Input/
            Modal/
```

```
            index.ts
        package.json

    utils/                      # Shared utilities
        src/
            date.ts
            validation.ts
            formatting.ts
            index.ts
        package.json

    api-client/                 # Shared API client
        src/
            client.ts
            endpoints/
            index.ts
        package.json

infrastructure/                 # Infrastructure as Code
    terraform/                  # Terraform configs
        environments/
            dev/
            staging/
            production/
        modules/
    kubernetes/                 # K8s manifests
        base/
        overlays/
    docker/                     # Dockerfiles
        Dockerfile.web
        Dockerfile.api
        docker-compose.yml

scripts/                        # Build and automation scripts
    setup.sh
    migrate.sh
    deploy.sh

docs/                           # Documentation
    api/                        # API documentation
    architecture/               # Architecture diagrams
    guides/                     # Developer guides

.github/                        # GitHub configuration
    workflows/                  # CI/CD workflows
        web.yml
        api.yml
        mobile-ios.yml
```

```
        mobile-android.yml
    ISSUE_TEMPLATE/

package.json                    # Root package.json
pnpm-workspace.yaml             # pnpm workspace config
turbo.json                      # Turborepo config
.eslintrc.js                    # ESLint config
.prettierrc                     # Prettier config
tsconfig.json                   # Base TypeScript config
README.md
```

4.2.2 Monorepo Tooling

Build System: Turborepo

```
// turbo.json
{
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "dist/**", "build/**"]
    },
    "test": {
      "dependsOn": ["^build"],
      "outputs": ["coverage/**"]
    },
    "lint": {
      "outputs": []
    },
    "dev": {
      "cache": false,
      "persistent": true
    }
  }
}
```

Package Manager: pnpm - Efficient disk space usage - Faster installations - Strict dependency resolution - Workspace protocol support

Advantages: - Code sharing across apps (types, utils, UI components) - Unified dependency management - Consistent tooling and linting - Atomic commits across all apps - Simplified CI/CD

4.3 Frontend Architecture (Next.js)

4.3.1 Next.js Application Structure

App Router Architecture:

```tsx
// app/(individual)/layout.tsx
export default function IndividualLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <div className="individual-mode">
      <IndividualSidebar />
      <main>{children}</main>
    </div>
  )
}

// app/(enterprise)/layout.tsx
export default function EnterpriseLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <div className="enterprise-mode">
      <EnterpriseSidebar />
      <main>{children}</main>
    </div>
  )
}
```

Route Organization:

```
app/
   (auth)/
      login/page.tsx
      signup/page.tsx
      mode-selection/page.tsx    # Choose Individual/Enterprise

   (individual)/
      dashboard/page.tsx
      tasks/
         page.tsx
         [id]/page.tsx
      calendar/page.tsx
      finance/page.tsx
      habits/page.tsx
      ...

   (enterprise)/
      dashboard/page.tsx
      projects/
```

```
        page.tsx
        [id]/
            page.tsx
            tasks/page.tsx
            team/page.tsx
    inventory/page.tsx
    orders/page.tsx
    ...

  api/                          # API routes for SSR/ISR
      auth/[...nextauth]/route.ts
      webhooks/stripe/route.ts
```

4.3.2 State Management Strategy

Client State: Zustand

```
// store/userStore.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface UserState {
  user: User | null
  mode: 'individual' | 'enterprise'
  setUser: (user: User) => void
  setMode: (mode: 'individual' | 'enterprise') => void
  logout: () => void
}

export const useUserStore = create<UserState>()(
  persist(
    (set) => ({
      user: null,
      mode: 'individual',
      setUser: (user) => set({ user }),
      setMode: (mode) => set({ mode }),
      logout: () => set({ user: null }),
    }),
    {
      name: 'user-storage',
    }
  )
)
```

Server State: React Query

```
// hooks/useTasks.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
```

```
export function useTasks() {
  return useQuery({
    queryKey: ['tasks'],
    queryFn: async () => {
      const res = await fetch('/api/tasks')
      return res.json()
    },
  })
}

export function useCreateTask() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: async (task: NewTask) => {
      const res = await fetch('/api/tasks', {
        method: 'POST',
        body: JSON.stringify(task),
      })
      return res.json()
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['tasks'] })
    },
  })
}
```

4.3.3 Component Architecture

Component Hierarchy:

```
Pages (Route Components)
     Layouts (Shared structure)
           Features (Business logic components)
                 UI Components (Reusable, presentational)
```

Example Feature Component:

```
// components/features/TaskList/TaskList.tsx
'use client'

import { useTasks } from '@/hooks/useTasks'
import { TaskItem } from './TaskItem'
import { Button, Spinner } from '@/components/ui'

export function TaskList() {
  const { data: tasks, isLoading, error } = useTasks()
  const createTask = useCreateTask()
```

```
  if (isLoading) return <Spinner />
  if (error) return <ErrorMessage error={error} />

  return (
    <div className="space-y-4">
      <div className="flex justify-between items-center">
        <h2 className="text-2xl font-bold">Tasks</h2>
        <Button onClick={() => createTask.mutate({ title: 'New Task' })}>
          Add Task
        </Button>
      </div>

      <div className="space-y-2">
        {tasks?.map(task => (
          <TaskItem key={task.id} task={task} />
        ))}
      </div>
    </div>
  )
}
```

4.3.4 Performance Optimizations

Code Splitting:

```
// Dynamic imports for heavy components
import dynamic from 'next/dynamic'

const Chart = dynamic(() => import('@/components/Chart'), {
  loading: () => <Spinner />,
  ssr: false, // Disable SSR for client-only components
})
```

Image Optimization:

```
import Image from 'next/image'

<Image
  src="/product.jpg"
  alt="Product"
  width={500}
  height={300}
  loading="lazy"
  placeholder="blur"
/>
```

Caching Strategy: - Static pages: ISR with revalidation - Dynamic pages: SSR with aggressive caching - API routes: Cache-Control headers - CDN caching for static assets

4.3.5 Offline Support (PWA)

Service Worker:

```
// next.config.js with next-pwa
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development',
})

module.exports = withPWA({
  // Next.js config
})
```

Offline Data Sync:

```
// lib/offlineSync.ts
import { openDB } from 'idb'

const db = await openDB('prms-offline', 1, {
  upgrade(db) {
    db.createObjectStore('pendingActions')
  },
})

export async function queueAction(action: Action) {
  await db.add('pendingActions', action)
}

export async function syncPendingActions() {
  const actions = await db.getAll('pendingActions')
  for (const action of actions) {
    await processAction(action)
    await db.delete('pendingActions', action.id)
  }
}
```

4.4 Mobile Architecture (Kotlin & Swift)

4.4.1 iOS Architecture (Swift)

MVVM + Clean Architecture:

```
PrMS iOS App
   Presentation Layer
       ViewModels (ObservableObject)
       Views (SwiftUI)
```

```
Domain Layer
    Use Cases
    Entities
    Repository Protocols

Data Layer
    Repository Implementations
    Network (URLSession/Alamofire)
    Local Storage (CoreData)
    Cache (UserDefaults)
```

Example Feature Module:

```swift
// Features/Tasks/Domain/TaskRepository.swift
protocol TaskRepository {
    func getTasks() async throws -> [Task]
    func createTask(_ task: Task) async throws -> Task
    func updateTask(_ task: Task) async throws
    func deleteTask(id: String) async throws
}

// Features/Tasks/Data/TaskRepositoryImpl.swift
class TaskRepositoryImpl: TaskRepository {
    private let apiClient: APIClient
    private let localStorage: LocalStorage

    func getTasks() async throws -> [Task] {
        do {
            let tasks = try await apiClient.get("/tasks")
            await localStorage.save(tasks)
            return tasks
        } catch {
            // Return cached data if offline
            return await localStorage.getTasks()
        }
    }
}

// Features/Tasks/Presentation/TaskListViewModel.swift
@MainActor
class TaskListViewModel: ObservableObject {
    @Published var tasks: [Task] = []
    @Published var isLoading = false
    @Published var error: Error?

    private let repository: TaskRepository

    init(repository: TaskRepository) {
```

```
        self.repository = repository
    }

    func loadTasks() async {
        isLoading = true
        do {
            tasks = try await repository.getTasks()
        } catch {
            self.error = error
        }
        isLoading = false
    }
}

// Features/Tasks/Presentation/TaskListView.swift
struct TaskListView: View {
    @StateObject var viewModel: TaskListViewModel

    var body: some View {
        List(viewModel.tasks) { task in
            TaskRow(task: task)
        }
        .task {
            await viewModel.loadTasks()
        }
    }
}
```

Networking Layer:

```
// Core/Network/APIClient.swift
class APIClient {
    private let baseURL = "https://api.prms.app/v1"
    private let session: URLSession

    func get<T: Decodable>(_ endpoint: String) async throws -> T {
        let url = URL(string: baseURL + endpoint)!
        var request = URLRequest(url: url)
        request.addValue("Bearer \(authToken)", forHTTPHeaderField: "Authorization")

        let (data, response) = try await session.data(for: request)

        guard let httpResponse = response as? HTTPURLResponse,
              (200...299).contains(httpResponse.statusCode) else {
            throw APIError.invalidResponse
        }

        return try JSONDecoder().decode(T.self, from: data)
    }
```

```
}
```

Offline Sync:

```
// Core/Sync/SyncManager.swift
actor SyncManager {
    private let coreData: CoreDataStack
    private let apiClient: APIClient

    func sync() async {
        // 1. Upload pending changes
        let pending = await coreData.getPendingChanges()
        for change in pending {
            do {
                try await uploadChange(change)
                await coreData.markAsSynced(change)
            } catch {
                print("Sync failed: \(error)")
            }
        }

        // 2. Download remote changes
        let lastSync = await coreData.getLastSyncTimestamp()
        let changes = try? await apiClient.get("/sync?since=\(lastSync)")
        await coreData.applyRemoteChanges(changes)
    }
}
```

4.4.2 Android Architecture (Kotlin)

MVVM + Clean Architecture:

```
// features/tasks/domain/repository/TaskRepository.kt
interface TaskRepository {
    suspend fun getTasks(): Flow<Result<List<Task>>>
    suspend fun createTask(task: Task): Result<Task>
    suspend fun updateTask(task: Task): Result<Unit>
    suspend fun deleteTask(id: String): Result<Unit>
}

// features/tasks/data/repository/TaskRepositoryImpl.kt
class TaskRepositoryImpl @Inject constructor(
    private val apiService: ApiService,
    private val taskDao: TaskDao,
    private val networkMonitor: NetworkMonitor
) : TaskRepository {

    override suspend fun getTasks(): Flow<Result<List<Task>>> = flow {
        // Try local cache first
```

```
        emit(Result.Loading)

        val cachedTasks = taskDao.getAllTasks().first()
        if (cachedTasks.isNotEmpty()) {
            emit(Result.Success(cachedTasks))
        }

        // Fetch from network if online
        if (networkMonitor.isOnline.first()) {
            try {
                val remoteTasks = apiService.getTasks()
                taskDao.insertAll(remoteTasks)
                emit(Result.Success(remoteTasks))
            } catch (e: Exception) {
                emit(Result.Error(e))
            }
        }
    }
}

// features/tasks/presentation/TaskListViewModel.kt
@HiltViewModel
class TaskListViewModel @Inject constructor(
    private val repository: TaskRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow<TaskListUiState>(TaskListUiState.Loading)
    val uiState: StateFlow<TaskListUiState> = _uiState.asStateFlow()

    init {
        loadTasks()
    }

    private fun loadTasks() {
        viewModelScope.launch {
            repository.getTasks().collect { result ->
                _uiState.value = when (result) {
                    is Result.Loading -> TaskListUiState.Loading
                    is Result.Success -> TaskListUiState.Success(result.data)
                    is Result.Error -> TaskListUiState.Error(result.exception)
                }
            }
        }
    }
}

// features/tasks/presentation/TaskListScreen.kt
@Composable
```

```
fun TaskListScreen(
    viewModel: TaskListViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsState()

    when (val state = uiState) {
        is TaskListUiState.Loading -> LoadingIndicator()
        is TaskListUiState.Success -> {
            LazyColumn {
                items(state.tasks) { task ->
                    TaskItem(task = task)
                }
            }
        }
        is TaskListUiState.Error -> ErrorMessage(state.exception)
    }
}
```

Local Database (Room):

```
// core/database/entities/TaskEntity.kt
@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey val id: String,
    val title: String,
    val description: String?,
    val status: String,
    val dueDate: Long?,
    val createdAt: Long,
    val updatedAt: Long,
    @ColumnInfo(name = "sync_status")
    val syncStatus: SyncStatus
)

// core/database/dao/TaskDao.kt
@Dao
interface TaskDao {
    @Query("SELECT * FROM tasks WHERE sync_status != 'DELETED'")
    fun getAllTasks(): Flow<List<TaskEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(tasks: List<TaskEntity>)

    @Update
    suspend fun update(task: TaskEntity)

    @Query("DELETE FROM tasks WHERE id = :taskId")
    suspend fun delete(taskId: String)
```

```
    @Query("SELECT * FROM tasks WHERE sync_status = 'PENDING'")
    suspend fun getPendingSync(): List<TaskEntity>
}
```

Dependency Injection (Hilt):

```
// di/AppModule.kt
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.prms.app/v1/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun provideApiService(retrofit: Retrofit): ApiService {
        return retrofit.create(ApiService::class.java)
    }

    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): PrmsDatabase {
        return Room.databaseBuilder(
            context,
            PrmsDatabase::class.java,
            "prms_database"
        ).build()
    }
}
```

4.4.3 Mobile Offline Sync Strategy

Sync Protocol:

```
1. Queue local changes (CRUD operations)
2. Periodically sync when online:
   a. Upload pending local changes
   b. Download remote changes since last sync
   c. Resolve conflicts (last-write-wins or manual)
3. Update local database
4. Notify UI of changes
```

Conflict Resolution:

```
data class SyncConflict(
    val localVersion: Entity,
    val remoteVersion: Entity,
    val timestamp: Long
)

sealed class ConflictResolution {
    object UseLocal : ConflictResolution()
    object UseRemote : ConflictResolution()
    data class Merge(val merged: Entity) : ConflictResolution()
}
```

4.5 Backend Architecture (Express.js)

4.5.1 Application Structure

Layered Architecture:

```
Routes Layer                    ← HTTP endpoints

Controllers Layer               ← Request handling

Services Layer                  ← Business logic

Repositories Layer              ← Data access

Models Layer                    ← Database schemas
```

Example Implementation:

```
// routes/tasks.routes.ts
import { Router } from 'express'
import { TaskController } from '../controllers/task.controller'
import { authenticate } from '../middleware/auth'
import { validate } from '../middleware/validation'
import { createTaskSchema } from '../validators/task.validator'

const router = Router()
const controller = new TaskController()

router.use(authenticate)

router.get('/', controller.getTasks)
router.post('/', validate(createTaskSchema), controller.createTask)
router.put('/:id', controller.updateTask)
router.delete('/:id', controller.deleteTask)
```

```
export default router

// controllers/task.controller.ts
export class TaskController {
  private taskService = new TaskService()

  getTasks = async (req: Request, res: Response) => {
    try {
      const tasks = await this.taskService.getUserTasks(req.user.id)
      res.json({ success: true, data: tasks })
    } catch (error) {
      next(error)
    }
  }

  createTask = async (req: Request, res: Response, next: NextFunction) => {
    try {
      const task = await this.taskService.createTask({
        ...req.body,
        userId: req.user.id
      })
      res.status(201).json({ success: true, data: task })
    } catch (error) {
      next(error)
    }
  }
}

// services/task.service.ts
export class TaskService {
  private taskRepository = new TaskRepository()
  private notificationService = new NotificationService()
  private syncService = new SyncService()

  async getUserTasks(userId: string): Promise<Task[]> {
    return this.taskRepository.findByUserId(userId)
  }

  async createTask(data: CreateTaskDTO): Promise<Task> {
    // Business logic
    const task = await this.taskRepository.create(data)

    // Side effects
    await this.notificationService.sendTaskCreated(task)
    await this.syncService.broadcastUpdate(data.userId, 'task_created', task)

    return task
```

```
  }
}

// repositories/task.repository.ts
export class TaskRepository {
  private model = TaskModel

  async findByUserId(userId: string): Promise<Task[]> {
    return this.model.find({ userId, deleted: false }).sort({ createdAt: -1 })
  }

  async create(data: CreateTaskDTO): Promise<Task> {
    const task = new this.model(data)
    return task.save()
  }
}
```

4.5.2 API Design Patterns

RESTful API Structure:

```
/api/v1
   /auth
      POST /register
      POST /login
      POST /logout
      POST /refresh
      POST /forgot-password

   /users
      GET /:id
      PUT /:id
      DELETE /:id

   /individual
      /tasks
         GET /
         POST /
         GET /:id
         PUT /:id
         DELETE /:id
      /calendar
      /finance
      ...

   /enterprise
       /projects
       /inventory
```

```
      /orders
      ...
```

Response Format:

```typescript
interface ApiResponse<T> {
  success: boolean
  data?: T
  error?: {
    code: string
    message: string
    details?: any
  }
  meta?: {
    total?: number
    page?: number
    limit?: number
  }
}

// Success response
{
  "success": true,
  "data": [...],
  "meta": {
    "total": 100,
    "page": 1,
    "limit": 20
  }
}

// Error response
{
  "success": false,
  "error": {
    "code": "TASK_NOT_FOUND",
    "message": "Task with ID 123 not found"
  }
}
```

4.5.3 Middleware Stack

```typescript
// app.ts
import express from 'express'
import helmet from 'helmet'
import cors from 'cors'
import compression from 'compression'
import rateLimit from 'express-rate-limit'
```

```
const app = express()

// Security
app.use(helmet())
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(','),
  credentials: true
}))

// Body parsing
app.use(express.json({ limit: '10mb' }))
app.use(express.urlencoded({ extended: true }))

// Compression
app.use(compression())

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP'
})
app.use('/api/', limiter)

// Logging
app.use(morgan('combined'))

// Custom middleware
app.use(requestId)
app.use(authenticate)

// Routes
app.use('/api/v1', routes)

// Error handling
app.use(notFoundHandler)
app.use(errorHandler)
```

4.5.4 Real-Time Communication

WebSocket Server:

```
// server.ts
import { Server } from 'socket.io'
import { createServer } from 'http'

const httpServer = createServer(app)
const io = new Server(httpServer, {
```

```
  cors: {
    origin: process.env.ALLOWED_ORIGINS?.split(','),
    credentials: true
  }
})

// Authentication middleware
io.use(async (socket, next) => {
  const token = socket.handshake.auth.token
  try {
    const user = await verifyToken(token)
    socket.data.user = user
    next()
  } catch (err) {
    next(new Error('Authentication failed'))
  }
})

// Connection handling
io.on('connection', (socket) => {
  const userId = socket.data.user.id

  // Join user's personal room
  socket.join(`user:${userId}`)

  // Join enterprise rooms if applicable
  if (socket.data.user.enterpriseId) {
    socket.join(`enterprise:${socket.data.user.enterpriseId}`)
  }

  // Event handlers
  socket.on('task:update', async (data) => {
    const task = await taskService.updateTask(data)
    socket.to(`user:${userId}`).emit('task:updated', task)
  })

  socket.on('disconnect', () => {
    console.log('User disconnected:', userId)
  })
})

// Broadcasting updates from services
export function broadcastToUser(userId: string, event: string, data: any) {
  io.to(`user:${userId}`).emit(event, data)
}
```

4.5.5 Background Jobs

Job Queue (Bull):

```
// queues/emailQueue.ts
import Queue from 'bull'
import { sendEmail } from '../services/email.service'

export const emailQueue = new Queue('email', {
  redis: {
    host: process.env.REDIS_HOST,
    port: Number(process.env.REDIS_PORT)
  }
})

emailQueue.process(async (job) => {
  const { to, subject, body } = job.data
  await sendEmail(to, subject, body)
})

// Usage in service
await emailQueue.add({
  to: 'user@example.com',
  subject: 'Welcome to PrMS',
  body: '...'
}, {
  attempts: 3,
  backoff: {
    type: 'exponential',
    delay: 2000
  }
})
```

Scheduled Jobs (node-cron):

```
// jobs/scheduledJobs.ts
import cron from 'node-cron'

// Daily reminder notifications - 8 AM
cron.schedule('0 8 * * *', async () => {
  await sendDailyReminders()
})

// Weekly reports - Sunday 9 PM
cron.schedule('0 21 * * 0', async () => {
  await generateWeeklyReports()
})

// Cleanup old data - Daily 2 AM
```

```
cron.schedule('0 2 * * *', async () => {
  await cleanupOldData()
})
```

4.6 Database Design (MongoDB)

4.6.1 Database Architecture

MongoDB Atlas Cluster: - Multi-region replica set - Auto-scaling storage - Automated backups (point-in-time recovery) - Performance monitoring

Connection Strategy:

```
// config/database.ts
import mongoose from 'mongoose'

export async function connectDatabase() {
  try {
    await mongoose.connect(process.env.MONGODB_URI!, {
      maxPoolSize: 10,
      serverSelectionTimeoutMS: 5000,
      socketTimeoutMS: 45000,
    })

    console.log('MongoDB connected successfully')

    // Handle connection events
    mongoose.connection.on('error', (err) => {
      console.error('MongoDB connection error:', err)
    })

    mongoose.connection.on('disconnected', () => {
      console.warn('MongoDB disconnected')
    })
  } catch (error) {
    console.error('Failed to connect to MongoDB:', error)
    process.exit(1)
  }
}
```

4.6.2 Schema Design Principles

1. Embed vs. Reference: - Embed: One-to-few, data accessed together - Reference: One-to-many, many-to-many, independent data

2. Denormalization for Performance: - Duplicate frequently accessed data - Use caching for computed values - Balance between read and write performance

Example Schema:

```
// models/User.model.ts
import { Schema, model } from 'mongoose'

const userSchema = new Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true,
    select: false // Don't return in queries by default
  },
  profile: {
    firstName: String,
    lastName: String,
    avatar: String,
    timezone: { type: String, default: 'UTC' }
  },
  mode: {
    type: String,
    enum: ['individual', 'enterprise', 'both'],
    default: 'individual'
  },
  subscriptions: [{
    feature: String,
    tier: { type: String, enum: ['free', 'premium', 'premium_plus'] },
    status: { type: String, enum: ['active', 'cancelled', 'expired'] },
    startDate: Date,
    endDate: Date
  }],
  settings: {
    notifications: {
      email: { type: Boolean, default: true },
      push: { type: Boolean, default: true },
      sms: { type: Boolean, default: false }
    },
    language: { type: String, default: 'en' },
    theme: { type: String, enum: ['light', 'dark', 'auto'], default: 'auto' }
  },
  metadata: {
    lastLogin: Date,
    loginCount: { type: Number, default: 0 },
    createdAt: { type: Date, default: Date.now },
    updatedAt: { type: Date, default: Date.now }
```

```
  }
}, {
  timestamps: true,
  toJSON: { virtuals: true },
  toObject: { virtuals: true }
})

// Indexes
userSchema.index({ email: 1 })
userSchema.index({ 'metadata.lastLogin': 1 })

// Virtual fields
userSchema.virtual('fullName').get(function() {
  return `${this.profile.firstName} ${this.profile.lastName}`
})

// Middleware
userSchema.pre('save', async function(next) {
  if (this.isModified('password')) {
    this.password = await bcrypt.hash(this.password, 12)
  }
  next()
})

export const User = model('User', userSchema)
```

4.6.3 Caching Strategy

Redis Cache:

```
// services/cache.service.ts
import Redis from 'ioredis'

const redis = new Redis({
  host: process.env.REDIS_HOST,
  port: Number(process.env.REDIS_PORT),
  password: process.env.REDIS_PASSWORD
})

export class CacheService {
  async get<T>(key: string): Promise<T | null> {
    const data = await redis.get(key)
    return data ? JSON.parse(data) : null
  }

  async set(key: string, value: any, ttl: number = 3600): Promise<void> {
    await redis.setex(key, ttl, JSON.stringify(value))
  }
```

```
  async del(key: string): Promise<void> {
    await redis.del(key)
  }

  async invalidatePattern(pattern: string): Promise<void> {
    const keys = await redis.keys(pattern)
    if (keys.length > 0) {
      await redis.del(...keys)
    }
  }
}

// Usage
const cache = new CacheService()

async function getUserTasks(userId: string) {
  const cacheKey = `tasks:${userId}`

  // Try cache first
  let tasks = await cache.get<Task[]>(cacheKey)

  if (!tasks) {
    // Cache miss - fetch from DB
    tasks = await TaskModel.find({ userId })
    await cache.set(cacheKey, tasks, 300) // 5 min TTL
  }

  return tasks
}
```

4.7 API Gateway & Microservices

4.7.1 API Gateway

Purpose: - Single entry point for all clients - Request routing - Authentication & authorization - Rate limiting - API versioning - Response caching - Request/response transformation

Implementation (Kong/Express Gateway):

```
# kong.yml
services:
  - name: auth-service
    url: http://auth:3001
    routes:
      - name: auth-route
        paths:
          - /api/v1/auth
```

```yaml
  - name: individual-service
    url: http://individual:3002
    routes:
      - name: individual-route
        paths:
          - /api/v1/individual
        plugins:
          - name: jwt
          - name: rate-limiting
            config:
              minute: 60

  - name: enterprise-service
    url: http://enterprise:3003
    routes:
      - name: enterprise-route
        paths:
          - /api/v1/enterprise
```

4.7.2 Service Communication

REST for External, gRPC for Internal:

```proto
// protos/user.proto
syntax = "proto3";

service UserService {
  rpc GetUser (GetUserRequest) returns (UserResponse);
  rpc UpdateUser (UpdateUserRequest) returns (UserResponse);
}

message GetUserRequest {
  string user_id = 1;
}

message UserResponse {
  string id = 1;
  string email = 2;
  string first_name = 3;
  string last_name = 4;
}
```

4.8 Authentication & Authorization

4.8.1 Authentication Flow

JWT-Based Authentication:

```
// services/auth.service.ts
import jwt from 'jsonwebtoken'
import bcrypt from 'bcrypt'

export class AuthService {
  async login(email: string, password: string) {
    const user = await User.findOne({ email }).select('+password')

    if (!user || !(await bcrypt.compare(password, user.password))) {
      throw new Error('Invalid credentials')
    }

    const accessToken = this.generateAccessToken(user)
    const refreshToken = this.generateRefreshToken(user)

    // Store refresh token
    await RefreshToken.create({
      userId: user.id,
      token: refreshToken,
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
    })

    return { accessToken, refreshToken, user }
  }

  private generateAccessToken(user: any): string {
    return jwt.sign(
      {
        userId: user.id,
        email: user.email,
        mode: user.mode
      },
      process.env.JWT_SECRET!,
      { expiresIn: '15m' }
    )
  }

  private generateRefreshToken(user: any): string {
    return jwt.sign(
      { userId: user.id },
      process.env.JWT_REFRESH_SECRET!,
      { expiresIn: '7d' }
    )
  }
}
```

4.8.2 Authorization (RBAC)

Role-Based Access Control:

```
// middleware/authorize.ts
export function authorize(...allowedRoles: string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Unauthorized' })
    }

    const hasRole = req.user.roles.some(role =>
      allowedRoles.includes(role)
    )

    if (!hasRole) {
      return res.status(403).json({ error: 'Forbidden' })
    }

    next()
  }
}

// Usage
router.delete('/users/:id',
  authenticate,
  authorize('admin', 'super_admin'),
  deleteUser
)
```

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Engineering Team - Status: Draft - Technical Design Phase

Next Chapter: Chapter 5: Database Schemas

Chapter 5: Database Schemas

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Database Design Phase

Table of Contents

- 5.1 User & Authentication Schemas
- 5.2 Individual Features Schemas
- 5.3 Enterprise Features Schemas
- 5.4 Shared Schemas

- 5.5 Indexing Strategy
- 5.6 Data Migration Strategy

5.1 User & Authentication Schemas

5.1.1 Users Collection

```
// Collection: users
{
  _id: ObjectId,
  email: String,                    // Unique, indexed
  password: String,                 // Hashed with bcrypt

  profile: {
    firstName: String,
    lastName: String,
    avatar: String,                 // S3 URL
    phone: String,
    dateOfBirth: Date,
    timezone: String,               // Default: "UTC"
    language: String,               // Default: "en"
  },

  mode: String,                     // "individual" | "enterprise" | "both"

  // Subscription Management
  subscriptions: [{
    feature: String,                // e.g., "finance_manager", "habit_tracker"
    tier: String,                   // "free" | "premium" | "premium_plus"
    status: String,                 // "active" | "cancelled" | "expired" | "trialing"
    startDate: Date,
    endDate: Date,
    autoRenew: Boolean,
    paymentMethod: String,
    stripeSubscriptionId: String,
  }],

  // Enterprise Association
  enterpriseId: ObjectId,           // Reference to enterprises collection
  enterpriseRole: String,           // "owner" | "admin" | "manager" | "member"

  // Settings
  settings: {
    notifications: {
      email: Boolean,               // Default: true
      push: Boolean,                // Default: true
      sms: Boolean,                 // Default: false
```

```
      inApp: Boolean,                // Default: true
    },
    theme: String,                   // "light" | "dark" | "auto"
    dateFormat: String,              // "MM/DD/YYYY" | "DD/MM/YYYY"
    timeFormat: String,              // "12h" | "24h"
    weekStart: Number,               // 0-6 (Sunday-Saturday)
    currency: String,                // "USD", "EUR", etc.
  },

  // Security
  twoFactorEnabled: Boolean,
  twoFactorSecret: String,

  // Metadata
  emailVerified: Boolean,
  emailVerificationToken: String,
  passwordResetToken: String,
  passwordResetExpires: Date,

  lastLogin: Date,
  loginCount: Number,
  lastActiveAt: Date,

  status: String,                    // "active" | "suspended" | "deleted"

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ enterpriseId: 1 })
db.users.createIndex({ lastActiveAt: 1 })
db.users.createIndex({ "subscriptions.status": 1 })
```

5.1.2 Refresh Tokens Collection

```
// Collection: refresh_tokens
{
  _id: ObjectId,
  userId: ObjectId,                  // Reference to users
  token: String,                     // Hashed token
  deviceInfo: {
    userAgent: String,
    ip: String,
    platform: String,                // "web" | "ios" | "android"
    deviceId: String,
  },
```

```
  expiresAt: Date,
  createdAt: Date,
}

// Indexes
db.refresh_tokens.createIndex({ userId: 1 })
db.refresh_tokens.createIndex({ token: 1 }, { unique: true })
db.refresh_tokens.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 }) // TTL index
```

5.1.3 Sessions Collection

```
// Collection: sessions
{
  _id: ObjectId,
  userId: ObjectId,
  sessionId: String,                // Unique session identifier
  socketId: String,                 // WebSocket connection ID
  deviceInfo: {
    userAgent: String,
    ip: String,
    platform: String,
  },
  lastActivity: Date,
  expiresAt: Date,
  createdAt: Date,
}

// Indexes
db.sessions.createIndex({ userId: 1 })
db.sessions.createIndex({ sessionId: 1 }, { unique: true })
db.sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
```

5.2 Individual Features Schemas

5.2.1 Tasks Collection

```
// Collection: tasks
{
  _id: ObjectId,
  userId: ObjectId,                 // Reference to users

  title: String,
  description: String,

  status: String,                   // "not_started" | "in_progress" | "completed" | "cancelle
  priority: String,                 // "low" | "medium" | "high" | "urgent"
```

```
dueDate: Date,
dueTime: String,                    // "HH:mm" format

estimatedDuration: Number,          // minutes
actualDuration: Number,             // minutes

// Organization
projectId: ObjectId,                // Reference to projects (for enterprise)
tags: [String],
category: String,

// Hierarchy
parentTaskId: ObjectId,             // For subtasks
subtasks: [ObjectId],
checklist: [{
  id: String,
  text: String,
  completed: Boolean,
  order: Number,
}],

// Relationships
goalId: ObjectId,                   // Reference to goals
calendarEventId: ObjectId,          // Reference to calendar_events
dependencies: [{
  taskId: ObjectId,
  type: String,                     // "finish_to_start" | "start_to_start"
}],

// Recurrence
recurring: Boolean,
recurrence: {
  frequency: String,                // "daily" | "weekly" | "monthly"
  interval: Number,
  daysOfWeek: [Number],             // 0-6
  endDate: Date,
  parentTaskId: ObjectId,           // Original recurring task
},

// Reminders
reminders: [{
  type: String,                     // "notification" | "email" | "sms"
  triggerAt: Date,
  sent: Boolean,
}],

// Attachments
attachments: [String],              // File IDs
```

```
  // Metadata
  notes: String,
  effortLevel: String,              // "easy" | "moderate" | "hard"
  context: String,                  // "home" | "office" | "phone" | "errands"
  energyRequired: String,           // "low" | "medium" | "high"

  completedAt: Date,
  completedBy: ObjectId,

  deleted: Boolean,
  deletedAt: Date,

  // Sync
  syncStatus: String,               // "synced" | "pending" | "conflict"
  lastSyncedAt: Date,
  version: Number,                  // For conflict resolution

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.tasks.createIndex({ userId: 1, deleted: 1 })
db.tasks.createIndex({ userId: 1, status: 1 })
db.tasks.createIndex({ userId: 1, dueDate: 1 })
db.tasks.createIndex({ projectId: 1 })
db.tasks.createIndex({ parentTaskId: 1 })
db.tasks.createIndex({ tags: 1 })
```

5.2.2 Calendar Events Collection

```
// Collection: calendar_events
{
  _id: ObjectId,
  userId: ObjectId,

  title: String,
  description: String,

  startDateTime: Date,
  endDateTime: Date,
  allDay: Boolean,

  category: String,                 // "work" | "personal" | "health" | "social"
  color: String,                    // Hex color

  location: {
```

```
  type: String,                    // "physical" | "virtual"
  address: String,
  coordinates: {
    lat: Number,
    lng: Number,
  },
  virtualLink: String,             // Zoom/Meet link
},

// Recurrence
recurring: Boolean,
recurrence: {
  frequency: String,
  interval: Number,
  daysOfWeek: [Number],
  endType: String,                 // "after" | "until" | "never"
  endCount: Number,
  endDate: Date,
  exceptions: [Date],              // Dates to skip
},

// Reminders
reminders: [{
  type: String,
  minutes: Number,                 // Minutes before event
}],

// Relationships
taskId: ObjectId,
linkedGoals: [ObjectId],

// Collaboration (future)
attendees: [{
  userId: ObjectId,
  email: String,
  status: String,                  // "accepted" | "declined" | "tentative"
}],

// Metadata
attachments: [String],
status: String,                    // "scheduled" | "completed" | "cancelled"
visibility: String,                // "private" | "public"
timezone: String,

deleted: Boolean,

syncStatus: String,
version: Number,
```

```
  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.calendar_events.createIndex({ userId: 1, deleted: 1 })
db.calendar_events.createIndex({ userId: 1, startDateTime: 1 })
db.calendar_events.createIndex({ userId: 1, endDateTime: 1 })
```

5.2.3 Finance Transactions Collection

```
// Collection: finance_transactions
{
  _id: ObjectId,
  userId: ObjectId,

  type: String,                     // "expense" | "income" | "transfer"
  amount: Number,
  currency: String,

  category: String,
  subcategory: String,

  date: Date,
  time: String,

  merchant: String,
  paymentMethod: String,            // "cash" | "credit_card" | "debit_card" | "bank_transfer"

  description: String,
  tags: [String],

  // Accounts
  fromAccount: ObjectId,            // For transfers
  toAccount: ObjectId,
  accountId: ObjectId,              // Main account

  // Relationships
  budgetId: ObjectId,
  calendarEventId: ObjectId,        // Business lunch, etc.

  // Details
  receipt: String,                  // File ID
  location: {
    lat: Number,
    lng: Number,
  },
```

```
  recurring: Boolean,
  recurrence: {
    frequency: String,
    interval: Number,
    endDate: Date,
  },

  // Status
  verified: Boolean,
  reconciled: Boolean,

  deleted: Boolean,

  syncStatus: String,
  version: Number,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.finance_transactions.createIndex({ userId: 1, deleted: 1 })
db.finance_transactions.createIndex({ userId: 1, date: -1 })
db.finance_transactions.createIndex({ userId: 1, category: 1 })
db.finance_transactions.createIndex({ accountId: 1 })
```

5.2.4 Finance Accounts Collection

```
// Collection: finance_accounts
{
  _id: ObjectId,
  userId: ObjectId,

  name: String,
  type: String,                      // "cash" | "checking" | "savings" | "credit_card" | "inve

  currency: String,
  initialBalance: Number,
  currentBalance: Number,            // Calculated from transactions

  // Credit Card specific
  creditLimit: Number,

  // Bank Connection (Plaid)
  plaidAccessToken: String,
  plaidItemId: String,
  plaidAccountId: String,
```

```
  bankSyncEnabled: Boolean,
  lastSynced: Date,

  color: String,
  icon: String,

  status: String,                    // "active" | "inactive" | "closed"

  deleted: Boolean,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.finance_accounts.createIndex({ userId: 1, deleted: 1 })
db.finance_accounts.createIndex({ userId: 1, status: 1 })
```

5.2.5 Budgets Collection

```
// Collection: budgets
{
  _id: ObjectId,
  userId: ObjectId,

  name: String,
  category: String,

  budgetType: String,                // "fixed" | "percentage" | "goal"
  amount: Number,
  period: String,                    // "monthly" | "weekly" | "yearly"

  // Alerts
  alertThresholds: [Number],         // [50, 75, 90, 100]

  // Settings
  rollover: Boolean,

  startDate: Date,
  endDate: Date,

  spent: Number,                     // Current period spending
  remaining: Number,

  status: String,                    // "active" | "inactive"

  createdAt: Date,
  updatedAt: Date,
```

```
}

// Indexes
db.budgets.createIndex({ userId: 1 })
db.budgets.createIndex({ userId: 1, category: 1 })
```

5.2.6 Habits Collection

```
// Collection: habits
{
  _id: ObjectId,
  userId: ObjectId,

  name: String,
  description: String,
  icon: String,
  color: String,

  // Frequency
  frequency: {
    type: String,                   // "daily" | "weekly" | "monthly" | "custom"
    target: Number,                 // Times per period
    specificDays: [Number],         // For daily/weekly
    timeOfDay: String,              // "morning" | "afternoon" | "evening" | "anytime"
    preferredTime: String,          // "HH:mm"
  },

  // Tracking
  trackingType: String,             // "checkbox" | "counter" | "duration" | "value"
  unit: String,                     // For value tracking
  targetValue: Number,

  // History
  completions: [{
    date: Date,
    completed: Boolean,
    value: Number,                  // For counter/duration/value types
    note: String,
  }],

  // Streaks
  currentStreak: Number,
  longestStreak: Number,
  totalCompletions: Number,

  // Reminders
  reminders: [{
    time: String,
```

```
    days: [Number],
    enabled: Boolean,
  }],

  // Organization
  category: String,                // "health" | "productivity" | "learning" | "social"
  goalId: ObjectId,
  habitStackId: ObjectId,

  startDate: Date,
  archived: Boolean,
  archivedAt: Date,
  difficulty: String,              // "easy" | "medium" | "hard"

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.habits.createIndex({ userId: 1, archived: 1 })
db.habits.createIndex({ userId: 1, category: 1 })
```

5.2.7 Goals Collection

```
// Collection: goals
{
  _id: ObjectId,
  userId: ObjectId,

  title: String,
  description: String,

  // SMART Framework
  specific: String,
  measurable: {
    metric: String,
    target: Number,
    current: Number,
    unit: String,
  },
  achievable: String,
  relevant: String,
  timeBound: {
    startDate: Date,
    targetDate: Date,
    deadline: Date,
  },
```

```
// Organization
category: String,
priority: String,
status: String,                    // "not_started" | "in_progress" | "completed" | "on_hold"

// Hierarchy
parentGoalId: ObjectId,
subGoals: [ObjectId],

// Milestones
milestones: [{
  id: String,
  title: String,
  targetDate: Date,
  completed: Boolean,
  completedDate: Date,
}],

// Progress
progress: {
  percentage: Number,
  lastUpdated: Date,
  updateFrequency: String,         // "daily" | "weekly" | "monthly"
  calculationType: String,         // "manual" | "automatic"
},

// Connections
linkedTasks: [ObjectId],
linkedHabits: [ObjectId],
linkedEvents: [ObjectId],

requiredResources: [String],

// Check-ins
checkIns: [{
  date: Date,
  progress: Number,
  note: String,
  mood: String,
}],

// Metadata
visibility: String,
tags: [String],
motivationNotes: String,
obstaclesAnticipated: [String],

completedAt: Date,
```

```
  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.goals.createIndex({ userId: 1, status: 1 })
db.goals.createIndex({ userId: 1, category: 1 })
db.goals.createIndex({ parentGoalId: 1 })
```

5.2.8 Time Entries Collection

```
// Collection: time_entries
{
  _id: ObjectId,
  userId: ObjectId,

  type: String,                     // "active" | "manual" | "automatic"

  activity: String,
  description: String,

  startTime: Date,
  endTime: Date,
  duration: Number,                 // seconds

  // Organization
  projectId: ObjectId,
  taskId: ObjectId,

  // Billing
  billable: Boolean,
  hourlyRate: Number,
  totalCost: Number,
  billingStatus: String,            // "not_invoiced" | "invoiced" | "paid"

  tags: [String],

  // Pauses
  pauses: [{
    pausedAt: Date,
    resumedAt: Date,
  }],

  automaticallyStopped: Boolean,
  idleTime: Number,                 // seconds

  deleted: Boolean,
```

```
  syncStatus: String,
  version: Number,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.time_entries.createIndex({ userId: 1, deleted: 1 })
db.time_entries.createIndex({ userId: 1, startTime: -1 })
db.time_entries.createIndex({ projectId: 1 })
db.time_entries.createIndex({ taskId: 1 })
```

5.2.9 Notes Collection

```
// Collection: notes
{
  _id: ObjectId,
  userId: ObjectId,

  title: String,
  content: String,                  // Rich text HTML or Markdown
  contentType: String,              // "richtext" | "markdown" | "checklist"

  // Organization
  notebookId: ObjectId,
  tags: [String],
  category: String,

  // Metadata
  pinned: Boolean,
  starred: Boolean,
  archived: Boolean,
  encrypted: Boolean,

  // Attachments
  attachments: [String],
  images: [String],

  // Connections
  linkedTasks: [ObjectId],
  linkedEvents: [ObjectId],
  linkedNotes: [ObjectId],          // Backlinks

  // Version History
  versions: [{
    version: Number,
```

```
    content: String,
    createdAt: Date,
    createdBy: ObjectId,
  }],

  currentVersion: Number,

  deleted: Boolean,
  deletedAt: Date,

  syncStatus: String,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.notes.createIndex({ userId: 1, deleted: 1 })
db.notes.createIndex({ userId: 1, notebookId: 1 })
db.notes.createIndex({ userId: 1, tags: 1 })
db.notes.createIndex({ title: "text", content: "text" }) // Full-text search
```

5.2.10 Notebooks Collection

```
// Collection: notebooks
{
  _id: ObjectId,
  userId: ObjectId,

  name: String,
  description: String,

  color: String,
  icon: String,

  parentNotebookId: ObjectId,       // For nested notebooks

  status: String,                   // "active" | "archived"

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.notebooks.createIndex({ userId: 1 })
db.notebooks.createIndex({ parentNotebookId: 1 })
```

5.2.11 Health Metrics Collection

```
// Collection: health_metrics
{
  _id: ObjectId,
  userId: ObjectId,

  date: Date,

  // Physical Health
  weight: Number,
  bodyFat: Number,
  sleep: {
    duration: Number,               // hours
    quality: Number,                // 1-10
    deepSleep: Number,
    remSleep: Number,
  },
  exercise: [{
    type: String,
    duration: Number,
    intensity: String,
    caloriesBurned: Number,
  }],
  water: Number,                    // liters
  nutrition: {
    calories: Number,
    protein: Number,
    carbs: Number,
    fat: Number,
  },
  vitals: {
    heartRate: Number,
    bloodPressure: {
      systolic: Number,
      diastolic: Number,
    },
  },
  steps: Number,

  // Mental Wellness
  mood: String,
  moodScore: Number,                // 1-10
  energy: Number,                   // 1-10
  stress: Number,                   // 1-10
  anxiety: Number,                  // 1-10
  mindfulnessMinutes: Number,
```

```
  // Habits
  caffeine: Number,
  alcohol: Number,
  screenTime: Number,
  outdoorTime: Number,

  notes: String,
  photos: [String],

  // Data Sources
  source: String,                    // "manual" | "apple_health" | "google_fit" | "fitbit"

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.health_metrics.createIndex({ userId: 1, date: -1 })
```

5.2.12 Journal Entries Collection

```
// Collection: journal_entries
{
  _id: ObjectId,
  userId: ObjectId,

  date: Date,

  // Content
  content: String,

  // Daily Prompts
  whatWentWell: String,
  whatCouldBeBetter: String,
  gratitude: [String],               // 3 things grateful for
  wins: [String],
  lessons: [String],
  tomorrowIntentions: String,

  // Mood
  mood: String,
  moodScore: Number,                 // 1-10
  moodContext: [String],             // What influenced mood

  // Media
  photos: [String],

  // Metadata
```

```
  weather: String,
  location: String,

  template: String,                  // Template used

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.journal_entries.createIndex({ userId: 1, date: -1 })
```

5.3 Enterprise Features Schemas

5.3.1 Enterprises Collection

```
// Collection: enterprises
{
  _id: ObjectId,

  // Company Info
  name: String,
  legalName: String,
  taxId: String,

  industry: String,
  size: String,                      // "1-10" | "11-50" | "51-200" | "201-500" | "500+"

  // Contact
  email: String,
  phone: String,
  website: String,

  // Address
  address: {
    street: String,
    city: String,
    state: String,
    zipCode: String,
    country: String,
  },

  // Branding
  logo: String,
  primaryColor: String,

  // Subscription
```

```
  plan: String,                     // "free" | "starter" | "professional" | "enterprise"
  billingCycle: String,             // "monthly" | "annual"

  features: [{
    name: String,
    enabled: Boolean,
    tier: String,
  }],

  // Limits
  maxUsers: Number,
  maxProjects: Number,
  storageQuota: Number,             // GB
  storageUsed: Number,

  // Settings
  settings: {
    timezone: String,
    currency: String,
    fiscalYearStart: Number,        // Month (1-12)
    weekStart: Number,
  },

  // Billing
  stripeCustomerId: String,
  stripeSubscriptionId: String,

  // Owner
  ownerId: ObjectId,                // Reference to users

  status: String,                   // "active" | "suspended" | "trial" | "cancelled"
  trialEndsAt: Date,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.enterprises.createIndex({ ownerId: 1 })
db.enterprises.createIndex({ status: 1 })
```

5.3.2 Enterprise Members Collection

```
// Collection: enterprise_members
{
  _id: ObjectId,
  enterpriseId: ObjectId,
  userId: ObjectId,
```

```
  role: String,                     // "owner" | "admin" | "manager" | "member"
  title: String,                    // Job title
  department: String,

  permissions: [String],            // Specific permissions

  // Employment
  employeeNumber: String,
  hireDate: Date,
  employmentType: String,           // "full_time" | "part_time" | "contractor"

  // Compensation
  salary: Number,
  hourlyRate: Number,
  currency: String,

  // Time Off
  vacationDays: Number,
  sickDays: Number,
  vacationBalance: Number,
  sickBalance: Number,

  status: String,                   // "active" | "inactive" | "terminated"

  invitedAt: Date,
  joinedAt: Date,
  terminatedAt: Date,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.enterprise_members.createIndex({ enterpriseId: 1 })
db.enterprise_members.createIndex({ userId: 1 })
db.enterprise_members.createIndex({ enterpriseId: 1, status: 1 })
```

5.3.3 Projects Collection

```
// Collection: projects
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  projectCode: String,
  name: String,
  description: String,
```

```
// Organization
portfolioId: ObjectId,
clientId: ObjectId,

// Methodology
methodology: String,              // "agile" | "waterfall" | "hybrid"
framework: String,                // "scrum" | "kanban" | "custom"

// Team
projectManagerId: ObjectId,
team: [{
  userId: ObjectId,
  role: String,
  allocation: Number,             // percentage
  hourlyRate: Number,
}],

// Timeline
startDate: Date,
endDate: Date,
status: String,                   // "planning" | "active" | "on_hold" | "completed" | "canc

// Budget
budgetType: String,               // "fixed" | "time_and_materials"
estimatedHours: Number,
actualHours: Number,
budgetAmount: Number,
actualCost: Number,

// Progress
progress: Number,
tasksTotal: Number,
tasksCompleted: Number,

// Milestones
milestones: [{
  id: String,
  name: String,
  dueDate: Date,
  completed: Boolean,
  completedDate: Date,
}],

// Sprints (for Agile)
sprints: [{
  id: String,
  sprintNumber: Number,
```

```
    name: String,
    startDate: Date,
    endDate: Date,
    goal: String,
    status: String,
    totalStoryPoints: Number,
    completedStoryPoints: Number,
  }],

  // Settings
  visibility: String,               // "team" | "company" | "client"
  archived: Boolean,

  tags: [String],
  customFields: {},

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.projects.createIndex({ enterpriseId: 1, archived: 1 })
db.projects.createIndex({ enterpriseId: 1, status: 1 })
db.projects.createIndex({ clientId: 1 })
```

5.3.4 Inventory Products Collection

```
// Collection: inventory_products
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  sku: String,                      // Unique within enterprise
  name: String,
  description: String,

  // Classification
  category: String,
  subcategory: String,
  brand: String,
  tags: [String],

  // Variants
  hasVariants: Boolean,
  variants: [{
    id: String,
    sku: String,
    attributes: {},                 // { color: "Black", size: "L" }
```

```
  barcode: String,
  stockLevel: Number,
}],

// Inventory Tracking
trackInventory: Boolean,
inventoryMethod: String,          // "FIFO" | "LIFO" | "FEFO" | "Average"
stockLevel: Number,               // Total across variants/locations
reservedStock: Number,
availableStock: Number,
reorderPoint: Number,
reorderQuantity: Number,
safetyStock: Number,

// Multi-location
stockByLocation: [{
  locationId: ObjectId,
  quantity: Number,
}],

// Batches/Lots
batchTracking: Boolean,
batches: [{
  batchNumber: String,
  quantity: Number,
  manufacturingDate: Date,
  expiryDate: Date,
  locationId: ObjectId,
  cost: Number,
}],

// Pricing
costPrice: Number,
sellingPrice: Number,
wholesalePrice: Number,
margin: Number,
currency: String,
taxable: Boolean,
taxRate: Number,

// Physical Properties
weight: Number,
dimensions: {
  length: Number,
  width: Number,
  height: Number,
},
```

```
  // Supplier Info
  primarySupplierId: ObjectId,
  supplierSKU: String,
  leadTime: Number,                  // days
  moq: Number,                       // minimum order quantity

  // Media
  images: [String],
  documents: [String],

  status: String,                    // "active" | "discontinued" | "out_of_stock"
  lastRestocked: Date,

  createdBy: ObjectId,
  lastModifiedBy: ObjectId,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.inventory_products.createIndex({ enterpriseId: 1, status: 1 })
db.inventory_products.createIndex({ enterpriseId: 1, sku: 1 }, { unique: true })
db.inventory_products.createIndex({ category: 1 })
```

5.3.5 Stock Movements Collection

```
// Collection: stock_movements
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  type: String,                      // "adjustment" | "received" | "sold" | "transferred" | "r
  productId: ObjectId,
  variantId: String,
  batchNumber: String,

  quantity: Number,
  quantityBefore: Number,
  quantityAfter: Number,

  fromLocationId: ObjectId,
  toLocationId: ObjectId,

  date: Date,
  reason: String,
  notes: String,
```

```
  relatedDocument: {
    type: String,                   // "purchase_order" | "sale" | "transfer" | "adjustment"
    id: ObjectId,
  },

  costPerUnit: Number,
  totalCost: Number,

  performedBy: ObjectId,
  approvedBy: ObjectId,

  attachments: [String],

  createdAt: Date,
}

// Indexes
db.stock_movements.createIndex({ enterpriseId: 1, date: -1 })
db.stock_movements.createIndex({ productId: 1 })
```

5.3.6 Orders Collection

```
// Collection: orders
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  orderNumber: String,              // Unique
  type: String,                     // "sales_order" | "purchase_order"

  // Customer Info
  customerId: ObjectId,
  customerName: String,
  customerEmail: String,
  customerPhone: String,
  customerType: String,             // "retail" | "wholesale" | "b2b"

  // Order Items
  items: [{
    productId: ObjectId,
    variantId: String,
    sku: String,
    name: String,
    quantity: Number,
    unitPrice: Number,
    discount: Number,
    tax: Number,
    total: Number,
```

```
}],

// Pricing
subtotal: Number,
discountTotal: Number,
taxTotal: Number,
shippingCost: Number,
totalAmount: Number,
currency: String,

// Addresses
billingAddress: {
  street: String,
  city: String,
  state: String,
  zipCode: String,
  country: String,
},
shippingAddress: {},              // Same structure

// Fulfillment
fulfillmentStatus: String,        // "pending" | "processing" | "packed" | "shipped" | "deli
paymentStatus: String,            // "pending" | "authorized" | "paid" | "refunded" | "faile

warehouseId: ObjectId,
shippingMethod: String,
carrier: String,
trackingNumber: String,

// Dates
orderDate: Date,
expectedShipDate: Date,
expectedDeliveryDate: Date,
actualShipDate: Date,
actualDeliveryDate: Date,

// Payment
paymentMethod: String,
paymentReference: String,

// Source
channel: String,                  // "website" | "mobile_app" | "in_store" | "phone"
salesPersonId: ObjectId,

// Notes
customerNotes: String,
internalNotes: String,
```

```
  // Related
  invoiceId: ObjectId,
  shipmentId: ObjectId,

  // Status History
  statusHistory: [{
    status: String,
    timestamp: Date,
    userId: ObjectId,
  }],

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.orders.createIndex({ enterpriseId: 1, orderNumber: 1 }, { unique: true })
db.orders.createIndex({ enterpriseId: 1, fulfillmentStatus: 1 })
db.orders.createIndex({ enterpriseId: 1, orderDate: -1 })
db.orders.createIndex({ customerId: 1 })
```

5.3.7 CRM Contacts Collection

```
// Collection: crm_contacts
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  type: String,                      // "lead" | "contact" | "customer"

  // Personal Info
  firstName: String,
  lastName: String,
  email: String,
  phone: String,
  mobile: String,

  // Company (B2B)
  companyId: ObjectId,
  companyName: String,
  position: String,
  department: String,

  // Lead Source
  source: String,                    // "website" | "referral" | "cold_call" | "event"
  sourceDetails: String,

  // Status
```

```
  status: String,                    // "new" | "contacted" | "qualified" | "unqualified" | "cu
  leadScore: Number,                 // 0-100

  // Assignment
  ownerId: ObjectId,
  teamId: ObjectId,

  // Relationships
  deals: [ObjectId],
  orders: [ObjectId],
  invoices: [ObjectId],

  // Communication
  lastContactedDate: Date,
  nextFollowUpDate: Date,

  // Preferences
  communicationPreference: String,  // "email" | "phone" | "sms"
  doNotContact: Boolean,
  emailOptIn: Boolean,

  // Custom Fields
  customFields: {},

  // Social
  linkedin: String,
  twitter: String,

  tags: [String],
  notes: String,

  createdBy: ObjectId,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.crm_contacts.createIndex({ enterpriseId: 1, type: 1 })
db.crm_contacts.createIndex({ enterpriseId: 1, status: 1 })
db.crm_contacts.createIndex({ ownerId: 1 })
db.crm_contacts.createIndex({ email: 1 })
```

5.3.8 CRM Deals Collection

```
// Collection: crm_deals
{
  _id: ObjectId,
```

```
enterpriseId: ObjectId,

dealName: String,

contactId: ObjectId,
companyId: ObjectId,

// Pipeline
pipelineId: ObjectId,
stageId: ObjectId,
stageHistory: [{
  stage: String,
  enteredDate: Date,
  exitDate: Date,
}],

// Value
dealValue: Number,
currency: String,

// Probability
probability: Number,              // percentage
expectedRevenue: Number,          // dealValue × probability

// Timeline
expectedCloseDate: Date,
actualCloseDate: Date,

// Status
status: String,                   // "open" | "won" | "lost"
lostReason: String,

// Assignment
ownerId: ObjectId,

// Products
products: [{
  productId: ObjectId,
  quantity: Number,
  unitPrice: Number,
  discount: Number,
  total: Number,
}],

// Activities
nextActivityType: String,
nextActivityDate: Date,
nextActivityDescription: String,
```

```
  competitors: [String],

  notes: String,
  source: String,

  createdDate: Date,
  lastActivityDate: Date,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.crm_deals.createIndex({ enterpriseId: 1, status: 1 })
db.crm_deals.createIndex({ enterpriseId: 1, stageId: 1 })
db.crm_deals.createIndex({ ownerId: 1 })
```

5.3.9 Invoices Collection

```
// Collection: invoices
{
  _id: ObjectId,
  enterpriseId: ObjectId,

  invoiceNumber: String,

  customerId: ObjectId,

  invoiceDate: Date,
  dueDate: Date,

  lineItems: [{
    description: String,
    quantity: Number,
    unitPrice: Number,
    amount: Number,
    taxable: Boolean,
  }],

  subtotal: Number,
  taxRate: Number,
  taxAmount: Number,
  discount: Number,
  total: Number,
  currency: String,

  paymentTerms: String,             // "net_30" | "net_60" | "due_on_receipt"
```

```
  status: String,                    // "draft" | "sent" | "viewed" | "partial" | "paid" | "ove

  paymentsMade: [{
    date: Date,
    amount: Number,
    method: String,
    reference: String,
  }],

  amountPaid: Number,
  amountDue: Number,

  sentDate: Date,
  viewedDate: Date,
  paidDate: Date,

  reminders: [{
    sentDate: Date,
    type: String,                    // "upcoming" | "overdue"
  }],

  linkedOrderId: ObjectId,
  linkedProjectId: ObjectId,

  notes: String,
  terms: String,

  attachments: [String],

  createdBy: ObjectId,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.invoices.createIndex({ enterpriseId: 1, invoiceNumber: 1 }, { unique: true })
db.invoices.createIndex({ enterpriseId: 1, status: 1 })
db.invoices.createIndex({ enterpriseId: 1, dueDate: 1 })
db.invoices.createIndex({ customerId: 1 })
```

5.3.10 Time Clock Collection

```
// Collection: time_clock
{
  _id: ObjectId,
  enterpriseId: ObjectId,
```

```
  employeeId: ObjectId,

  clockInTime: Date,
  clockOutTime: Date,

  clockInMethod: String,            // "web" | "mobile" | "kiosk" | "manual"
  clockInLocation: {
    lat: Number,
    lng: Number,
    address: String,
  },

  breakStart: Date,
  breakEnd: Date,

  totalHours: Number,
  regularHours: Number,
  overtimeHours: Number,
  breakTime: Number,

  scheduledShiftId: ObjectId,
  late: Boolean,
  earlyLeave: Boolean,

  workLocation: String,             // "office" | "remote" | "client_site"
  projectId: ObjectId,

  notes: String,

  approved: Boolean,
  approvedBy: ObjectId,
  approvedDate: Date,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.time_clock.createIndex({ enterpriseId: 1, employeeId: 1, clockInTime: -1 })
db.time_clock.createIndex({ enterpriseId: 1, approved: 1 })
```

5.3.11 Vendors Collection

```
// Collection: vendors
{
  _id: ObjectId,
  enterpriseId: ObjectId,
```

```
  vendorNumber: String,
  name: String,

  contactInfo: {
    email: String,
    phone: String,
    website: String,
    address: {
      street: String,
      city: String,
      state: String,
      zipCode: String,
      country: String,
    },
  },

  category: String,

  paymentTerms: String,
  preferredPaymentMethod: String,

  taxId: String,

  rating: Number,                   // 1-5

  contracts: [ObjectId],

  products: [ObjectId],             // Products they supply

  performance: {
    onTimeDeliveryRate: Number,     // percentage
    qualityScore: Number,
    averageResponseTime: String,
  },

  totalPurchases: Number,

  status: String,                   // "active" | "inactive" | "blocked"

  notes: String,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.vendors.createIndex({ enterpriseId: 1, status: 1 })
db.vendors.createIndex({ enterpriseId: 1, vendorNumber: 1 }, { unique: true })
```

5.4 Shared Schemas

5.4.1 Files Collection

```
// Collection: files
{
  _id: ObjectId,
  userId: ObjectId,
  enterpriseId: ObjectId,           // Optional for enterprise files

  filename: String,
  originalFilename: String,
  mimeType: String,
  size: Number,                     // bytes

  // Storage
  storageProvider: String,          // "s3" | "gcs" | "azure"
  storagePath: String,
  storageKey: String,
  url: String,                      // CDN URL

  // Organization
  folderId: ObjectId,

  // Metadata
  tags: [String],
  description: String,

  // Version Control
  version: Number,
  parentFileId: ObjectId,           // Previous version

  // Access
  visibility: String,               // "private" | "team" | "public"
  sharedWith: [{
    userId: ObjectId,
    permission: String,             // "view" | "edit"
  }],

  // Virus Scan
  scanned: Boolean,
  scanResult: String,               // "clean" | "infected"

  uploadedBy: ObjectId,

  deleted: Boolean,
```

```
  deletedAt: Date,

  createdAt: Date,
  updatedAt: Date,
}

// Indexes
db.files.createIndex({ userId: 1, deleted: 1 })
db.files.createIndex({ enterpriseId: 1, deleted: 1 })
db.files.createIndex({ folderId: 1 })
```

5.4.2 Notifications Collection

```
// Collection: notifications
{
  _id: ObjectId,
  userId: ObjectId,

  type: String,                      // "task_due" | "payment_reminder" | "team_mention"
  title: String,
  message: String,

  // Related Entity
  relatedEntity: {
    type: String,                    // "task" | "event" | "invoice"
    id: ObjectId,
  },

  // Delivery
  channels: {
    inApp: Boolean,
    email: Boolean,
    push: Boolean,
    sms: Boolean,
  },

  // Status
  read: Boolean,
  readAt: Date,

  sentAt: Date,

  // Actions
  actions: [{
    label: String,
    url: String,
    action: String,
  }],
```

```
  expiresAt: Date,

  createdAt: Date,
}

// Indexes
db.notifications.createIndex({ userId: 1, read: 1, createdAt: -1 })
db.notifications.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
```

5.4.3 Activity Log Collection

```
// Collection: activity_log
{
  _id: ObjectId,
  userId: ObjectId,
  enterpriseId: ObjectId,

  action: String,                   // "created" | "updated" | "deleted"
  entityType: String,               // "task" | "project" | "order"
  entityId: ObjectId,

  changes: {                        // What changed
    field: String,
    oldValue: Mixed,
    newValue: Mixed,
  },

  ipAddress: String,
  userAgent: String,

  timestamp: Date,
}

// Indexes
db.activity_log.createIndex({ userId: 1, timestamp: -1 })
db.activity_log.createIndex({ enterpriseId: 1, timestamp: -1 })
db.activity_log.createIndex({ entityType: 1, entityId: 1 })
```

5.5 Indexing Strategy

5.5.1 Index Principles

1. Query Patterns: - Index fields used in WHERE clauses - Index fields used in sorting (ORDER BY) - Compound indexes for multi-field queries

2. Index Types: - Single Field: Most common queries - Compound: Multi-field query optimization - Text: Full-text search - Geospatial: Location-based queries - TTL: Auto-delete expired documents

3. Index Optimization:

```
// Good: Compound index covers query
db.tasks.createIndex({ userId: 1, status: 1, dueDate: 1 })

// Query: Find user's active tasks due this week
db.tasks.find({
  userId: "123",
  status: "in_progress",
  dueDate: { $gte: startOfWeek, $lte: endOfWeek }
})

// Bad: Too many indexes slow writes
// Only create indexes for frequent query patterns
```

5.5.2 Critical Indexes

High-Priority Indexes:

```
// Users
db.users.createIndex({ email: 1 }, { unique: true })

// Tasks
db.tasks.createIndex({ userId: 1, deleted: 1, status: 1 })
db.tasks.createIndex({ userId: 1, dueDate: 1 })

// Calendar Events
db.calendar_events.createIndex({ userId: 1, startDateTime: 1 })

// Orders
db.orders.createIndex({ enterpriseId: 1, orderDate: -1 })
db.orders.createIndex({ enterpriseId: 1, fulfillmentStatus: 1 })

// Projects
db.projects.createIndex({ enterpriseId: 1, status: 1 })
```

5.6 Data Migration Strategy

5.6.1 Migration Framework

Tool: migrate-mongo

```
// migrations/20260119-add-sync-fields.js
module.exports = {
  async up(db) {
    await db.collection('tasks').updateMany(
      { syncStatus: { $exists: false } },
      {
        $set: {
```

```
          syncStatus: 'synced',
          version: 1,
          lastSyncedAt: new Date()
        }
      }
    )
  },

  async down(db) {
    await db.collection('tasks').updateMany(
      {},
      {
        $unset: {
          syncStatus: '',
          version: '',
          lastSyncedAt: ''
        }
      }
    )
  }
}
```

5.6.2 Schema Versioning

Document Version Field:

```
{
  _id: ObjectId,
  schemaVersion: 2,                  // Track schema version
  // ... rest of document
}
```

Migration Strategy: 1. Add new fields with default values 2. Mark old fields as deprecated 3. Run background migration job 4. Remove old fields after migration complete

5.6.3 Data Validation

MongoDB Schema Validation:

```
db.createCollection("tasks", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["userId", "title", "status"],
      properties: {
        userId: {
          bsonType: "objectId",
          description: "must be an objectId and is required"
        },
        title: {
```

```
          bsonType: "string",
          description: "must be a string and is required"
        },
        status: {
          enum: ["not_started", "in_progress", "completed", "cancelled"],
          description: "must be one of the allowed values"
        }
      }
    }
  }
})
```

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Database Team - Status: Draft - Database Design Phase

Next Chapter: Chapter 6: API Specifications

Chapter 6: API Specifications

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - API Design Phase

Table of Contents

- 6.1 API Overview & Conventions
- 6.2 Authentication APIs
- 6.3 Individual Feature APIs
- 6.4 Enterprise Feature APIs
- 6.5 Payment & Subscription APIs
- 6.6 WebSocket Events
- 6.7 Error Handling

6.1 API Overview & Conventions

6.1.1 Base URLs

```
Production:  https://api.prms.io/v1
Staging:     https://api-staging.prms.io/v1
Development: http://localhost:3000/api/v1
```

6.1.2 API Standards

Request Format: - Content-Type: `application/json` - Character Encoding: UTF-8 - Date Format: ISO 8601 (`2026-01-19T10:30:00Z`)

Response Format:

```
// Success Response
{
  success: true,
  data: {}, // or []
  meta?: {
    pagination?: {
      page: number,
      limit: number,
      total: number,
      totalPages: number
    }
  }
}

// Error Response
{
  success: false,
  error: {
    code: string,
    message: string,
    details?: any
  }
}
```

Pagination:

```
GET /api/v1/tasks?page=1&limit=20&sortBy=dueDate&order=asc
```

Filtering:

```
GET /api/v1/tasks?status=in_progress&priority=high&tags=urgent
```

Field Selection:

```
GET /api/v1/tasks?fields=title,status,dueDate
```

6.1.3 Rate Limiting

Limits by Tier: - Free: 100 requests/minute - Premium: 500 requests/minute - Enterprise: 2000 requests/minute

Headers:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1705663200
```

6.1.4 Versioning

- Version specified in URL: `/v1/`, `/v2/`
- Breaking changes require new version

- Old versions supported for 12 months

6.2 Authentication APIs

6.2.1 Register

Endpoint: `POST /auth/register`

Request:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "firstName": "John",
  "lastName": "Doe",
  "mode": "individual"
}
```

Response: `201 Created`

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "email": "user@example.com",
      "firstName": "John",
      "lastName": "Doe",
      "mode": "individual",
      "emailVerified": false
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "8f9a7b6c5d4e3f2a1b0c9d8e7f6a5b4c",
      "expiresIn": 3600
    }
  }
}
```

Validation Rules: - Email: Valid format, unique - Password: Min 8 chars, 1 uppercase, 1 number, 1 special char - Names: Required, 2-50 characters

6.2.2 Login

Endpoint: `POST /auth/login`

Request:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
```

```
  "deviceInfo": {
    "platform": "web",
    "userAgent": "Mozilla/5.0...",
    "deviceId": "unique-device-id"
  }
}
```

Response: `200 OK`

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "email": "user@example.com",
      "firstName": "John",
      "lastName": "Doe",
      "avatar": "https://cdn.prms.io/avatars/...",
      "subscriptions": [
        {
          "feature": "finance_manager",
          "tier": "premium",
          "status": "active"
        }
      ]
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "8f9a7b6c5d4e3f2a1b0c9d8e7f6a5b4c",
      "expiresIn": 3600
    }
  }
}
```

Errors: - `401`: Invalid credentials - `423`: Account locked (too many failed attempts)

6.2.3 Refresh Token

Endpoint: `POST /auth/refresh`

Request:

```
{
  "refreshToken": "8f9a7b6c5d4e3f2a1b0c9d8e7f6a5b4c"
}
```

Response: `200 OK`

```
{
  "success": true,
  "data": {
```

```
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 3600
  }
}
```

6.2.4 Logout

Endpoint: `POST /auth/logout`

Headers: `Authorization: Bearer <token>`

Request:

```
{
  "refreshToken": "8f9a7b6c5d4e3f2a1b0c9d8e7f6a5b4c"
}
```

Response: `204 No Content`

6.2.5 Password Reset

Request Reset:

```
POST /auth/password/reset-request
{
  "email": "user@example.com"
}
```

Response: `200 OK`

```
{
  "success": true,
  "message": "Password reset email sent"
}
```

Confirm Reset:

```
POST /auth/password/reset-confirm
{
  "token": "reset-token-from-email",
  "newPassword": "NewSecurePass123!"
}
```

6.3 Individual Feature APIs

6.3.1 Tasks API

Create Task

```
POST /tasks
Authorization: Bearer <token>

{
```

```
  "title": "Complete project proposal",
  "description": "Draft and submit Q1 proposal",
  "status": "not_started",
  "priority": "high",
  "dueDate": "2026-01-25T17:00:00Z",
  "estimatedDuration": 120,
  "tags": ["work", "urgent"],
  "category": "projects"
}

Response: 201 Created
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "title": "Complete project proposal",
    "status": "not_started",
    "priority": "high",
    "dueDate": "2026-01-25T17:00:00Z",
    "createdAt": "2026-01-19T10:30:00Z",
    "updatedAt": "2026-01-19T10:30:00Z"
  }
}
```

Get Tasks

```
GET /tasks?status=in_progress&page=1&limit=20&sortBy=dueDate&order=asc

Response: 200 OK
{
  "success": true,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "title": "Complete project proposal",
      "status": "in_progress",
      "priority": "high",
      "dueDate": "2026-01-25T17:00:00Z"
    }
  ],
  "meta": {
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 45,
      "totalPages": 3
    }
  }
}
```

Update Task

```
PATCH /tasks/:id
```

```
{
  "status": "completed",
  "actualDuration": 150
}
```

```
Response: 200 OK
```

Delete Task

```
DELETE /tasks/:id
```

```
Response: 204 No Content
```

Bulk Operations

```
POST /tasks/bulk
{
  "action": "update",
  "taskIds": ["id1", "id2", "id3"],
  "updates": {
    "status": "completed"
  }
}
```

6.3.2 Calendar Events API

Create Event

```
POST /calendar/events
```

```
{
  "title": "Team Meeting",
  "description": "Weekly sync",
  "startDateTime": "2026-01-20T14:00:00Z",
  "endDateTime": "2026-01-20T15:00:00Z",
  "category": "work",
  "location": {
    "type": "virtual",
    "virtualLink": "https://meet.google.com/abc-defg-hij"
  },
  "reminders": [
    {
      "type": "notification",
      "minutes": 15
    }
  ]
}
```

Get Events by Date Range

```
GET /calendar/events?startDate=2026-01-20&endDate=2026-01-27

Response: 200 OK
{
  "success": true,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "title": "Team Meeting",
      "startDateTime": "2026-01-20T14:00:00Z",
      "endDateTime": "2026-01-20T15:00:00Z",
      "category": "work"
    }
  ]
}
```

Create Recurring Event

```
POST /calendar/events

{
  "title": "Daily Standup",
  "startDateTime": "2026-01-20T09:00:00Z",
  "endDateTime": "2026-01-20T09:15:00Z",
  "recurring": true,
  "recurrence": {
    "frequency": "daily",
    "interval": 1,
    "daysOfWeek": [1, 2, 3, 4, 5],
    "endType": "until",
    "endDate": "2026-06-30T00:00:00Z"
  }
}
```

6.3.3 Finance API

Create Transaction

```
POST /finance/transactions

{
  "type": "expense",
  "amount": 42.50,
  "currency": "USD",
  "category": "groceries",
  "date": "2026-01-19T12:30:00Z",
  "merchant": "Whole Foods",
  "paymentMethod": "credit_card",
```

```
  "accountId": "507f1f77bcf86cd799439011",
  "description": "Weekly groceries",
  "tags": ["food", "essential"]
}

Response: 201 Created
```

Get Transactions

```
GET /finance/transactions?startDate=2026-01-01&endDate=2026-01-31&category=groceries

Response: 200 OK
{
  "success": true,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "type": "expense",
      "amount": 42.50,
      "category": "groceries",
      "date": "2026-01-19T12:30:00Z"
    }
  ],
  "meta": {
    "summary": {
      "totalIncome": 5000.00,
      "totalExpenses": 2342.50,
      "netIncome": 2657.50
    }
  }
}
```

Get Analytics

```
GET /finance/analytics?period=monthly&year=2026&month=1

Response: 200 OK
{
  "success": true,
  "data": {
    "period": "2026-01",
    "totalIncome": 5000.00,
    "totalExpenses": 2342.50,
    "netIncome": 2657.50,
    "expensesByCategory": {
      "groceries": 450.00,
      "transportation": 200.00,
      "utilities": 150.00
    },
    "savingsRate": 53.15,
```

```
    "budgetComparison": {
      "groceries": {
        "budget": 500.00,
        "spent": 450.00,
        "remaining": 50.00,
        "percentage": 90
      }
    }
  }
}
```

Create Budget

```
POST /finance/budgets

{
  "name": "Monthly Groceries",
  "category": "groceries",
  "budgetType": "fixed",
  "amount": 500.00,
  "period": "monthly",
  "alertThresholds": [50, 75, 90, 100],
  "rollover": false
}
```

6.3.4 Habits API

Create Habit

```
POST /habits

{
  "name": "Morning Meditation",
  "description": "10 minutes of mindfulness",
  "icon": "meditation",
  "color": "#4A90E2",
  "frequency": {
    "type": "daily",
    "target": 1,
    "timeOfDay": "morning",
    "preferredTime": "06:30"
  },
  "trackingType": "duration",
  "targetValue": 10,
  "unit": "minutes",
  "category": "health",
  "reminders": [
    {
      "time": "06:30",
```

```
      "days": [1, 2, 3, 4, 5, 6, 0],
      "enabled": true
    }
  ]
}
```

Log Habit Completion

```
POST /habits/:id/completions

{
  "date": "2026-01-19T06:35:00Z",
  "completed": true,
  "value": 12,
  "note": "Felt very focused today"
}

Response: 200 OK
{
  "success": true,
  "data": {
    "habitId": "507f1f77bcf86cd799439011",
    "currentStreak": 15,
    "longestStreak": 23,
    "completionRate": 87.5
  }
}
```

Get Habit Statistics

```
GET /habits/:id/stats?period=30days

Response: 200 OK
{
  "success": true,
  "data": {
    "habit": {
      "id": "507f1f77bcf86cd799439011",
      "name": "Morning Meditation"
    },
    "stats": {
      "currentStreak": 15,
      "longestStreak": 23,
      "totalCompletions": 26,
      "completionRate": 86.67,
      "averageValue": 11.5,
      "consistency": "excellent"
    },
    "history": [
      {
```

```
        "date": "2026-01-19",
        "completed": true,
        "value": 12
      }
    ]
  }
}
```

6.3.5 Goals API

Create Goal

```
POST /goals
```

```
{
  "title": "Launch SaaS Product",
  "description": "Build and launch PrMS to 1000 users",
  "specific": "Launch productivity app with 1000 active users",
  "measurable": {
    "metric": "Active Users",
    "target": 1000,
    "current": 0,
    "unit": "users"
  },
  "timeBound": {
    "startDate": "2026-01-01T00:00:00Z",
    "targetDate": "2026-06-30T00:00:00Z"
  },
  "category": "business",
  "priority": "high",
  "milestones": [
    {
      "title": "MVP Launch",
      "targetDate": "2026-03-01T00:00:00Z"
    },
    {
      "title": "Beta Release",
      "targetDate": "2026-04-15T00:00:00Z"
    },
    {
      "title": "Public Launch",
      "targetDate": "2026-05-30T00:00:00Z"
    }
  ]
}
```

Update Goal Progress

```
PATCH /goals/:id/progress
```

```
{
  "measurable": {
    "current": 250
  },
  "checkIn": {
    "date": "2026-01-19T00:00:00Z",
    "progress": 25,
    "note": "Great progress this week",
    "mood": "motivated"
  }
}
```

6.3.6 Time Tracking API

Start Timer

```
POST /time-tracking/start

{
  "activity": "Development",
  "description": "Working on user authentication",
  "projectId": "507f1f77bcf86cd799439011",
  "taskId": "507f1f77bcf86cd799439012",
  "billable": true,
  "hourlyRate": 75.00
}

Response: 201 Created
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439013",
    "startTime": "2026-01-19T10:30:00Z",
    "running": true
  }
}
```

Stop Timer

```
POST /time-tracking/:id/stop

Response: 200 OK
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439013",
    "startTime": "2026-01-19T10:30:00Z",
    "endTime": "2026-01-19T12:45:00Z",
```

```
    "duration": 8100,
    "totalCost": 168.75
  }
}
```

Get Time Reports

```
GET /time-tracking/reports?startDate=2026-01-01&endDate=2026-01-31&groupBy=project

Response: 200 OK
{
  "success": true,
  "data": {
    "period": {
      "start": "2026-01-01T00:00:00Z",
      "end": "2026-01-31T23:59:59Z"
    },
    "totalHours": 152.5,
    "billableHours": 120.0,
    "totalRevenue": 9000.00,
    "byProject": [
      {
        "projectId": "507f1f77bcf86cd799439011",
        "projectName": "Client Website",
        "hours": 45.5,
        "revenue": 3412.50
      }
    ]
  }
}
```

6.3.7 Notes API

Create Note

```
POST /notes

{
  "title": "Meeting Notes - Q1 Planning",
  "content": "<h1>Q1 Objectives</h1><ul><li>Launch feature X</li></ul>",
  "contentType": "richtext",
  "notebookId": "507f1f77bcf86cd799439011",
  "tags": ["meeting", "planning", "q1"],
  "linkedTasks": ["507f1f77bcf86cd799439012"]
}
```

Search Notes

```
GET /notes/search?q=project+planning&tags=meeting
```

```
Response: 200 OK
{
  "success": true,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "title": "Meeting Notes - Q1 Planning",
      "snippet": "...project planning session where we discussed...",
      "tags": ["meeting", "planning", "q1"],
      "updatedAt": "2026-01-19T10:30:00Z"
    }
  ]
}
```

6.3.8 Health & Wellness API

Log Health Metrics

```
POST /health/metrics

{
  "date": "2026-01-19T00:00:00Z",
  "weight": 75.5,
  "sleep": {
    "duration": 7.5,
    "quality": 8,
    "deepSleep": 1.8,
    "remSleep": 1.2
  },
  "exercise": [
    {
      "type": "running",
      "duration": 30,
      "intensity": "moderate",
      "caloriesBurned": 300
    }
  ],
  "water": 2.5,
  "mood": "energetic",
  "moodScore": 8,
  "stress": 3,
  "steps": 8500
}
```

Get Health Trends

```
GET /health/trends?metric=weight&period=30days

Response: 200 OK
```

```
{
  "success": true,
  "data": {
    "metric": "weight",
    "unit": "kg",
    "current": 75.5,
    "change": -2.3,
    "changePercentage": -2.96,
    "trend": "decreasing",
    "dataPoints": [
      {
        "date": "2026-01-19",
        "value": 75.5
      }
    ]
  }
}
```

6.3.9 Journal API

Create Entry

```
POST /journal/entries
```

```
{
  "date": "2026-01-19T00:00:00Z",
  "content": "Today was productive...",
  "whatWentWell": "Completed all planned tasks",
  "whatCouldBeBetter": "Could have started earlier",
  "gratitude": [
    "Good health",
    "Supportive team",
    "Beautiful weather"
  ],
  "mood": "content",
  "moodScore": 8
}
```

6.4 Enterprise Feature APIs

6.4.1 Projects API

Create Project

```
POST /enterprise/projects
```

```
{
  "name": "Website Redesign",
```

```
  "description": "Complete overhaul of company website",
  "projectCode": "WEB-2026-001",
  "methodology": "agile",
  "framework": "scrum",
  "projectManagerId": "507f1f77bcf86cd799439011",
  "team": [
    {
      "userId": "507f1f77bcf86cd799439012",
      "role": "developer",
      "allocation": 80,
      "hourlyRate": 75.00
    }
  ],
  "startDate": "2026-02-01T00:00:00Z",
  "endDate": "2026-05-31T00:00:00Z",
  "budgetType": "fixed",
  "budgetAmount": 50000.00
}
```


Response: 201 Created

Get Project Dashboard

```
GET /enterprise/projects/:id/dashboard
```

Response: 200 OK


```
{
  "success": true,
  "data": {
    "project": {
      "id": "507f1f77bcf86cd799439011",
      "name": "Website Redesign",
      "status": "active",
      "progress": 42.5
    },
    "metrics": {
      "tasksTotal": 120,
      "tasksCompleted": 51,
      "hoursLogged": 342.5,
      "budgetUsed": 25687.50,
      "budgetRemaining": 24312.50
    },
    "currentSprint": {
      "sprintNumber": 3,
      "name": "Sprint 3",
      "startDate": "2026-01-15T00:00:00Z",
      "endDate": "2026-01-28T00:00:00Z",
      "totalStoryPoints": 42,
      "completedStoryPoints": 28
```

```
    }
  }
}
```

Create Sprint

```
POST /enterprise/projects/:projectId/sprints

{
  "sprintNumber": 4,
  "name": "Sprint 4",
  "startDate": "2026-01-29T00:00:00Z",
  "endDate": "2026-02-11T00:00:00Z",
  "goal": "Complete homepage and navigation",
  "totalStoryPoints": 45
}
```

6.4.2 Inventory API

Create Product

```
POST /enterprise/inventory/products

{
  "sku": "PROD-001",
  "name": "Wireless Mouse",
  "description": "Ergonomic wireless mouse",
  "category": "electronics",
  "brand": "TechCo",
  "trackInventory": true,
  "stockLevel": 150,
  "reorderPoint": 20,
  "reorderQuantity": 50,
  "costPrice": 15.00,
  "sellingPrice": 29.99,
  "weight": 0.15,
  "dimensions": {
    "length": 12,
    "width": 7,
    "height": 4
  }
}
```

Update Stock

```
POST /enterprise/inventory/products/:id/stock

{
  "type": "adjustment",
  "quantity": 50,
```

```
  "reason": "New shipment received",
  "locationId": "507f1f77bcf86cd799439011",
  "costPerUnit": 15.00
}

Response: 200 OK
{
  "success": true,
  "data": {
    "productId": "507f1f77bcf86cd799439011",
    "previousStock": 150,
    "newStock": 200,
    "movement": {
      "id": "507f1f77bcf86cd799439013",
      "type": "received",
      "quantity": 50,
      "date": "2026-01-19T10:30:00Z"
    }
  }
}
```

Get Low Stock Report

```
GET /enterprise/inventory/reports/low-stock

Response: 200 OK
{
  "success": true,
  "data": [
    {
      "productId": "507f1f77bcf86cd799439011",
      "sku": "PROD-001",
      "name": "Wireless Mouse",
      "currentStock": 15,
      "reorderPoint": 20,
      "reorderQuantity": 50,
      "status": "below_reorder_point"
    }
  ]
}
```

6.4.3 Orders API

Create Order

```
POST /enterprise/orders

{
  "type": "sales_order",
```

```
  "customerId": "507f1f77bcf86cd799439011",
  "items": [
    {
      "productId": "507f1f77bcf86cd799439012",
      "quantity": 2,
      "unitPrice": 29.99
    }
  ],
  "shippingAddress": {
    "street": "123 Main St",
    "city": "San Francisco",
    "state": "CA",
    "zipCode": "94102",
    "country": "USA"
  },
  "shippingMethod": "standard",
  "channel": "website"
}

Response: 201 Created
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439013",
    "orderNumber": "ORD-2026-00123",
    "subtotal": 59.98,
    "taxTotal": 5.40,
    "shippingCost": 8.99,
    "total": 74.37,
    "fulfillmentStatus": "pending",
    "paymentStatus": "pending"
  }
}
```

Update Order Status

```
PATCH /enterprise/orders/:id/status

{
  "fulfillmentStatus": "shipped",
  "carrier": "UPS",
  "trackingNumber": "1Z999AA10123456784",
  "actualShipDate": "2026-01-19T14:30:00Z"
}
```

6.4.4 CRM API

Create Contact

```
POST /enterprise/crm/contacts

{
  "type": "lead",
  "firstName": "Sarah",
  "lastName": "Johnson",
  "email": "sarah.johnson@example.com",
  "phone": "+1-555-0123",
  "companyName": "Acme Corp",
  "position": "CTO",
  "source": "website",
  "status": "new",
  "ownerId": "507f1f77bcf86cd799439011"
}
```

Create Deal

```
POST /enterprise/crm/deals

{
  "dealName": "Acme Corp - Enterprise Plan",
  "contactId": "507f1f77bcf86cd799439011",
  "dealValue": 50000,
  "currency": "USD",
  "pipelineId": "507f1f77bcf86cd799439012",
  "stageId": "507f1f77bcf86cd799439013",
  "probability": 60,
  "expectedCloseDate": "2026-03-31T00:00:00Z",
  "products": [
    {
      "productId": "507f1f77bcf86cd799439014",
      "quantity": 100,
      "unitPrice": 500
    }
  ]
}
```

Get Sales Pipeline

```
GET /enterprise/crm/pipelines/:id

Response: 200 OK
{
  "success": true,
  "data": {
    "pipeline": {
      "id": "507f1f77bcf86cd799439012",
      "name": "Sales Pipeline"
    },
    "stages": [
```

```
      {
        "stageId": "stage1",
        "name": "Prospecting",
        "dealCount": 15,
        "totalValue": 750000,
        "deals": [...]
      }
    ],
    "metrics": {
      "totalDeals": 45,
      "totalValue": 2250000,
      "averageDealSize": 50000,
      "winRate": 42.5
    }
  }
}
```

6.4.5 Invoices API

Create Invoice

```
POST /enterprise/invoices

{
  "customerId": "507f1f77bcf86cd799439011",
  "invoiceDate": "2026-01-19T00:00:00Z",
  "dueDate": "2026-02-18T00:00:00Z",
  "lineItems": [
    {
      "description": "Enterprise Plan - Monthly",
      "quantity": 1,
      "unitPrice": 299.00,
      "amount": 299.00,
      "taxable": true
    },
    {
      "description": "Additional Users (5)",
      "quantity": 5,
      "unitPrice": 20.00,
      "amount": 100.00,
      "taxable": true
    }
  ],
  "paymentTerms": "net_30",
  "notes": "Thank you for your business!"
}

Response: 201 Created
```

```
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439013",
    "invoiceNumber": "INV-2026-00045",
    "subtotal": 399.00,
    "taxAmount": 35.91,
    "total": 434.91,
    "status": "draft"
  }
}
```

Send Invoice

```
POST /enterprise/invoices/:id/send

{
  "sendEmail": true,
  "customMessage": "Thank you for your business!"
}

Response: 200 OK
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439013",
    "status": "sent",
    "sentDate": "2026-01-19T15:00:00Z"
  }
}
```

Record Payment

```
POST /enterprise/invoices/:id/payments

{
  "date": "2026-02-10T00:00:00Z",
  "amount": 434.91,
  "method": "bank_transfer",
  "reference": "TXN-123456"
}

Response: 200 OK
{
  "success": true,
  "data": {
    "invoiceId": "507f1f77bcf86cd799439013",
    "status": "paid",
    "paidDate": "2026-02-10T00:00:00Z",
    "amountPaid": 434.91,
```

```
    "amountDue": 0
  }
}
```

6.4.6 Time & Attendance API

Clock In

```
POST /enterprise/time-clock/clock-in

{
  "clockInMethod": "mobile",
  "clockInLocation": {
    "lat": 37.7749,
    "lng": -122.4194,
    "address": "123 Market St, San Francisco, CA"
  },
  "workLocation": "office"
}

Response: 201 Created
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "employeeId": "507f1f77bcf86cd799439012",
    "clockInTime": "2026-01-19T09:00:00Z",
    "status": "clocked_in"
  }
}
```

Clock Out

```
POST /enterprise/time-clock/:id/clock-out

Response: 200 OK
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "clockInTime": "2026-01-19T09:00:00Z",
    "clockOutTime": "2026-01-19T17:30:00Z",
    "totalHours": 8.5,
    "regularHours": 8.0,
    "overtimeHours": 0.5
  }
}
```

Get Attendance Report

```
GET /enterprise/time-clock/reports?startDate=2026-01-01&endDate=2026-01-31&employeeId=507f1f77b
```

```
Response: 200 OK
{
  "success": true,
  "data": {
    "employee": {
      "id": "507f1f77bcf86cd799439012",
      "name": "John Doe"
    },
    "period": {
      "start": "2026-01-01",
      "end": "2026-01-31"
    },
    "summary": {
      "totalDays": 20,
      "daysPresent": 19,
      "daysAbsent": 1,
      "totalHours": 156.5,
      "regularHours": 152.0,
      "overtimeHours": 4.5,
      "lateArrivals": 2
    }
  }
}
```

6.5 Payment & Subscription APIs

6.5.1 Subscribe to Feature

Endpoint: `POST /subscriptions/subscribe`

Request:

```
{
  "feature": "finance_manager",
  "tier": "premium",
  "billingCycle": "monthly",
  "paymentMethodId": "pm_1234567890"
}
```

Response: `201 Created`

```
{
  "success": true,
  "data": {
    "subscription": {
      "id": "507f1f77bcf86cd799439011",
      "feature": "finance_manager",
```

```
      "tier": "premium",
      "status": "active",
      "startDate": "2026-01-19T00:00:00Z",
      "endDate": "2026-02-19T00:00:00Z",
      "amount": 6.00,
      "currency": "USD"
    },
    "invoice": {
      "id": "in_1234567890",
      "amount": 6.00,
      "status": "paid"
    }
  }
}
```

6.5.2 Cancel Subscription

Endpoint: `DELETE /subscriptions/:id`

Request:

```
{
  "cancelAtPeriodEnd": true,
  "reason": "No longer needed"
}
```

Response: `200 OK`

```
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "status": "active",
    "cancelAtPeriodEnd": true,
    "currentPeriodEnd": "2026-02-19T00:00:00Z"
  }
}
```

6.5.3 Get Payment Methods

Endpoint: `GET /payments/methods`

Response: `200 OK`

```
{
  "success": true,
  "data": [
    {
      "id": "pm_1234567890",
      "type": "card",
      "card": {
```

```
        "brand": "visa",
        "last4": "4242",
        "expMonth": 12,
        "expYear": 2028
      },
      "isDefault": true
    }
  ]
}
```

6.5.4 Get Billing History

Endpoint: `GET /payments/history?page=1&limit=20`

Response: `200 OK`

```
{
  "success": true,
  "data": [
    {
      "id": "in_1234567890",
      "date": "2026-01-19T00:00:00Z",
      "description": "Finance Manager - Monthly",
      "amount": 6.00,
      "currency": "USD",
      "status": "paid",
      "invoicePdf": "https://invoices.stripe.com/..."
    }
  ]
}
```

6.6 WebSocket Events

6.6.1 Connection

Connect:

```
const socket = io('wss://api.prms.io', {
  auth: {
    token: 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'
  }
})

socket.on('connect', () => {
  console.log('Connected to WebSocket server')
})
```

6.6.2 Real-time Events

Task Updates:

```
// Subscribe to task updates
socket.emit('subscribe', { channel: 'tasks' })

// Listen for task changes
socket.on('task:created', (data) => {
  console.log('New task:', data)
})

socket.on('task:updated', (data) => {
  console.log('Task updated:', data)
})

socket.on('task:deleted', (data) => {
  console.log('Task deleted:', data)
})
```

Notification Events:

```
socket.on('notification', (notification) => {
  // {
  //   id: "507f1f77bcf86cd799439011",
  //   type: "task_due",
  //   title: "Task Due Soon",
  //   message: "Complete project proposal is due in 1 hour",
  //   timestamp: "2026-01-19T16:00:00Z"
  // }
})
```

Team Collaboration:

```
// Subscribe to project channel
socket.emit('subscribe', {
  channel: 'project',
  projectId: '507f1f77bcf86cd799439011'
})

socket.on('project:member_joined', (data) => {
  console.log('Member joined:', data)
})

socket.on('project:task_assigned', (data) => {
  console.log('Task assigned:', data)
})
```

Sync Events:

```
socket.on('sync:required', (data) => {
```

```
  // { entities: ['tasks', 'events'], reason: 'data_changed' }
  // Trigger sync process
})
```

6.7 Error Handling

6.7.1 Error Response Format

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": {
      "field": "email",
      "constraint": "Must be a valid email address"
    }
  }
}
```

6.7.2 HTTP Status Codes

- `200 OK` - Success
- `201 Created` - Resource created
- `204 No Content` - Success with no response body
- `400 Bad Request` - Invalid request data
- `401 Unauthorized` - Missing or invalid authentication
- `403 Forbidden` - Insufficient permissions
- `404 Not Found` - Resource not found
- `409 Conflict` - Resource conflict
- `422 Unprocessable Entity` - Validation error
- `429 Too Many Requests` - Rate limit exceeded
- `500 Internal Server Error` - Server error
- `503 Service Unavailable` - Service temporarily unavailable

6.7.3 Error Codes

Authentication Errors: - `AUTH_REQUIRED` - Authentication required - `INVALID_TOKEN` - Invalid or expired token - `INVALID_CREDENTIALS` - Invalid email/password - `ACCOUNT_LOCKED` - Too many failed login attempts

Authorization Errors: - `INSUFFICIENT_PERMISSIONS` - User lacks required permissions - `SUBSCRIPTION_REQUIRED` - Feature requires premium subscription - `PLAN_LIMIT_EXCEEDED` - Plan limit reached

Validation Errors: - `VALIDATION_ERROR` - Request data validation failed - `INVALID_FORMAT` - Invalid data format - `REQUIRED_FIELD` - Required field missing

Resource Errors: - `NOT_FOUND` - Resource not found - `ALREADY_EXISTS` - Resource already exists - `CONFLICT` - Resource conflict

Rate Limiting: - `RATE_LIMIT_EXCEEDED` - Too many requests

6.7.4 Example Error Responses

Validation Error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request data",
    "details": [
      {
        "field": "email",
        "message": "Must be a valid email address"
      },
      {
        "field": "password",
        "message": "Must be at least 8 characters"
      }
    ]
  }
}
```

Subscription Required:

```
{
  "success": false,
  "error": {
    "code": "SUBSCRIPTION_REQUIRED",
    "message": "This feature requires a premium subscription",
    "details": {
      "feature": "finance_manager",
      "requiredTier": "premium",
      "currentTier": "free",
      "upgradeUrl": "/subscriptions/upgrade"
    }
  }
}
```

Rate Limit Exceeded:

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many requests. Please try again later.",
    "details": {
```

```
      "retryAfter": 60,
      "limit": 100,
      "resetAt": "2026-01-19T17:00:00Z"
    }
  }
}
```

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: API Team - Status: Draft - API Design Phase

Next Chapter: Chapter 7: UI/UX Design Specifications

Chapter 7: UI/UX Design Specifications

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Design Phase

Table of Contents

- 7.1 Design System
- 7.2 Component Library
- 7.3 User Flows
- 7.4 Page Layouts & Wireframes
- 7.5 Responsive Design Strategy
- 7.6 Accessibility Guidelines
- 7.7 Design Patterns

7.1 Design System

7.1.1 Color Palette

Primary Colors:

```
/* Brand Colors */
--color-primary-50: #EEF2FF;
--color-primary-100: #E0E7FF;
--color-primary-200: #C7D2FE;
--color-primary-300: #A5B4FC;
--color-primary-400: #818CF8;
--color-primary-500: #6366F1;    /* Main Brand */
--color-primary-600: #4F46E5;
--color-primary-700: #4338CA;
--color-primary-800: #3730A3;
--color-primary-900: #312E81;
```

```
/* Secondary Colors */
--color-secondary-50: #F0FDF4;
--color-secondary-100: #DCFCE7;
--color-secondary-500: #22C55E;
--color-secondary-700: #15803D;
```

Neutral Colors:

```
/* Grays */
--color-gray-50: #F9FAFB;
--color-gray-100: #F3F4F6;
--color-gray-200: #E5E7EB;
--color-gray-300: #D1D5DB;
--color-gray-400: #9CA3AF;
--color-gray-500: #6B7280;
--color-gray-600: #4B5563;
--color-gray-700: #374151;
--color-gray-800: #1F2937;
--color-gray-900: #111827;

/* Pure */
--color-white: #FFFFFF;
--color-black: #000000;
```

Semantic Colors:

```
/* Success */
--color-success-50: #F0FDF4;
--color-success-500: #22C55E;
--color-success-700: #15803D;

/* Warning */
--color-warning-50: #FFFBEB;
--color-warning-500: #F59E0B;
--color-warning-700: #B45309;

/* Error */
--color-error-50: #FEF2F2;
--color-error-500: #EF4444;
--color-error-700: #B91C1C;

/* Info */
--color-info-50: #EFF6FF;
--color-info-500: #3B82F6;
--color-info-700: #1D4ED8;
```

Feature-Specific Colors:

```
/* Tasks */
--color-task: #6366F1;
```

```
/* Calendar */
--color-calendar: #8B5CF6;

/* Finance */
--color-finance: #10B981;

/* Habits */
--color-habits: #F59E0B;

/* Goals */
--color-goals: #EC4899;

/* Time Tracking */
--color-time: #06B6D4;

/* Notes */
--color-notes: #FBBF24;

/* Health */
--color-health: #F43F5E;
```

7.1.2 Typography

Font Families:

```
/* Primary Font - Inter */
--font-primary: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;

/* Monospace - For code/numbers */
--font-mono: 'JetBrains Mono', 'Fira Code', monospace;

/* Headings - Optional accent */
--font-display: 'Inter', sans-serif;
```

Font Sizes:

```
/* Type Scale (1.250 - Major Third) */
--text-xs: 0.75rem;      /* 12px */
--text-sm: 0.875rem;     /* 14px */
--text-base: 1rem;       /* 16px */
--text-lg: 1.125rem;     /* 18px */
--text-xl: 1.25rem;      /* 20px */
--text-2xl: 1.5rem;      /* 24px */
--text-3xl: 1.875rem;    /* 30px */
--text-4xl: 2.25rem;     /* 36px */
--text-5xl: 3rem;        /* 48px */
--text-6xl: 3.75rem;     /* 60px */
```

Font Weights:

```
--font-weight-light: 300;
--font-weight-normal: 400;
--font-weight-medium: 500;
--font-weight-semibold: 600;
--font-weight-bold: 700;
--font-weight-extrabold: 800;
```

Line Heights:

```
--leading-none: 1;
--leading-tight: 1.25;
--leading-snug: 1.375;
--leading-normal: 1.5;
--leading-relaxed: 1.625;
--leading-loose: 2;
```

7.1.3 Spacing System

Scale: 4px base unit

```
--spacing-0: 0;
--spacing-1: 0.25rem;    /* 4px */
--spacing-2: 0.5rem;     /* 8px */
--spacing-3: 0.75rem;    /* 12px */
--spacing-4: 1rem;       /* 16px */
--spacing-5: 1.25rem;    /* 20px */
--spacing-6: 1.5rem;     /* 24px */
--spacing-8: 2rem;       /* 32px */
--spacing-10: 2.5rem;    /* 40px */
--spacing-12: 3rem;      /* 48px */
--spacing-16: 4rem;      /* 64px */
--spacing-20: 5rem;      /* 80px */
--spacing-24: 6rem;      /* 96px */
```

7.1.4 Border Radius

```
--radius-none: 0;
--radius-sm: 0.125rem;   /* 2px */
--radius-base: 0.25rem;  /* 4px */
--radius-md: 0.375rem;   /* 6px */
--radius-lg: 0.5rem;     /* 8px */
--radius-xl: 0.75rem;    /* 12px */
--radius-2xl: 1rem;      /* 16px */
--radius-3xl: 1.5rem;    /* 24px */
--radius-full: 9999px;   /* Circular */
```

7.1.5 Shadows

```
/* Elevation System */
--shadow-xs: 0 1px 2px 0 rgb(0 0 0 / 0.05);
```

```
--shadow-sm: 0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1);
--shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
--shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
--shadow-xl: 0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.1);
--shadow-2xl: 0 25px 50px -12px rgb(0 0 0 / 0.25);

/* Interactive States */
--shadow-focus: 0 0 0 3px rgba(99, 102, 241, 0.2);
```

7.1.6 Animation & Transitions

```
/* Duration */
--duration-fast: 150ms;
--duration-normal: 300ms;
--duration-slow: 500ms;

/* Easing */
--ease-linear: linear;
--ease-in: cubic-bezier(0.4, 0, 1, 1);
--ease-out: cubic-bezier(0, 0, 0.2, 1);
--ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
--ease-bounce: cubic-bezier(0.68, -0.55, 0.265, 1.55);

/* Common Transitions */
--transition-colors: color var(--duration-normal) var(--ease-in-out),
                     background-color var(--duration-normal) var(--ease-in-out),
                     border-color var(--duration-normal) var(--ease-in-out);
--transition-transform: transform var(--duration-normal) var(--ease-in-out);
--transition-all: all var(--duration-normal) var(--ease-in-out);
```

7.1.7 Breakpoints

```
/* Mobile First Approach */
--breakpoint-sm: 640px;   /* Small tablets */
--breakpoint-md: 768px;   /* Tablets */
--breakpoint-lg: 1024px;  /* Laptops */
--breakpoint-xl: 1280px;  /* Desktops */
--breakpoint-2xl: 1536px; /* Large displays */
```

7.2 Component Library

7.2.1 Buttons

Primary Button:

```
interface ButtonProps {
  variant: 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger'
  size: 'xs' | 'sm' | 'md' | 'lg' | 'xl'
```

```
  disabled?: boolean
  loading?: boolean
  icon?: ReactNode
  fullWidth?: boolean
}

// Styling
.button-primary {
  background: var(--color-primary-500);
  color: white;
  padding: var(--spacing-2) var(--spacing-4);
  border-radius: var(--radius-md);
  font-weight: var(--font-weight-medium);
  transition: var(--transition-colors);
}

.button-primary:hover {
  background: var(--color-primary-600);
}

.button-primary:active {
  background: var(--color-primary-700);
}

.button-primary:focus {
  outline: none;
  box-shadow: var(--shadow-focus);
}
```

Button Sizes:

```
.button-xs { padding: 0.25rem 0.5rem; font-size: var(--text-xs); }
.button-sm { padding: 0.375rem 0.75rem; font-size: var(--text-sm); }
.button-md { padding: 0.5rem 1rem; font-size: var(--text-base); }
.button-lg { padding: 0.75rem 1.5rem; font-size: var(--text-lg); }
.button-xl { padding: 1rem 2rem; font-size: var(--text-xl); }
```

7.2.2 Input Fields

Text Input:

```
interface InputProps {
  type: 'text' | 'email' | 'password' | 'number'
  label: string
  placeholder?: string
  error?: string
  hint?: string
  required?: boolean
  disabled?: boolean
```

```
  icon?: ReactNode
  onIconClick?: () => void
}

// Styling
.input-wrapper {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-1);
}

.input-label {
  font-size: var(--text-sm);
  font-weight: var(--font-weight-medium);
  color: var(--color-gray-700);
}

.input-field {
  padding: var(--spacing-2) var(--spacing-3);
  border: 1px solid var(--color-gray-300);
  border-radius: var(--radius-md);
  font-size: var(--text-base);
  transition: var(--transition-colors);
}

.input-field:focus {
  outline: none;
  border-color: var(--color-primary-500);
  box-shadow: var(--shadow-focus);
}

.input-field.error {
  border-color: var(--color-error-500);
}
```

7.2.3 Cards

Base Card:

```
interface CardProps {
  variant: 'flat' | 'outlined' | 'elevated'
  padding?: 'none' | 'sm' | 'md' | 'lg'
  hoverable?: boolean
  clickable?: boolean
}

.card {
  background: white;
```

```
  border-radius: var(--radius-lg);
  overflow: hidden;
}

.card-elevated {
  box-shadow: var(--shadow-md);
}

.card-outlined {
  border: 1px solid var(--color-gray-200);
}

.card-hoverable:hover {
  box-shadow: var(--shadow-lg);
  transform: translateY(-2px);
  transition: all var(--duration-normal) var(--ease-out);
}
```

7.2.4 Modal/Dialog

Modal Structure:

```
 Overlay (backdrop)

     Modal Container

       Header
       [Title]        [Close X]

       Body
       (Scrollable content)

       Footer
       [Cancel]  [Save Button]
```

Modal Sizes:

```
.modal-sm { max-width: 400px; }
.modal-md { max-width: 600px; }
.modal-lg { max-width: 800px; }
.modal-xl { max-width: 1200px; }
.modal-full { max-width: 100%; height: 100%; }
```

7.2.5 Navigation

Sidebar Navigation:

```
[Logo]

 Dashboard
 Tasks
 Calendar
 Finance
Goals

 Settings
 Profile
```

Navigation Item States:

```
.nav-item {
  padding: var(--spacing-3) var(--spacing-4);
  display: flex;
  align-items: center;
  gap: var(--spacing-3);
  color: var(--color-gray-700);
  border-radius: var(--radius-md);
  transition: var(--transition-colors);
}

.nav-item:hover {
  background: var(--color-gray-100);
}

.nav-item.active {
  background: var(--color-primary-50);
  color: var(--color-primary-700);
  font-weight: var(--font-weight-medium);
}
```

7.2.6 Tables

Data Table:

```
        [Search] [Filter  ] [Export]    [+ Add]


 Name       Status      Due Date     Priority   ...

 Task 1       Active    Jan 20       High
 Task 2       Pending   Jan 21       Medium
 Task 3       Done      Jan 19       Low
```

```
  Showing 1-10 of 45        [< 1 2 3 4 5 >]
```

Table Features: - Sortable columns - Row selection (checkbox) - Inline editing - Expandable rows - Sticky header - Responsive (cards on mobile)

7.2.7 Toast Notifications

Toast Types:

```
interface ToastProps {
  type: 'success' | 'error' | 'warning' | 'info'
  title: string
  message?: string
  duration?: number // Auto-dismiss
  action?: { label: string; onClick: () => void }
}

// Position: Top-right
// Max visible: 3 toasts
// Stack direction: Top to bottom
```

Toast Design:

```
   Success
  Task created successfully
                 [Undo]
```

7.2.8 Form Elements

Select/Dropdown:

```
interface SelectProps {
  options: Array<{ value: string; label: string; icon?: ReactNode }>
  multiple?: boolean
  searchable?: boolean
  clearable?: boolean
  placeholder?: string
}
```

Date Picker:

```
  January 2026

  Su Mo Tu We Th Fr Sa
            1  2  3  4
   5  6  7  8  9 10 11
  12 13 14 15 16 17 18
  [19]20 21 22 23 24 25
```

```
  26 27 28 29 30 31

  [Cancel]     [Apply]
```

Time Picker:

```
  10 : 30  AM
```

7.3 User Flows

7.3.1 Onboarding Flow

```
Start
  ↓
[Welcome Screen]
  ↓
[Mode Selection]
  → Individual → [Individual Setup]
  → Enterprise → [Enterprise Setup]
  → Both → [Hybrid Setup]
  ↓
[Feature Selection]
  - Show available features
  - Highlight free vs premium
  - Allow selection of initial features
  ↓
[Profile Setup]
  - Basic info
  - Avatar upload
  - Preferences
  ↓
[Quick Tour] (Optional)
  ↓
[Dashboard]
```

Individual Setup Steps: 1. Personal goals question 2. Work/life balance preferences 3. Preferred productivity method 4. Time zone & calendar sync 5. Import data (optional)

Enterprise Setup Steps: 1. Company information 2. Industry selection 3. Team size 4. Invite team members 5. Project setup 6. Integration preferences

7.3.2 Task Creation Flow

```
Entry Points:
```

```
  - Quick add button (+ icon)
  - "Add Task" button on Tasks page
  - Keyboard shortcut (Cmd/Ctrl + K)
  - Calendar event conversion
  - Email/message capture

Flow:
  ↓
[Quick Add Modal]
  - Title (required)
  - Due date (quick select)
  - Priority
  - [More Options] button
  ↓
[Detailed Form] (Optional)
  - Description
  - Project/Category
  - Tags
  - Subtasks
  - Attachments
  - Recurrence
  - Reminders
  ↓
[AI Suggestions] (Premium)
  - Suggested duration
  - Optimal time slot
  - Related tasks
  ↓
[Confirmation]
  - Toast notification
  - Auto-navigate to task
  - Quick actions (Edit, Delete, Share)
```

7.3.3 Payment/Subscription Flow

```
[Feature Locked State]
  - Show upgrade prompt
  - Benefits list
  - Pricing info
  ↓
[Pricing Page]
  - Feature comparison
  - Monthly/Annual toggle
  - FAQ section
  ↓
[Checkout]
  - Feature summary
  - Payment method
```

```
  - Billing details
  - Promo code
  ↓
[Payment Processing]
  - Stripe integration
  - Loading state
  ↓
[Success State]
  - Confirmation message
  - Feature now unlocked
  - Receipt email sent
  ↓
[Dashboard]
  - Feature badge
  - Quick tour of new feature
```

7.3.4 Team Collaboration Flow

```
[Project Created]
  ↓
[Invite Members]
  - Email invitations
  - Role assignment
  - Permission settings
  ↓
[Member Accepts]
  - Email notification
  - Join project
  ↓
[Collaboration Activities]
  - Assign tasks
  - Share files
  - @mentions
  - Comments
  - Real-time updates
  ↓
[Notifications]
  - In-app notifications
  - Email digests
  - Mobile push
```

7.4 Page Layouts & Wireframes

7.4.1 Dashboard Layout

```
[Logo]      [Search...]      [Notifications]   [User]
```

```
        Good morning, John
        You have 5 tasks due today


Nav
Menu    Tasks           Events          Time
        5 due today     3 upcoming      4.5h logged



       Today's Schedule       Recent Activity

       09:00 Team Meeting     • Task completed
        11:00 Project Work     • Budget updated
       14:00 Client Call      • Goal milestone hit
```

Dashboard Widgets: - Overview stats (customizable) - Upcoming tasks/events - Quick actions - Recent activity feed - Progress charts - Habit streaks - Goal progress - Financial summary

7.4.2 Task List View

```
Tasks                                        [+ New Task]

     [All] [Active] [Completed]  [Search...] [Filter  ]


TODAY

   Complete project proposal            High   2:00 PM
     Work ·   Jan 19

   Review design mockups                Medium   4:00 PM
     Design ·   Jan 19


TOMORROW

   Client presentation                   High
     Work ·   Jan 20
```

```
UPCOMING

   Update documentation                  Low
     Docs ·   Jan 25
```

Task Card Components: - Checkbox (completion) - Title - Priority indicator - Due date/time - Project/category tags - Assignee (for team tasks) - Subtask progress (3/5) - Quick actions menu ()

7.4.3 Calendar View

```
Calendar                      January 2026    [Month  ]

Sun    Mon    Tue    Wed    Thu    Fri    Sat

                       1      2      3      4

  5      6      7      8      9     10     11


 12     13     14     15     16     17     18


[19]    20     21     22     23     24     25
9:00   14:00
Team   Client
14:30
Design

 26     27     28     29     30     31
```

Calendar Views: - Month view (default) - Week view (time slots) - Day view (hourly breakdown) - Agenda view (list) - Year view (mini calendars)

7.4.4 Finance Dashboard

```
Finance                             January 2026


  Income          Expenses        Net Income
  $5,000.00       $2,342.50       $2,657.50
  ↑ 10%           ↓ 5%            ↑ 15%
```

```
Spending by Category
        Groceries      $450 (19%)
      Transportation     $300 (13%)
    Utilities            $200 (9%)
   Entertainment          $150 (6%)



Recent Transactions       Budget Overview

-$42.50 Whole Foods       Groceries: $450/$500 (90%)
 Jan 19

-$85.00 Gas Station       Transport: $300/$400 (75%)
 Jan 18

+$5000 Salary             Dining: $250/$300 (83%)
 Jan 15
```

7.4.5 Project Board (Kanban)

```
Website Redesign Project                    [Board  ] [+ Add]

Team: 5 members  Sprint 3  Progress: 42%

To Do          In Progress   Review        Done


Homepage     Navigation   Login Page   Footer
Design       Component    UI           Component

  Sarah      John       Mike       Done
  8 pts      5 pts      3 pts     Jan 15



Contact      Product                    Header
Form         Catalog                    Component

  Lisa       Alex                     Done
  5 pts      13 pts                  Jan 12

   (3)          (2)          (1)          (15)
```

7.5 Responsive Design Strategy

7.5.1 Breakpoint Strategy

Mobile First Approach:

```
/* Base styles for mobile (320px+) */
.container {
  padding: var(--spacing-4);
}

/* Small tablets (640px+) */
@media (min-width: 640px) {
  .container {
    padding: var(--spacing-6);
  }
}

/* Tablets (768px+) */
@media (min-width: 768px) {
  .container {
    padding: var(--spacing-8);
    max-width: 768px;
  }
}

/* Laptops (1024px+) */
@media (min-width: 1024px) {
  .container {
    max-width: 1024px;
  }
}

/* Desktop (1280px+) */
@media (min-width: 1280px) {
  .container {
    max-width: 1280px;
  }
}
```

7.5.2 Layout Transformations

Navigation: - Mobile (<768px): Bottom tab bar or hamburger menu - Tablet (768-1024px): Collapsible sidebar - Desktop (>1024px): Fixed sidebar with labels

Task List: - Mobile: Single column, card stack - Tablet: Two columns or enhanced cards - Desktop: Multi-column with sidebar filters

Calendar: - Mobile: Agenda view or single day - Tablet: Week view - Desktop: Month view with sidebar

7.5.3 Touch Targets

Minimum Sizes: - Buttons: 44Œ44px (iOS), 48Œ48px (Android) - List items: 56px height - Form inputs: 48px height - Icons: 24Œ24px minimum

Spacing: - Minimum 8px between interactive elements - Larger hit areas on mobile (invisible padding)

7.5.4 Mobile-Specific Patterns

Bottom Sheet:

```
  Main Content


      Handle       ← Drag handle


    Sheet Content
```

Pull to Refresh:

```
 ↓ (Pull down)

     Loading...

 Content
```

Swipe Actions:

```
 Task Item     ← Swipe
 [Archive] [Delete]
```

7.6 Accessibility Guidelines

7.6.1 WCAG 2.1 AA Compliance

Color Contrast: - Normal text: 4.5:1 minimum - Large text (18pt+): 3:1 minimum - UI components: 3:1 minimum

Contrast Examples:

```
/* Good Contrast */
.text-on-white {
  color: #374151; /* Gray-700, ratio 8.5:1 */
}

.primary-button {
  background: #4F46E5; /* Primary-600 */
  color: #FFFFFF; /* Ratio 8.2:1 */
}

/* Avoid */
.low-contrast {
  color: #D1D5DB; /* Gray-300 on white, ratio 1.8:1  */
}
```

7.6.2 Keyboard Navigation

Focus Indicators:

```
*:focus {
  outline: 2px solid var(--color-primary-500);
  outline-offset: 2px;
}

/* Skip to main content */
.skip-link {
  position: absolute;
  top: -40px;
  left: 0;
  background: var(--color-primary-500);
  color: white;
  padding: 8px;
  z-index: 100;
}

.skip-link:focus {
  top: 0;
}
```

Tab Order: 1. Logo/Skip link 2. Main navigation 3. Search 4. Primary content 5. Secondary content 6. Footer

Keyboard Shortcuts: - `Cmd/Ctrl + K`: Quick add - `Cmd/Ctrl + /`: Search - `Esc`: Close modal/cancel - `Tab`: Next element - `Shift + Tab`: Previous element - `Enter`: Activate/Submit - `Space`: Select/Toggle

7.6.3 Screen Reader Support

ARIA Labels:

```
<!-- Navigation -->
<nav aria-label="Main navigation">
  <button aria-label="Open menu" aligned="false">

  </button>
</nav>

<!-- Status indicators -->
<span class="status-badge" aria-label="Status: Active">
    Active
</span>

<!-- Loading states -->
<button disabled aria-busy="true">
  Loading...
  <span class="sr-only">Please wait</span>
</button>

<!-- Form validation -->
<input
  type="email"
  aria-invalid="true"
  aria-describedby="email-error"
/>
<span id="email-error" role="alert">
  Please enter a valid email
</span>
```

Landmark Regions:

```
<header role="banner">
<nav role="navigation">
<main role="main">
<aside role="complementary">
<footer role="contentinfo">
```

7.6.4 Visual Accommodations

Text Scaling: - Support 200% zoom without horizontal scroll - Responsive font sizes with `rem` units - Preserve layout at different zoom levels

Reduced Motion:

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
```

```
    transition-duration: 0.01ms !important;
  }
}
```

High Contrast Mode:

```
@media (prefers-contrast: high) {
  .button {
    border: 2px solid currentColor;
  }

  .card {
    border: 2px solid var(--color-gray-900);
  }
}
```

Dark Mode:

```
@media (prefers-color-scheme: dark) {
  :root {
    --color-background: #111827;
    --color-text: #F9FAFB;
    --color-surface: #1F2937;
  }
}
```

7.7 Design Patterns

7.7.1 Empty States

No Tasks:

```
No tasks yet
Create your first task
to get started

[+ Create Task]
```

No Search Results:

```
No results found
Try different keywords
```

```
[Clear Search]
```

7.7.2 Loading States

Skeleton Screens:

Progress Indicators: - Spinner: Short operations (<5s) - Progress bar: Known duration - Skeleton: Content loading - Shimmer effect: List loading

7.7.3 Error States

Inline Validation:

```
Email

invalid@

Please enter valid email
```

Error Pages:

```
      404
  Page Not Found

The page you're looking
for doesn't exist

[← Back to Home]
```

7.7.4 Success States

Confirmation:

```
  Success!
Task created successfully
```

```
[View Task] [Dismiss]
```

Progress Celebration:

```
Streak Achievement!
30 days of meditation
[Share] [Continue]
```

7.7.5 Contextual Actions

Floating Action Button:

```
                +     ← Always visible
```

Quick Actions Menu:

```
Task Item

                      Edit
                      Delete
                      Share
                      Duplicate
```

7.7.6 Progressive Disclosure

Basic to Advanced:

```
[Create Task]
  ↓ Click

  Title: ________________
  Due Date: [Select]

  [  More Options]

  ↓ Expand

  Title: ________________
  Due Date: [Select]
  Priority: [Select]
  Tags: ________________
  Project: [Select]
  Description: _________
  Subtasks: [+ Add]
```

```
Attachments: [Upload]
```

7.7.7 Micro-interactions

Button Click: - Scale down on press (transform: scale(0.95)) - Ripple effect from click point - Haptic feedback on mobile

Toggle Switch: - Smooth slide animation (300ms) - Color transition - Haptic feedback

Checkbox: - Checkmark animation - Scale bounce effect - Color transition

Like/Star: - Scale pop animation - Color fill animation - Particle effect (optional)

7.7.8 Data Visualization

Charts: - Line charts: Trends over time - Bar charts: Comparisons - Pie charts: Category breakdown - Heatmaps: Activity patterns - Progress rings: Goal completion

Color Coding: - Green: Positive, success, income - Red: Negative, danger, expenses - Blue: Information, neutral - Yellow/Orange: Warning, attention - Purple: Premium features

---

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Design Team - Status: Draft - Design Phase

Next Chapter: Chapter 8: Business Model & Monetization

Chapter 8: Business Model & Monetization

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Business Planning Phase

Table of Contents

- 8.1 Business Model Overview
- 8.2 Pricing Strategy
- 8.3 Revenue Streams
- 8.4 Customer Acquisition
- 8.5 Financial Projections
- 8.6 Unit Economics
- 8.7 Competitive Analysis

8.1 Business Model Overview

8.1.1 Core Business Model

Type: Freemium SaaS with À La Carte Features

Value Proposition: - Individual users get core productivity tools for free - Pay only for features you need - No forced bundling - Flexibility to build custom toolkit - Enterprise plans for team collaboration

Target Markets: 1. Individual Productivity (B2C) - Freelancers - Students - Remote workers - Side hustlers - Personal use

2. Enterprise Operations (B2B)
    - Small businesses (1-50 employees)
    - Mid-market (51-500 employees)
    - Startups
    - Growing teams

8.1.2 Revenue Model

Individual Features:

```
Free Tier (3 features):
  Event Tracker & Calendar
  Task Management
  Note Taking

Premium Features (Pay per feature):
  Finance Manager: $6/month
  Habit Tracker: $4/month
  Goal Setting: $5/month
  Time Tracking: $5/month
  Health & Wellness: $4/month
  Focus Timer: $3/month
  Dashboard: $8/month
  File Manager: $5/month
  Journal: $4/month

Bundle Options:
  Productivity Bundle: $15/month (Tasks, Calendar, Time, Notes, Focus)
  Wellness Bundle: $12/month (Health, Habits, Goals, Journal)
  Complete Bundle: $25/month (All 12 features, save 42%)
```

Enterprise Features:

```
Free Tier (3 features):
  Project Management
  Team Communication
  Time & Attendance

Premium Features (Pay per feature + per user):
  Inventory Management: $75/month base + $5/user
  Order Management: $60/month base + $4/user
  Finance & Accounting: $80/month base + $6/user
  CRM: $65/month base + $5/user
```

```
  Invoice & Billing: $55/month base + $4/user
  Analytics Dashboard: $70/month base + $5/user
  Resource Planning: $60/month base + $5/user
  Document Management: $50/month base + $3/user
  Vendor Management: $45/month base + $3/user

Enterprise Plans:
  Starter (up to 10 users): $199/month (3 free + 2 premium features)
  Professional (up to 50 users): $499/month (All features included)
  Enterprise (unlimited users): $999/month (All features + priority support)
  Custom: Contact sales
```

8.1.3 Revenue Strategy

Phase 1: Individual Focus (Months 1-12) - Launch with 3 free individual features - Release 2 premium features per month - Build user base through free tier - Convert 5-10% to paid features

Phase 2: Enterprise Entry (Months 13-24) - Launch enterprise features - Target small businesses first - Offer migration discounts - Expand team collaboration

Phase 3: Scale & Optimize (Months 25+) - AI-powered features (premium add-on) - Advanced analytics - White-label options - API access for enterprises

---

8.2 Pricing Strategy

8.2.1 Individual Feature Pricing

Pricing Principles: 1. Value-Based: Price based on user benefit 2. Competitive: 20-30% below competitors 3. Psychological: $4, $5, $6 (avoid $4.99) 4. Modular: Pay for what you use

Price Points Breakdown:

Feature	Monthly	Annual	Value Drivers
Finance Manager	$6	$60 (save $12)	Budget tracking, analytics, bank sync
Dashboard	$8	$80 (save $16)	Unified view, custom widgets, insights
Habit Tracker	$4	$40 (save $8)	Simple tracking, streak gamification
Goal Setting	$5	$50 (save $10)	SMART framework, milestone tracking
Time Tracking	$5	$50 (save $10)	Billable hours, client invoicing
Health & Wellness	$4	$40 (save $8)	Basic health logging
Focus Timer	$3	$30 (save $6)	Pomodoro, minimal features
File Manager	$5	$50 (save $10)	Cloud storage, organization
Journal	$4	$40 (save $8)	Daily reflection, prompts

Bundle Pricing Strategy:

Productivity Bundle: $15/month ($25 value individually) - Tasks (Free) - Calendar (Free) - Time Tracking ($5) - Notes (Free) - Focus Timer ($3) - Dashboard ($8) - Savings: 40% ($10/month)

Wellness Bundle: $12/month ($17 value) - Health & Wellness ($4) - Habit Tracker ($4) - Goal Setting ($5) - Journal ($4) - Savings: 29% ($5/month)

Complete Individual: $25/month ($44 value) - All 12 features - Savings: 43% ($19/month)

8.2.2 Enterprise Pricing

Pricing Model: Base fee + Per-user fee

Example: CRM Feature - Base: $65/month (platform access, admin tools) - Per user: $5/month - 20 users: $65 + (20 Œ $5) = $165/month

Enterprise Plans:

Starter Plan: $199/month - Up to 10 users - 3 free features (Projects, Communication, Time & Attendance) - Choose 2 premium features - 10GB storage per user - Email support - Target: Small businesses, startups

Professional Plan: $499/month - Up to 50 users - All 12 features included - 50GB storage per user - Priority email support - Dedicated account manager - Monthly training sessions - Target: Growing companies

Enterprise Plan: $999/month - Unlimited users - All features included - Unlimited storage - 24/7 phone/chat support - Dedicated success manager - Custom integrations - SLA guarantee (99.9% uptime) - Advanced security (SSO, SAML) - Target: Large organizations

Custom Plan: Contact Sales - White-label options - On-premise deployment - Custom feature development - Dedicated infrastructure - Legal/compliance customization

8.2.3 Discounting Strategy

Annual Subscriptions: - 16.7% discount (2 months free) - Improved cash flow - Higher retention

Volume Discounts (Enterprise): - 51-100 users: 10% off per-user fees - 101-250 users: 15% off - 251-500 users: 20% off - 500+ users: Custom pricing

Promotional Offers: - Launch Special: 50% off first 3 months - Student Discount: 40% off with .edu email - Non-Profit: 30% off all plans - Referral Program: $10 credit for referrer & referee - Migration Discount: 25% off first year (from competitors)

Lifetime Deals (Limited): - Early adopters only - $499 one-time (equivalent to 20 months) - All individual features forever - Limited to first 1,000 users

8.2.4 Trial Strategy

Individual Features: - 14-day free trial on any premium feature - No credit card required - Full feature access - Conversion optimization at day 7, 10, 13

Enterprise Plans: - 30-day free trial - Includes onboarding call - Up to 25 users during trial - Dedicated support during trial

---

8.3 Revenue Streams

8.3.1 Primary Revenue Streams

1. Subscription Revenue (85% of total)

Individual Subscriptions:

```
Conservative Scenario (Year 1):
- Total users: 50,000
- Free users: 42,500 (85%)
- Paid users: 7,500 (15%)
- Average features per paid user: 2.5
- Average revenue per feature: $5/month
- MRR: 7,500 × 2.5 × $5 = $93,750
- ARR: $1,125,000
```

Enterprise Subscriptions:

```
Conservative Scenario (Year 1):
- Starter plan customers: 100 × $199 = $19,900/month
- Professional plan customers: 30 × $499 = $14,970/month
- Enterprise plan customers: 5 × $999 = $4,995/month
- MRR: $39,865
- ARR: $478,380
```

2. Transaction Fees (5% of total)

Payment Processing (Invoice & Billing feature): - 1% fee on invoices processed through platform - Target: $10M in invoice volume (Year 2) - Revenue: $100,000/year

Marketplace/Add-ons: - Third-party integrations - Template marketplace - Custom workflows - 30% commission on sales

3. Professional Services (5% of total)

Implementation Services: - Enterprise onboarding: $2,500 per customer - Data migration: $1,500-$5,000 - Custom integration: $5,000-$25,000

Training & Consulting: - Group training: $500/session - 1-on-1 consulting: $200/hour - Custom workflow design: $1,500-$5,000

4. Data & Analytics (Future) - Aggregated, anonymized insights - Industry benchmarking reports - Trend analysis subscriptions

8.3.2 Revenue Projections by Stream

Year 1:

```
Individual Subscriptions:    $1,125,000  (70%)
Enterprise Subscriptions:    $  478,380  (30%)
Transaction Fees:            $        0  (0%)
Professional Services:       $        0  (0%)

Total Revenue:               $1,603,380
```

Year 2:

```
Individual Subscriptions:    $3,375,000  (60%)
Enterprise Subscriptions:    $1,800,000  (32%)
Transaction Fees:            $  225,000  (4%)
Professional Services:       $  225,000  (4%)

Total Revenue:               $5,625,000
```

Year 3:

```
Individual Subscriptions:    $6,750,000  (54%)
Enterprise Subscriptions:    $4,500,000  (36%)
Transaction Fees:            $  625,000  (5%)
Professional Services:       $  625,000  (5%)

Total Revenue:               $12,500,000
```

8.4 Customer Acquisition

8.4.1 Acquisition Channels

Organic Channels (60% of users):

1. Content Marketing: - SEO-optimized blog posts (3-5 per week) - Productivity guides & templates - YouTube tutorials - Podcast sponsorships - Target CAC: $5-10

2. Product-Led Growth: - Freemium model drives signups - Viral loops (sharing, collaboration) - In-app referral program - Social proof (testimonials, case studies) - Target CAC: $0-5

3. Community Building: - Reddit/Discord communities - LinkedIn thought leadership - Twitter engagement - User-generated content - Target CAC: $0-3

Paid Channels (40% of users):

1. Paid Search (Google Ads): - Keyword targeting: productivity, task management, etc. - Budget: $50,000/month - Target CPA: $30 - Expected users: 1,667/month - CAC: $30

2. Social Media Ads: - Facebook/Instagram: Lifestyle targeting - LinkedIn: B2B/Enterprise targeting - Budget: $30,000/month - Target CPA: $25 - Expected users: 1,200/month - CAC: $25

3. Affiliate Marketing: - Productivity influencers - Tech reviewers - Commission: 30% of first year revenue - Budget: Variable (commission-based) - CAC: $40-60

4. Sponsorships: - Productivity podcasts - YouTube channels - Newsletters - Budget: $20,000/month - CAC: $20-40

8.4.2 Conversion Funnel

```
Website Visitors: 100,000/month
        ↓ (10% sign up)
Free Signups: 10,000/month
        ↓ (30% activate - use 3+ times)
```

```
Active Users: 3,000/month
        ↓ (15% convert to paid)
Paid Users: 450/month
        ↓
MRR: $5,625 (assuming $12.50 average)
```

Conversion Optimization:

Stage 1: Visitor Signup (10%) - Clear value proposition - Social proof (testimonials) - No credit card required - 1-click social signup

Stage 2: Signup Activation (30%) - Onboarding tour - Quick wins (create first task) - Email nurture sequence - In-app guidance

Stage 3: Activation Paid (15%) - Feature limitations at right time - Upgrade prompts (non-intrusive) - Trial offers (14 days) - Value demonstration

8.4.3 Customer Acquisition Cost (CAC)

Individual Users: - Organic CAC: $5 - Paid CAC: $30 - Blended CAC: $15 (60% organic, 40% paid)

Enterprise Customers: - Inbound (marketing): $1,000 - Outbound (sales): $3,000 - Blended CAC: $2,000

CAC Payback Period: - Individual: 3-4 months - Enterprise: 6-8 months

8.4.4 Retention Strategy

Retention Metrics Targets:

Monthly Retention: - Month 1: 60% - Month 3: 45% - Month 6: 35% - Month 12: 25%

Churn Reduction Tactics:

1. Engagement Monitoring: - Track usage patterns - Identify at-risk users - Proactive outreach - Re-engagement campaigns

2. Value Reinforcement: - Weekly usage reports - Achievement notifications - Progress milestones - ROI calculators

3. Customer Success: - Check-in emails at 7, 30, 90 days - Webinar training - Feature tips - Best practices guides

4. Pricing Flexibility: - Pause subscription option - Downgrade path - Win-back campaigns - Seasonal discounts

---

8.5 Financial Projections

8.5.1 Three-Year Revenue Forecast

Assumptions: - User Growth: 3x year-over-year - Conversion Rate: 15% free to paid - Average Features: 2.5 per paid user - Average Feature Price: $5/month - Enterprise Growth: 10 customers/month Year 1

Year 1 (Months 1-12):

Quarter	Total Users	Paid Users	Individual MRR	Enterprise MRR	Total MRR	ARR
Q1	10,000	1,500	$18,750	$9,966	$28,716	$344,592
Q2	25,000	3,750	$46,875	$19,932	$66,807	$801,684
Q3	40,000	6,000	$75,000	$29,898	$104,898	$1,258,776
Q4	50,000	7,500	$93,750	$39,865	$133,615	$1,603,380

Year 2 (Months 13-24):

Quarter	Total Users	Paid Users	Individual MRR	Enterprise MRR	Total MRR	ARR
Q1	75,000	11,250	$140,625	$60,000	$200,625	$2,407,500
Q2	110,000	16,500	$206,250	$90,000	$296,250	$3,555,000
Q3	135,000	20,250	$253,125	$120,000	$373,125	$4,477,500
Q4	150,000	22,500	$281,250	$150,000	$431,250	$5,175,000

Year 3 (Months 25-36):

Quarter	Total Users	Paid Users	Individual MRR	Enterprise MRR	Total MRR	ARR
Q1	225,000	33,750	$421,875	$225,000	$646,875	$7,762,500
Q2	330,000	49,500	$618,750	$300,000	$918,750	$11,025,000
Q3	405,000	60,750	$759,375	$375,000	$1,134,375	$13,612,500
Q4	450,000	67,500	$843,750	$450,000	$1,293,750	$15,525,000

8.5.2 Expense Forecast

Year 1 Operating Expenses:

Personnel (45% of revenue):

```
Engineering (5 people):        $500,000
Product/Design (2 people):     $200,000
Sales/Marketing (3 people):    $240,000
Customer Success (2 people):   $120,000
Operations/Admin (1 person):   $ 80,000

Total Personnel:               $1,140,000
```

Infrastructure (10% of revenue):

```
AWS/Cloud Hosting:             $ 50,000
MongoDB Atlas:                 $ 30,000
CDN/Storage:                   $ 20,000
Third-party APIs:              $ 20,000
```

```
Security/Monitoring:          $ 15,000

Total Infrastructure:         $135,000
```

Marketing (30% of revenue):

```
Paid Advertising:             $300,000
Content Creation:             $ 60,000
Tools & Software:             $ 40,000
Events/Sponsorships:          $ 50,000

Total Marketing:              $450,000
```

Other Operating Expenses (15% of revenue):

```
Office/Co-working:            $ 50,000
Legal/Accounting:             $ 40,000
Insurance:                    $ 30,000
Software/Tools:               $ 50,000
Miscellaneous:                $ 30,000

Total Other:                  $200,000
```

Year 1 Total Expenses: $1,925,000

8.5.3 Profit & Loss Statement

Year 1:

```
Revenue:                      $1,603,380
Cost of Revenue (10%):        $  160,338
Gross Profit:                 $1,443,042
Operating Expenses:           $1,925,000

Net Income (Loss):            ($ 481,958)
Gross Margin:                 90%
Net Margin:                   -30%
```

Year 2:

```
Revenue:                      $5,625,000
Cost of Revenue (10%):        $  562,500
Gross Profit:                 $5,062,500
Operating Expenses:           $3,937,500

Net Income:                   $1,125,000
Gross Margin:                 90%
Net Margin:                   20%
```

Year 3:

```
Revenue:                      $15,525,000
Cost of Revenue (10%):        $ 1,552,500
```

```
Gross Profit:                  $13,972,500
Operating Expenses:            $ 9,315,000

Net Income:                    $ 4,657,500
Gross Margin:                  90%
Net Margin:                    30%
```

8.5.4 Cash Flow Projection

Year 1 Cash Flow:

```
Beginning Cash:                $2,000,000 (seed funding)
Operating Cash Flow:           ($ 481,958)
Ending Cash:                   $1,518,042
Burn Rate:                     $ 40,163/month (avg)
Runway:                        37 months
```

Key Metrics: - Break-even month: Month 18 - Cash positive: Month 20 - Profitability: Month 24

8.6 Unit Economics

8.6.1 Individual User Economics

Average Revenue Per User (ARPU):

```
Paid Users: 15% of total
Average Features: 2.5 per paid user
Average Price: $5/feature/month
ARPU (paid users): $12.50/month
ARPU (all users): $1.88/month
```

Lifetime Value (LTV) Calculation:

```
Average Subscription Length: 24 months
Monthly Revenue: $12.50
Annual Revenue: $150
LTV = $12.50 × 24 = $300

Conservative (12 months): $150
Optimistic (36 months): $450
```

Customer Acquisition Cost (CAC):

```
Blended CAC: $15
LTV/CAC Ratio: 20:1 (excellent)
Payback Period: 1.2 months
```

Contribution Margin:

```
Monthly Revenue: $12.50
Cost to Serve: $0.50 (infrastructure, support)
Contribution Margin: $12.00
```

```
Contribution Margin %: 96%
```

8.6.2 Enterprise Customer Economics

Average Revenue Per Account (ARPA):

```
Starter Plan: $199/month
Professional Plan: $499/month
Enterprise Plan: $999/month
Weighted Average: $400/month
```

Lifetime Value (LTV):

```
Average Contract Length: 36 months
Monthly Revenue: $400
LTV = $400 × 36 = $14,400

Conservative (24 months): $9,600
Optimistic (48 months): $19,200
```

Customer Acquisition Cost (CAC):

```
Sales & Marketing: $2,000
LTV/CAC Ratio: 7.2:1 (healthy)
Payback Period: 5 months
```

Annual Contract Value (ACV):

```
Starter: $2,388
Professional: $5,988
Enterprise: $11,988
Average ACV: $4,800
```

8.6.3 Cohort Analysis

Month 1 Cohort (1,000 users):

Month	Active Users	Retention %	Paid Users	Conversion %	MRR	Cumulative Revenue
1	1,000	100%	150	15%	$1,875	$1,875
2	600	60%	120	20%	$1,500	$3,375
3	450	45%	110	24%	$1,375	$4,750
6	350	35%	105	30%	$1,313	$12,638
12	250	25%	100	40%	$1,250	$27,388

Cohort LTV: $27.39 per user

8.6.4 Key Metrics Targets

Growth Metrics: - Monthly User Growth: 20% MoM (Year 1) - Paid Conversion Rate: 15% - Viral Coefficient: 0.3 - Monthly Active Users (MAU): 40% of total users

Revenue Metrics: - MRR Growth: 25% MoM - ARR: $1.6M (Year 1), $5.6M (Year 2), $15.5M (Year 3) - Net Revenue Retention: 110% - Gross Revenue Retention: 85%

Efficiency Metrics: - CAC: $15 (individual), $2,000 (enterprise) - LTV: $300 (individual), $14,400 (enterprise) - LTV/CAC: 20:1 (individual), 7.2:1 (enterprise) - Gross Margin: 90% - Rule of 40: 50%+ (Growth % + Profit %)

Engagement Metrics: - Daily Active Users (DAU): 25% of MAU - Feature Adoption: 70% use 2+ features - Weekly Engagement: 3+ sessions/week - NPS Score: 50+

8.7 Competitive Analysis

8.7.1 Competitive Landscape

Direct Competitors:

1. Notion - Pricing: Free, Personal Pro $10/month, Team $15/user/month - Strengths: Flexibility, databases, collaboration - Weaknesses: Steep learning curve, performance issues - Our Advantage: Simpler UI, better mobile, à la carte pricing

2. ClickUp - Pricing: Free, Unlimited $7/user/month, Business $12/user/month - Strengths: Feature-rich, customization, views - Weaknesses: Complexity, overwhelming, expensive - Our Advantage: Simpler, pay-per-feature, better UX

3. Asana - Pricing: Free, Premium $10.99/user/month, Business $24.99/user/month - Strengths: Project management, team collaboration - Weaknesses: Limited free tier, expensive, focused on tasks - Our Advantage: More features, better pricing, individual focus

4. Monday.com - Pricing: Individual $8/seat/month, Basic $24/user/month, Standard $30/user/month - Strengths: Visual, customizable, integrations - Weaknesses: Very expensive, overkill for individuals - Our Advantage: 70% cheaper, individual features, simpler

5. Todoist - Pricing: Free, Pro $4/month, Business $6/user/month - Strengths: Simple, fast, reliable - Weaknesses: Limited features, basic, task-only - Our Advantage: More features, same price point, all-in-one

8.7.2 Competitive Positioning

Positioning Matrix:

```
                    High Price

        Monday.com             Asana

    ClickUp
Complexity                Simplicity

        Notion                 PrMS
                             Todoist
                    Low Price
```

Our Unique Position: - Simplicity + Affordability + Modularity - Better than Notion: Easier to use, better mobile - Better than ClickUp: Less complex, more affordable - Better than Monday.com: 70% cheaper - Better than Todoist: More features, same price

8.7.3 Competitive Advantages

1. À La Carte Pricing: - Only competitor: None (all require full subscriptions) - Advantage: Pay only for what you use - Market opportunity: $2B+ (underserved)

2. Dual Market Focus: - Individual + Enterprise in one platform - Seamless transition as users grow - Network effects between segments

3. Better Mobile Experience: - Offline-first architecture - Native iOS/Android apps - Fast, reliable sync

4. Fair Pricing: - 30-50% cheaper than competitors - Transparent pricing - No hidden fees - Annual discounts

5. Superior UX: - Intuitive design - Less clutter - Faster onboarding - Better accessibility

8.7.4 Market Opportunity

Total Addressable Market (TAM):

```
Individual Productivity Software:  $8B
Enterprise Productivity Software:  $12B
Total TAM:                         $20B
```

Serviceable Addressable Market (SAM):

```
English-speaking markets:          $12B
Our target segments:               $8B
```

Serviceable Obtainable Market (SOM):

```
Year 1: 0.02% of SAM = $1.6M
Year 3: 0.19% of SAM = $15.5M
Year 5: 1% of SAM = $80M (projected)
```

Market Share Targets: - Year 1: Capture 50,000 users (0.01% of market) - Year 3: Capture 450,000 users (0.1% of market) - Year 5: Capture 2M users (0.5% of market)

8.7.5 Barriers to Entry

Our Moats:

1. Network Effects: - Team collaboration drives virality - User-generated templates - Community knowledge base

2. Data Lock-in: - Years of user data and history - Switching cost increases over time - Export/migration friction

3. Brand & Trust: - First-mover in à la carte model - User reviews and testimonials - Industry recognition

4. Technology: - Proprietary sync algorithm - Offline-first architecture - AI/ML models (future)

5. Economies of Scale: - Lower cost per user over time - Better unit economics - Negotiating power with vendors

---

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Business Strategy Team - Status: Draft - Business Planning Phase

Next Chapter: Chapter 9: Development Roadmap

Chapter 9: Development Roadmap

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Planning Phase

Table of Contents

- 9.1 Development Phases
- 9.2 MVP Strategy
- 9.3 Timeline & Milestones
- 9.4 Resource Allocation
- 9.5 Sprint Planning
- 9.6 Risk Management
- 9.7 Success Metrics

9.1 Development Phases

9.1.1 Phase Overview

Phase 0: Foundation (Months 1-2) - Infrastructure setup - CI/CD pipeline - Design system - Core architecture - Team onboarding

Phase 1: MVP (Months 3-5) - 3 free individual features - Authentication & user management - Basic mobile apps - Payment integration - Core infrastructure

Phase 2: Premium Features (Months 6-12) - Release 2 premium features/month - Enhanced analytics - Integration ecosystem - Performance optimization - User feedback iteration

Phase 3: Enterprise Launch (Months 13-18) - Enterprise authentication (SSO) - Team collaboration features - Advanced permissions - Admin dashboard - Enterprise integrations

Phase 4: Scale & Optimize (Months 19-24) - AI-powered features - Advanced analytics - White-label options - Global expansion - Performance at scale

9.1.2 Development Methodology

Framework: Agile/Scrum

Sprint Structure: - Sprint Length: 2 weeks - Planning: Monday (4 hours) - Daily Standups: 15 minutes - Sprint Review: Last Friday (2 hours) - Retrospective: Last Friday (1 hour)

Release Cadence: - Web: Weekly releases (Friday) - Mobile: Bi-weekly releases (App Store review time) - Hotfixes: As needed (within 4 hours for critical)

Quality Gates: - Code review required - 80%+ test coverage - No critical bugs - Performance benchmarks met - Accessibility audit passed

---

9.2 MVP Strategy

9.2.1 MVP Scope

Core Philosophy: - Solve ONE problem exceptionally well - Get to market fast (3 months) - Learn from real users - Iterate based on feedback

MVP Features:

1. Task Management (FREE)

```
Must Have:
  Create/edit/delete tasks
  Due dates and priorities
  Simple organization (tags)
  Mobile apps (iOS/Android)
  Web app
  Real-time sync

Nice to Have (post-MVP):
  Subtasks
  Recurring tasks
  File attachments
  Advanced filters
```

2. Event Tracker & Calendar (FREE)

```
Must Have:
  Create/edit/delete events
  Calendar views (month, week, day)
  Reminders
  Google Calendar sync
  Mobile & web

Nice to Have (post-MVP):
  Recurring events
  Multiple calendars
  Calendar sharing
  Outlook integration
```

3. Note Taking (FREE)

```
Must Have:
  Create/edit/delete notes
  Rich text editor
  Organization (notebooks)
  Search
  Mobile & web

Nice to Have (post-MVP):
  Markdown support
  Code highlighting
  Templates
  Version history
```

Supporting Systems:

```
Authentication:
  Email/password signup
  Google OAuth
  Password reset
  Email verification

Payment System:
  Stripe integration
  Subscription management
  Payment methods
  Billing portal

User Management:
  Profile settings
  Preferences
  Account deletion
  Data export
```

9.2.2 MVP Success Criteria

Launch Criteria: - All 3 free features functional - Mobile apps in app stores - Payment system live - No critical bugs - Core user flows tested - Basic analytics tracking

Success Metrics (First 3 Months): - 10,000 signups - 30% activation rate (3+ sessions) - 10% conversion to paid (any premium feature) - <5% churn rate - NPS score: 40+ - Average session: 15+ minutes

9.2.3 MVP Timeline

Month 1-2: Foundation

```
Week 1-2: Infrastructure
- Set up monorepo
- Configure CI/CD
- Database setup (MongoDB Atlas)
```

```
- Authentication service
- Deployment pipeline

Week 3-4: Core Backend
- User API
- Task API
- Calendar API
- Notes API
- WebSocket server

Week 5-6: Frontend Foundation
- Design system implementation
- Component library
- State management
- API client
- Routing

Week 7-8: Mobile Foundation
- iOS project setup
- Android project setup
- Shared business logic
- Offline storage
- Sync service
```

Month 3-4: Feature Development

```
Week 9-10: Tasks Feature
- Backend complete
- Web UI complete
- iOS app complete
- Android app complete
- Testing

Week 11-12: Calendar Feature
- Backend complete
- Web UI complete
- Mobile apps complete
- Google Calendar integration
- Testing

Week 13-14: Notes Feature
- Backend complete
- Rich text editor
- Mobile apps complete
- Search functionality
- Testing

Week 15-16: Integration & Polish
- End-to-end testing
```

```
- Bug fixes
- Performance optimization
- UI polish
- Documentation
```

Month 5: Beta & Launch

```
Week 17-18: Private Beta
- Invite 100 beta users
- Monitor usage
- Fix critical bugs
- Gather feedback
- Iterate

Week 19-20: Public Launch
- Final QA
- App store submissions
- Marketing launch
- Monitor metrics
- Support setup
```

9.3 Timeline & Milestones

9.3.1 Detailed Timeline

Q1 2026 (Months 1-3): Foundation & MVP Development

January 2026: - Week 1: Team formation, tools setup - Week 2: Infrastructure & CI/CD - Week 3: Backend architecture - Week 4: Frontend foundation - Milestone: Infrastructure complete

February 2026: - Week 1: Mobile setup (iOS/Android) - Week 2: Tasks backend complete - Week 3: Tasks frontend complete - Week 4: Tasks mobile complete - Milestone: First feature complete

March 2026: - Week 1: Calendar backend - Week 2: Calendar frontend - Week 3: Notes feature - Week 4: Integration testing - Milestone: MVP feature-complete

Q2 2026 (Months 4-6): Beta Testing & Premium Features

April 2026: - Week 1-2: Private beta (100 users) - Week 3: Bug fixes and iteration - Week 4: Public launch preparation - Milestone: Private beta launch

May 2026: - Week 1: Public launch - Week 2-3: Finance Manager development - Week 4: Habit Tracker development - Milestone: Public launch + 2 premium features

June 2026: - Week 1-2: Goal Setting feature - Week 3-4: Time Tracking feature - Milestone: 4 premium features live

Q3 2026 (Months 7-9): Premium Feature Expansion

July 2026: - Week 1-2: Health & Wellness feature - Week 3-4: Focus Timer feature - Milestone: 6 premium features

August 2026: - Week 1-2: Dashboard feature - Week 3-4: File Manager feature - Milestone: 8 premium features

September 2026: - Week 1-2: Journal feature - Week 3-4: Polish & optimization - Milestone: All individual features complete

Q4 2026 (Months 10-12): Enterprise Preparation

October 2026: - Week 1-2: Enterprise authentication (SSO) - Week 3-4: Team management - Milestone: Enterprise infrastructure

November 2026: - Week 1-2: Project Management (free) - Week 3-4: Team Communication (free) - Milestone: First enterprise features

December 2026: - Week 1-2: Time & Attendance (free) - Week 3-4: Year-end optimization - Milestone: Enterprise free tier complete

9.3.2 Year 2 Roadmap (2027)

Q1 2027: Enterprise Premium Features - Inventory Management - Order Management - Finance & Accounting - Milestone: 3 enterprise premium features

Q2 2027: CRM & Invoicing - CRM system - Invoice & Billing - Analytics Dashboard - Milestone: 6 enterprise premium features

Q3 2027: Advanced Enterprise - Resource Planning - Document Management - Vendor Management - Milestone: All enterprise features complete

Q4 2027: AI & Advanced Features - AI task suggestions - Smart scheduling - Predictive analytics - Milestone: AI features launched

9.3.3 Critical Milestones

M1: Infrastructure Ready (Month 2) - All development environments setup - CI/CD pipeline operational - Monitoring and logging active - Team productivity established

M2: MVP Feature Complete (Month 3) - All 3 free features functional - Mobile apps built - Authentication working - Ready for beta testing

M3: Private Beta Launch (Month 4) - 100 beta users onboarded - Feedback collection system - Usage analytics tracking - Critical bugs < 5

M4: Public Launch (Month 5) - App store approvals - Marketing campaign live - Payment system operational - Support team ready - Success criteria: 1,000 signups in Week 1

M5: Premium Features Launched (Month 6-9) - 2 new features per month - Conversion rate $> 10\%$ - Revenue $>$ \$10K MRR - Retention $> 60\%$ (Month 1)

M6: Enterprise Launch (Month 12) - SSO authentication - Team features - First enterprise customer - Enterprise revenue $>$ \$5K MRR

9.4 Resource Allocation

9.4.1 Team Structure

Phase 1 (MVP - Months 1-5): 8 people

Engineering (5): - 1 Tech Lead / Full-stack Engineer - 2 Frontend Engineers (React/Next.js) - 1 iOS Engineer (Swift) - 1 Android Engineer (Kotlin)

Product & Design (2): - 1 Product Manager - 1 UI/UX Designer

Operations (1): - 1 DevOps/Infrastructure Engineer

Phase 2 (Premium Features - Months 6-12): 12 people

Engineering (8): - 1 Engineering Manager - 2 Backend Engineers (Node.js) - 2 Frontend Engineers - 1 iOS Engineer - 1 Android Engineer - 1 DevOps Engineer

Product & Design (2): - 1 Product Manager - 1 UI/UX Designer

Growth (2): - 1 Growth/Marketing Lead - 1 Customer Success Manager

Phase 3 (Enterprise - Months 13-18): 20 people

Engineering (12): - 1 VP of Engineering - 1 Engineering Manager (Frontend) - 1 Engineering Manager (Backend) - 3 Backend Engineers - 3 Frontend Engineers - 1 iOS Engineer - 1 Android Engineer - 1 DevOps Engineer

Product & Design (3): - 1 Head of Product - 1 Senior Product Manager - 1 UI/UX Designer

Sales & Marketing (3): - 1 Head of Growth - 1 Sales Manager (Enterprise) - 1 Marketing Manager

Operations (2): - 1 Customer Success Manager - 1 Operations Manager

9.4.2 Budget Allocation

Year 1 Budget: $2,000,000

Personnel (60%): $1,200,000

```
Engineering:       $700,000 (58%)
Product/Design:    $250,000 (21%)
Growth/Ops:        $250,000 (21%)
```

Infrastructure (8%): $160,000

```
AWS/Cloud:         $80,000
MongoDB Atlas:     $30,000
Stripe fees:       $20,000
Third-party APIs:  $30,000
```

Marketing (20%): $400,000

```
Paid advertising:  $240,000
Content creation:  $80,000
Tools/Software:    $40,000
Events:            $40,000
```

Operations (12%): $240,000

```
Office/Co-working: $60,000
Legal/Accounting:  $50,000
Software/Tools:    $80,000
Miscellaneous:     $50,000
```

9.4.3 Sprint Capacity Planning

Engineering Capacity:

Story Points per Sprint (2 weeks): - Senior Engineer: 13 points - Mid-level Engineer: 10 points - Junior Engineer: 8 points

MVP Phase Team Capacity:

```
1 Senior (Tech Lead): 13 points
2 Mid-level Frontend: 20 points
1 Mid-level iOS:      10 points
1 Mid-level Android:  10 points

Total: 53 points/sprint
```

Feature Complexity Estimates:

```
Simple feature:   13-21 points (1-2 sprints)
Medium feature:   34-55 points (3-4 sprints)
Complex feature:  89-144 points (6-8 sprints)
```

Example: Task Management (MVP)

```
Backend API:           21 points
Frontend Web:          21 points
iOS App:              13 points
Android App:          13 points
Testing & Polish:      13 points

Total:                81 points
Sprints needed:       ~3 sprints (6 weeks)
```

9.5 Sprint Planning

9.5.1 Sprint Structure

Sprint 0 (Pre-MVP): Infrastructure

```
Epic: Project Setup

  Set up monorepo with Turborepo       [8]
  Configure CI/CD pipeline             [5]
  Set up MongoDB Atlas cluster         [3]
  Configure AWS infrastructure         [8]
```

```
  Set up error tracking (Sentry)       [3]
  Configure analytics (Mixpanel)       [3]
  Design system foundations            [8]
  API documentation setup              [3]

Total: 41 points
```

Sprint 1-2: Authentication & User Management

```
Epic: User Authentication

  User registration API                [5]
  Login/logout API                     [3]
  Password reset flow                  [5]
  Email verification                   [5]
  Google OAuth integration             [8]
  JWT token management                 [5]
  Frontend auth pages                  [8]
  iOS auth screens                     [5]
  Android auth screens                 [5]
  Session management                   [3]

Total: 52 points (2 sprints)
```

Sprint 3-5: Task Management Feature

```
Epic: Task Management

Sprint 3: Backend
  Task CRUD APIs                       [8]
  Task filtering/sorting               [5]
  WebSocket updates                    [8]
  Database indexes                     [3]

Sprint 4: Frontend
  Task list view                       [8]
  Task creation modal                  [5]
  Task details page                    [5]
  Filters and sorting UI               [5]

Sprint 5: Mobile
  iOS task screens                     [13]
  Android task screens                 [13]
  Offline sync logic                   [8]
  Testing & bug fixes                  [5]

Total: 86 points (3 sprints)
```

9.5.2 Release Planning

Release Schedule:

v0.1.0 (Week 8): Alpha - Internal testing only - Core authentication - Basic task management - Known bugs acceptable

v0.5.0 (Week 16): Beta - All 3 MVP features - Mobile apps functional - Private beta users (100) - Critical bugs fixed

v1.0.0 (Week 20): Public Launch - Production ready - All features polished - App store approved - Zero critical bugs - Performance optimized

v1.1.0 - v1.9.0 (Monthly): Premium Features - v1.1.0: Finance Manager - v1.2.0: Habit Tracker - v1.3.0: Goal Setting - v1.4.0: Time Tracking - v1.5.0: Health & Wellness - v1.6.0: Focus Timer - v1.7.0: Dashboard - v1.8.0: File Manager - v1.9.0: Journal

v2.0.0 (Month 13): Enterprise Launch - Enterprise authentication - Team features - Admin dashboard - Breaking API changes allowed

9.5.3 Technical Debt Management

Debt Allocation: - 20% of each sprint for technical debt - Dedicated Tech Health sprint every 6 sprints - No new features in tech health sprint

Debt Categories:

Critical (Fix Immediately): - Security vulnerabilities - Data corruption risks - Performance degradation >50% - Accessibility blockers

High (Fix in 2 sprints): - Performance issues - Flaky tests - Poor error handling - Missing monitoring

Medium (Fix in 4 sprints): - Code duplication - Missing tests - Documentation gaps - Outdated dependencies

Low (Backlog): - Code style improvements - Refactoring opportunities - Nice-to-have tooling

Debt Tracking:

```
// Add tech debt label to tickets
{
  title: "Refactor task sync logic",
  type: "tech_debt",
  severity: "medium",
  effort: "8 points",
  impact: "Improves maintainability",
  created: "2026-03-15"
}
```

9.6 Risk Management

9.6.1 Technical Risks

Risk 1: Sync Conflicts

```
Risk Level: HIGH
Impact: Critical (data loss)
Probability: Medium

Mitigation:
- Implement CRDT (Conflict-free Replicated Data Types)
- Version vectors for conflict detection
- User-friendly conflict resolution UI
- Comprehensive testing
- Rollback capabilities

Contingency:
- Manual conflict resolution
- Data recovery procedures
- User support for conflicts
```

Risk 2: Mobile App Rejection

```
Risk Level: MEDIUM
Impact: High (delayed launch)
Probability: Medium

Mitigation:
- Review App Store guidelines thoroughly
- Test on real devices
- Beta testing via TestFlight/Play Console
- Legal review of ToS and Privacy Policy
- Compliance checklist

Contingency:
- Web app priority launch
- Appeal process preparation
- Alternative distribution (TestFlight)
```

Risk 3: Performance at Scale

```
Risk Level: MEDIUM
Impact: High (user experience)
Probability: Medium

Mitigation:
- Load testing before launch
- Database indexing strategy
- Caching layer (Redis)
- CDN for static assets
- Auto-scaling infrastructure

Contingency:
- Rate limiting
- Feature flags for degradation
```

```
- Horizontal scaling
```

Risk 4: Third-Party Service Outages

```
Risk Level: LOW
Impact: Medium
Probability: Low

Mitigation:
- Stripe: Payment retry logic
- Google Calendar: Queue sync jobs
- Email: Multiple providers (SendGrid + AWS SES)
- Monitoring and alerts

Contingency:
- Manual payment processing
- Graceful degradation
- Status page communication
```

9.6.2 Business Risks

Risk 5: Low Conversion Rate

```
Risk Level: HIGH
Impact: Critical (revenue)
Probability: Medium

Mitigation:
- A/B test pricing
- User research
- Clear value proposition
- Generous free trial
- Usage analytics

Contingency:
- Adjust pricing strategy
- Add more free features
- Introduce bundles earlier
- Extended free trials
```

Risk 6: High Churn Rate

```
Risk Level: HIGH
Impact: Critical (growth)
Probability: Medium

Mitigation:
- Onboarding optimization
- Usage monitoring
- Proactive support
```

```
- Feature engagement tracking
- Exit surveys

Contingency:
- Win-back campaigns
- Pause subscription option
- Pricing adjustments
- Feature improvements
```

Risk 7: Competitive Response

```
Risk Level: MEDIUM
Impact: High
Probability: High

Mitigation:
- Fast execution (time to market)
- Unique positioning (à la carte)
- Strong brand building
- Customer lock-in (data/habits)
- Patent defensive publishing

Contingency:
- Price competition
- Feature differentiation
- Better customer service
- Niche focus
```

9.6.3 Team Risks

Risk 8: Key Person Dependency

```
Risk Level: MEDIUM
Impact: High
Probability: Low

Mitigation:
- Documentation culture
- Pair programming
- Knowledge sharing sessions
- Cross-training
- Code reviews

Contingency:
- Contractor network
- Recruitment pipeline
- Retention bonuses
```

Risk 9: Burnout

```
Risk Level: MEDIUM
Impact: Medium
Probability: Medium

Mitigation:
- Realistic sprint planning
- No weekend work
- PTO encouraged
- Mental health support
- Regular check-ins

Contingency:
- Reduce scope
- Extend timeline
- Hire contractors
```

9.6.4 Risk Monitoring

Weekly Risk Review: - Review top 5 risks - Update probability/impact - Adjust mitigation strategies - Escalate as needed

Risk Dashboard Metrics:

```
Technical Health:
- Test coverage: >80%
- Build success rate: >95%
- P0 bugs: 0
- P1 bugs: <5

Business Health:
- Conversion rate: >10%
- Churn rate: <5%
- NPS: >40
- Support ticket volume: <100/week

Team Health:
- Sprint velocity: stable
- On-call incidents: <2/week
- Team satisfaction: >7/10
- Turnover: <10%/year
```

9.7 Success Metrics

9.7.1 Product Metrics

Activation Metrics:

```
Sign up to activation: <5 minutes
Activation rate: >30%
```

```
Time to first value: <2 minutes
Features adopted (first week): >2
```

Engagement Metrics:

```
Daily Active Users (DAU): 25% of MAU
Weekly Active Users (WAU): 50% of MAU
Monthly Active Users (MAU): 40% of total users
Sessions per week: >3
Session duration: >15 minutes
Feature usage: 70% use 2+ features
```

Retention Metrics:

```
Day 1 retention: >60%
Day 7 retention: >40%
Day 30 retention: >30%
Month 6 retention: >20%
Annual retention: >60%
```

Conversion Metrics:

```
Free to paid conversion: >15%
Trial to paid: >40%
Time to first purchase: <14 days
Average features per user: >2.5
Upgrade rate: >5% monthly
```

9.7.2 Technical Metrics

Performance:

```
Page load time: <2 seconds
API response time (p95): <200ms
Database query time (p95): <50ms
Mobile app launch: <1 second
Sync latency: <1 second
```

Reliability:

```
Uptime: >99.9%
Error rate: <0.1%
Failed API calls: <1%
Crash-free sessions: >99.9%
Data loss incidents: 0
```

Quality:

```
Test coverage: >80%
Build success rate: >95%
Code review turnaround: <4 hours
Deployment frequency: Daily (web)
Mean time to recovery: <1 hour
```

Security:

```
Vulnerability patches: <24 hours
Security audits: Quarterly
Penetration tests: Bi-annually
Data breaches: 0
Compliance violations: 0
```

9.7.3 Business Metrics

Growth Metrics:

```
Monthly user growth: >20%
MRR growth: >25%
Viral coefficient: >0.3
CAC payback: <4 months
LTV/CAC ratio: >5:1
```

Revenue Metrics:

```
MRR (Month 6): >$10,000
MRR (Month 12): >$100,000
ARR (Year 1): >$1,000,000
ARPU: >$10
Net revenue retention: >100%
```

Customer Satisfaction:

```
NPS score: >50
Customer satisfaction: >4.5/5
Support response time: <2 hours
Issue resolution time: <24 hours
Feature request response: <1 week
```

9.7.4 Milestone Success Criteria

MVP Launch Success: - 1,000 signups in first week - 30% activation rate - <1% critical bug rate - NPS >40 - Media coverage (3+ articles)

Month 6 Success: - 25,000 total users - 3,750 paid users (15%) - $10,000+ MRR - <5% monthly churn - 4 premium features launched

Year 1 Success: - 50,000 total users - 7,500 paid users (15%) - $100,000+ MRR ($1.2M ARR) - All individual features launched - Break-even or profitable

Year 2 Success: - 150,000 total users - 22,500 paid users (15%) - $400,000+ MRR ($4.8M ARR) - All enterprise features launched - 30%+ profit margin

9.7.5 Monitoring & Reporting

Daily Dashboard: - Signups (last 24h) - Active users (DAU) - Revenue (MRR) - Critical errors - System health

Weekly Review: - Growth metrics - Conversion funnel - Feature adoption - Churn analysis - Top issues

Monthly Business Review: - Financial performance - User growth - Product roadmap progress - Team velocity - Strategic initiatives

Quarterly Board Review: - Financial results vs. targets - Product milestones - Market position - Strategic pivots - Fundraising needs

---

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Product & Engineering Teams - Status: Draft - Planning Phase

Next Chapter: Chapter 10: Security, Compliance & Testing

Chapter 10: Security, Compliance & Testing

Document Version: 1.0
Last Updated: January 19, 2026
Status: Draft - Final Phase

Table of Contents

- 10.1 Security Architecture
- 10.2 Data Protection & Privacy
- 10.3 Compliance & Regulations
- 10.4 Testing Strategy
- 10.5 Performance Optimization
- 10.6 Monitoring & Logging
- 10.7 Disaster Recovery

10.1 Security Architecture

10.1.1 Security Principles

Security by Design: - Defense in depth - Principle of least privilege - Zero trust architecture - Fail securely - Security awareness

Security Layers:

```
Application Security
- Input validation
- Output encoding
- Authentication/Authorization

API Security
- Rate limiting
```

```
  - API key management
  - CORS policies

  Network Security
  - TLS/SSL encryption
  - VPC isolation
  - Firewall rules

  Infrastructure Security
  - Container security
  - Secrets management
  - Access controls

  Data Security
  - Encryption at rest
  - Encryption in transit
  - Backup encryption
```

10.1.2 Authentication & Authorization

Authentication Mechanisms:

JWT Implementation:

```
// src/services/auth/jwt.service.ts
import jwt from 'jsonwebtoken';
import { User } from '@/types';

interface TokenPayload {
  userId: string;
  email: string;
  mode: 'individual' | 'enterprise' | 'both';
  enterpriseId?: string;
  roles?: string[];
}

export class JWTService {
  private readonly accessSecret: string;
  private readonly refreshSecret: string;

  constructor() {
    this.accessSecret = process.env.JWT_ACCESS_SECRET!;
    this.refreshSecret = process.env.JWT_REFRESH_SECRET!;
  }

  generateAccessToken(user: User): string {
    const payload: TokenPayload = {
      userId: user.id,
```

```
      email: user.email,
      mode: user.mode,
      enterpriseId: user.enterpriseId,
      roles: user.roles
    };

    return jwt.sign(payload, this.accessSecret, {
      expiresIn: '1h',
      issuer: 'prms.io',
      audience: 'prms-api'
    });
  }

  generateRefreshToken(userId: string): string {
    return jwt.sign(
      { userId, type: 'refresh' },
      this.refreshSecret,
      { expiresIn: '7d' }
    );
  }

  verifyAccessToken(token: string): TokenPayload {
    try {
      return jwt.verify(token, this.accessSecret) as TokenPayload;
    } catch (error) {
      if (error instanceof jwt.TokenExpiredError) {
        throw new AuthError('Token expired');
      }
      throw new AuthError('Invalid token');
    }
  }

  verifyRefreshToken(token: string): { userId: string } {
    return jwt.verify(token, this.refreshSecret) as { userId: string };
  }
}
```

Role-Based Access Control (RBAC):

```
// src/middleware/authorization.ts
enum Permission {
  // User permissions
  USER_READ = 'user:read',
  USER_WRITE = 'user:write',
  USER_DELETE = 'user:delete',

  // Enterprise permissions
  ENTERPRISE_ADMIN = 'enterprise:admin',
  ENTERPRISE_MANAGE_USERS = 'enterprise:manage_users',
```

```
  ENTERPRISE_VIEW_ANALYTICS = 'enterprise:view_analytics',

  // Project permissions
  PROJECT_CREATE = 'project:create',
  PROJECT_EDIT = 'project:edit',
  PROJECT_DELETE = 'project:delete',
  PROJECT_VIEW = 'project:view'
}

enum Role {
  USER = 'user',
  ENTERPRISE_OWNER = 'enterprise_owner',
  ENTERPRISE_ADMIN = 'enterprise_admin',
  ENTERPRISE_MANAGER = 'enterprise_manager',
  ENTERPRISE_MEMBER = 'enterprise_member'
}

const rolePermissions: Record<Role, Permission[]> = {
  [Role.USER]: [
    Permission.USER_READ,
    Permission.USER_WRITE
  ],
  [Role.ENTERPRISE_OWNER]: [
    Permission.ENTERPRISE_ADMIN,
    Permission.ENTERPRISE_MANAGE_USERS,
    Permission.ENTERPRISE_VIEW_ANALYTICS,
    Permission.PROJECT_CREATE,
    Permission.PROJECT_EDIT,
    Permission.PROJECT_DELETE,
    Permission.PROJECT_VIEW
  ],
  [Role.ENTERPRISE_ADMIN]: [
    Permission.ENTERPRISE_MANAGE_USERS,
    Permission.ENTERPRISE_VIEW_ANALYTICS,
    Permission.PROJECT_CREATE,
    Permission.PROJECT_EDIT,
    Permission.PROJECT_VIEW
  ],
  [Role.ENTERPRISE_MANAGER]: [
    Permission.PROJECT_CREATE,
    Permission.PROJECT_EDIT,
    Permission.PROJECT_VIEW
  ],
  [Role.ENTERPRISE_MEMBER]: [
    Permission.PROJECT_VIEW
  ]
};
```

```
export function requirePermission(permission: Permission) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const user = req.user; // From auth middleware

    if (!user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }

    const userPermissions = user.roles.flatMap(
      role => rolePermissions[role] || []
    );

    if (!userPermissions.includes(permission)) {
      return res.status(403).json({ error: 'Forbidden' });
    }

    next();
  };
}
```

Multi-Factor Authentication (2FA):

```
// src/services/auth/two-factor.service.ts
import speakeasy from 'speakeasy';
import QRCode from 'qrcode';

export class TwoFactorService {
  async generateSecret(email: string): Promise<{
    secret: string;
    qrCode: string;
  }> {
    const secret = speakeasy.generateSecret({
      name: `PrMS (${email})`,
      issuer: 'PrMS'
    });

    const qrCode = await QRCode.toDataURL(secret.otpauth_url!);

    return {
      secret: secret.base32,
      qrCode
    };
  }

  verifyToken(secret: string, token: string): boolean {
    return speakeasy.totp.verify({
      secret,
      encoding: 'base32',
      token,
```

```
      window: 2 // Allow 2 time steps tolerance
    });
  }

  async enableTwoFactor(
    userId: string,
    secret: string,
    verificationToken: string
  ): Promise<void> {
    if (!this.verifyToken(secret, verificationToken)) {
      throw new Error('Invalid verification token');
    }

    await User.updateOne(
      { _id: userId },
      {
        twoFactorEnabled: true,
        twoFactorSecret: secret
      }
    );
  }
}
```

10.1.3 API Security

Rate Limiting:

```
// src/middleware/rate-limit.ts
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import { redis } from '@/config/redis';

// General API rate limit
export const apiLimiter = rateLimit({
  store: new RedisStore({
    client: redis,
    prefix: 'rl:api:'
  }),
  windowMs: 60 * 1000, // 1 minute
  max: async (req) => {
    const user = req.user;

    // Different limits by subscription tier
    if (!user) return 20; // Anonymous
    if (user.tier === 'free') return 100;
    if (user.tier === 'premium') return 500;
    return 2000; // Enterprise
  },
```

```
  message: 'Too many requests, please try again later',
  standardHeaders: true,
  legacyHeaders: false
});

// Strict limit for authentication endpoints
export const authLimiter = rateLimit({
  store: new RedisStore({
    client: redis,
    prefix: 'rl:auth:'
  }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 attempts
  skipSuccessfulRequests: true,
  message: 'Too many login attempts, please try again later'
});

// Payment endpoints
export const paymentLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 10,
  message: 'Too many payment requests'
});
```

Input Validation & Sanitization:

```
// src/middleware/validation.ts
import { body, param, query, validationResult } from 'express-validator';
import DOMPurify from 'isomorphic-dompurify';

export const validateTask = [
  body('title')
    .trim()
    .notEmpty().withMessage('Title is required')
    .isLength({ max: 200 }).withMessage('Title too long')
    .customSanitizer(value => DOMPurify.sanitize(value)),

  body('description')
    .optional()
    .trim()
    .isLength({ max: 5000 }).withMessage('Description too long')
    .customSanitizer(value => DOMPurify.sanitize(value)),

  body('priority')
    .optional()
    .isIn(['low', 'medium', 'high', 'urgent'])
    .withMessage('Invalid priority'),

  body('dueDate')
```

```
    .optional()
    .isISO8601().withMessage('Invalid date format')
    .custom((value) => {
      const date = new Date(value);
      if (date < new Date()) {
        throw new Error('Due date must be in the future');
      }
      return true;
    }),

  body('tags')
    .optional()
    .isArray().withMessage('Tags must be an array')
    .custom((tags: string[]) => {
      if (tags.length > 10) {
        throw new Error('Maximum 10 tags allowed');
      }
      return tags.every(tag => typeof tag === 'string' && tag.length <= 30);
    })
];

export const handleValidationErrors = (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(422).json({
      success: false,
      error: {
        code: 'VALIDATION_ERROR',
        message: 'Invalid request data',
        details: errors.array()
      }
    });
  }

  next();
};
```

SQL Injection & NoSQL Injection Prevention:

```
// Using Mongoose with proper sanitization
import mongoSanitize from 'express-mongo-sanitize';

app.use(mongoSanitize({
  replaceWith: '_',
```

```
  onSanitize: ({ req, key }) => {
    console.warn(`Sanitized key: ${key}`);
  }
}));

// Safe query construction
async function findTasks(userId: string, filters: any) {
  // BAD - vulnerable to injection
  // const tasks = await Task.find({ userId, ...filters });

  // GOOD - sanitized and validated
  const sanitizedFilters = {
    userId: new Types.ObjectId(userId),
    ...(filters.status && {
      status: { $in: ['not_started', 'in_progress', 'completed'] }
    }),
    ...(filters.priority && {
      priority: { $in: ['low', 'medium', 'high', 'urgent'] }
    })
  };

  return Task.find(sanitizedFilters);
}
```

10.1.4 Data Encryption

Encryption at Rest:

```
// src/services/encryption/encryption.service.ts
import crypto from 'crypto';

export class EncryptionService {
  private readonly algorithm = 'aes-256-gcm';
  private readonly key: Buffer;

  constructor() {
    // Key should be stored in secrets manager (AWS Secrets Manager, etc.)
    this.key = Buffer.from(process.env.ENCRYPTION_KEY!, 'hex');
  }

  encrypt(data: string): {
    encrypted: string;
    iv: string;
    authTag: string;
  } {
    const iv = crypto.randomBytes(16);
    const cipher = crypto.createCipheriv(this.algorithm, this.key, iv);
```

```
    let encrypted = cipher.update(data, 'utf8', 'hex');
    encrypted += cipher.final('hex');

    const authTag = cipher.getAuthTag();

    return {
      encrypted,
      iv: iv.toString('hex'),
      authTag: authTag.toString('hex')
    };
  }

  decrypt(encrypted: string, iv: string, authTag: string): string {
    const decipher = crypto.createDecipheriv(
      this.algorithm,
      this.key,
      Buffer.from(iv, 'hex')
    );

    decipher.setAuthTag(Buffer.from(authTag, 'hex'));

    let decrypted = decipher.update(encrypted, 'hex', 'utf8');
    decrypted += decipher.final('utf8');

    return decrypted;
  }
}

// MongoDB field-level encryption
const noteSchema = new Schema({
  userId: { type: ObjectId, required: true },
  title: String,
  content: {
    encrypted: String,
    iv: String,
    authTag: String
  },
  encrypted: { type: Boolean, default: false }
});

noteSchema.pre('save', async function(next) {
  if (this.encrypted && this.isModified('content')) {
    const encryptionService = new EncryptionService();
    const encrypted = encryptionService.encrypt(
      JSON.stringify(this.content)
    );
    this.content = encrypted;
  }
```

```
  next();
});
```

Password Hashing:

```
// src/services/auth/password.service.ts
import bcrypt from 'bcrypt';

export class PasswordService {
  private readonly saltRounds = 12;

  async hash(password: string): Promise<string> {
    return bcrypt.hash(password, this.saltRounds);
  }

  async verify(password: string, hash: string): Promise<boolean> {
    return bcrypt.compare(password, hash);
  }

  isStrongPassword(password: string): boolean {
    // At least 8 characters
    if (password.length < 8) return false;

    // Contains uppercase
    if (!/[A-Z]/.test(password)) return false;

    // Contains lowercase
    if (!/[a-z]/.test(password)) return false;

    // Contains number
    if (!/[0-9]/.test(password)) return false;

    // Contains special character
    if (!/[!@#$%^&*(),.?":{}|<>]/.test(password)) return false;

    return true;
  }
}
```

10.1.5 Secrets Management

AWS Secrets Manager Integration:

```
// src/config/secrets.ts
import {
  SecretsManagerClient,
  GetSecretValueCommand
} from '@aws-sdk/client-secrets-manager';
```

```
const client = new SecretsManagerClient({ region: 'us-east-1' });

export async function getSecret(secretName: string): Promise<any> {
  try {
    const response = await client.send(
      new GetSecretValueCommand({ SecretId: secretName })
    );

    if (response.SecretString) {
      return JSON.parse(response.SecretString);
    }

    throw new Error('Secret not found');
  } catch (error) {
    console.error('Error retrieving secret:', error);
    throw error;
  }
}

// Usage
async function initializeSecrets() {
  const secrets = await getSecret('prms/production/app-secrets');

  process.env.JWT_ACCESS_SECRET = secrets.jwtAccessSecret;
  process.env.JWT_REFRESH_SECRET = secrets.jwtRefreshSecret;
  process.env.ENCRYPTION_KEY = secrets.encryptionKey;
  process.env.STRIPE_SECRET_KEY = secrets.stripeSecretKey;
}
```

10.2 Data Protection & Privacy

10.2.1 Privacy by Design

Data Minimization: - Collect only necessary data - Clear purpose for each data point - Regular data retention reviews - Automatic deletion of old data

User Control:

```
// src/services/privacy/data-export.service.ts
export class DataExportService {
  async exportUserData(userId: string): Promise<ExportPackage> {
    const [
      profile,
      tasks,
      events,
      notes,
      transactions,
      habits,
```

```
      goals
    ] = await Promise.all([
      User.findById(userId).select('-password -twoFactorSecret'),
      Task.find({ userId }),
      CalendarEvent.find({ userId }),
      Note.find({ userId }),
      Transaction.find({ userId }),
      Habit.find({ userId }),
      Goal.find({ userId })
    ]);

    const exportData = {
      exportDate: new Date().toISOString(),
      profile,
      data: {
        tasks,
        events,
        notes,
        transactions,
        habits,
        goals
      },
      metadata: {
        version: '1.0',
        format: 'json'
      }
    };

    // Create ZIP file with all data
    const zip = await this.createZipArchive(exportData);

    return {
      filename: `prms-data-export-${userId}-${Date.now()}.zip`,
      data: zip
    };
  }
}

// Data deletion service
export class DataDeletionService {
  async deleteUserData(userId: string): Promise<void> {
    // Soft delete - mark as deleted for 30 days grace period
    await User.updateOne(
      { _id: userId },
      {
        status: 'deleted',
        deletedAt: new Date(),
        email: `deleted_${userId}@prms.io`, // Anonymize
```

```
        password: null
      }
    );

    // Schedule hard deletion after 30 days
    await this.scheduleHardDeletion(userId, 30);
  }

  async hardDeleteUserData(userId: string): Promise<void> {
    await Promise.all([
      User.deleteOne({ _id: userId }),
      Task.deleteMany({ userId }),
      CalendarEvent.deleteMany({ userId }),
      Note.deleteMany({ userId }),
      Transaction.deleteMany({ userId }),
      Habit.deleteMany({ userId }),
      Goal.deleteMany({ userId }),
      File.deleteMany({ userId })
    ]);

    // Delete from backups
    await this.removeFromBackups(userId);
  }
}
```

10.2.2 Data Retention Policy

Retention Periods:

```
const retentionPolicies = {
  // Active user data
  activeUserData: 'indefinite', // While account active

  // Deleted user data
  softDeletedData: 30, // days

  // Activity logs
  activityLogs: 90, // days

  // Audit logs
  auditLogs: 365, // days (1 year)

  // Backup data
  dailyBackups: 7, // days
  weeklyBackups: 30, // days
  monthlyBackups: 365, // days

  // Analytics data
```

```
  analyticsRaw: 90, // days
  analyticsAggregated: 730, // days (2 years)

  // Payment records
  paymentRecords: 2555, // days (7 years - legal requirement)
};

// Automated cleanup job
async function cleanupExpiredData() {
  const now = new Date();

  // Delete soft-deleted users after 30 days
  const deletionThreshold = new Date(now.getTime() - 30 * 24 * 60 * 60 * 1000);
  const expiredUsers = await User.find({
    status: 'deleted',
    deletedAt: { $lt: deletionThreshold }
  });

  for (const user of expiredUsers) {
    await hardDeleteUserData(user._id);
  }

  // Clean old activity logs
  const activityThreshold = new Date(now.getTime() - 90 * 24 * 60 * 60 * 1000);
  await ActivityLog.deleteMany({ timestamp: { $lt: activityThreshold } });

  console.log('Data cleanup completed');
}

// Run daily
cron.schedule('0 2 * * *', cleanupExpiredData);
```

10.2.3 Cookie Policy

```
// src/middleware/cookie-consent.ts
import cookieParser from 'cookie-parser';

app.use(cookieParser());

// Cookie categories
enum CookieCategory {
  ESSENTIAL = 'essential',
  FUNCTIONAL = 'functional',
  ANALYTICS = 'analytics',
  MARKETING = 'marketing'
}

const cookieConfig = {
```

```
  essential: {
    description: 'Required for basic site functionality',
    cookies: ['session', 'auth_token', 'csrf_token'],
    canDisable: false
  },
  functional: {
    description: 'Remember your preferences',
    cookies: ['theme', 'language', 'timezone'],
    canDisable: true
  },
  analytics: {
    description: 'Help us improve the product',
    cookies: ['_ga', '_gid', 'mixpanel'],
    canDisable: true
  },
  marketing: {
    description: 'Show relevant advertisements',
    cookies: ['fbp', 'ads'],
    canDisable: true
  }
};

// Set cookies based on consent
function setCookie(
  res: Response,
  name: string,
  value: string,
  category: CookieCategory,
  consent: Record<CookieCategory, boolean>
) {
  if (!consent[category] && cookieConfig[category].canDisable) {
    return;
  }

  res.cookie(name, value, {
    httpOnly: category === CookieCategory.ESSENTIAL,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: category === CookieCategory.ESSENTIAL ? 7 * 24 * 60 * 60 * 1000 : 30 * 24 * 60 * 6
  });
}
```

10.3 Compliance & Regulations

10.3.1 GDPR Compliance

User Rights Implementation:

Right to Access:

```
// GET /api/privacy/data-access
router.get('/privacy/data-access', authenticate, async (req, res) => {
  const userId = req.user.id;

  const userData = await DataExportService.exportUserData(userId);

  res.json({
    success: true,
    data: {
      personalData: userData.profile,
      dataCategories: Object.keys(userData.data),
      processingPurpose: {
        tasks: 'Task management and productivity tracking',
        events: 'Calendar and scheduling',
        notes: 'Note-taking and knowledge management'
      },
      dataSharing: 'We do not share your data with third parties',
      retentionPeriod: 'Data retained while account is active'
    }
  });
});
```

Right to Rectification:

```
// PATCH /api/privacy/update-personal-data
router.patch('/privacy/update-personal-data', authenticate, async (req, res) => {
  const userId = req.user.id;
  const updates = req.body;

  // Allow users to update their personal information
  await User.updateOne(
    { _id: userId },
    { $set: updates },
    { runValidators: true }
  );

  // Log the rectification
  await AuditLog.create({
    userId,
    action: 'data_rectification',
    details: updates,
    timestamp: new Date()
  });

  res.json({ success: true });
});
```

Right to Erasure (Right to be Forgotten):

```
// DELETE /api/privacy/delete-account
router.delete('/privacy/delete-account', authenticate, async (req, res) => {
  const userId = req.user.id;

  // Soft delete with 30-day grace period
  await DataDeletionService.deleteUserData(userId);

  // Send confirmation email
  await sendEmail({
    to: req.user.email,
    subject: 'Account Deletion Scheduled',
    template: 'account-deletion',
    data: {
      gracePeriod: 30,
      recoveryUrl: `${process.env.APP_URL}/recover-account`
    }
  });

  res.json({
    success: true,
    message: 'Account will be deleted in 30 days. You can recover it before then.'
  });
});
```

Right to Data Portability:

```
// POST /api/privacy/export-data
router.post('/privacy/export-data', authenticate, async (req, res) => {
  const userId = req.user.id;
  const format = req.body.format || 'json'; // json, csv, xml

  // Generate export package
  const exportPackage = await DataExportService.exportUserData(userId);

  // Convert to requested format
  const data = await convertFormat(exportPackage, format);

  // Store in S3 for download
  const downloadUrl = await uploadToS3(data, userId);

  // Send download link via email
  await sendEmail({
    to: req.user.email,
    subject: 'Your Data Export is Ready',
    template: 'data-export',
    data: { downloadUrl, expiresIn: '7 days' }
  });

  res.json({ success: true, downloadUrl });
```

```
});
```

Right to Object:

```
// POST /api/privacy/object-processing
router.post('/privacy/object-processing', authenticate, async (req, res) => {
  const userId = req.user.id;
  const { processingType } = req.body; // 'analytics', 'marketing', etc.

  await User.updateOne(
    { _id: userId },
    {
      $set: {
        [`consent.${processingType}`]: false
      }
    }
  );

  // Stop processing immediately
  await stopProcessing(userId, processingType);

  res.json({ success: true });
});
```

10.3.2 CCPA Compliance

California Consumer Rights:

```
// Privacy notice endpoint
router.get('/privacy/ccpa-notice', (req, res) => {
  res.json({
    categories: [
      {
        category: 'Identifiers',
        examples: 'Email, name, user ID',
        collected: true,
        sold: false,
        sharedForBusinessPurpose: false
      },
      {
        category: 'Commercial Information',
        examples: 'Purchase history, subscription data',
        collected: true,
        sold: false,
        sharedForBusinessPurpose: true,
        businessPurpose: 'Payment processing with Stripe'
      },
      {
        category: 'Internet Activity',
```

```
        examples: 'Usage data, feature adoption',
        collected: true,
        sold: false,
        sharedForBusinessPurpose: false
      }
    ],
    yourRights: [
      'Right to know what personal information is collected',
      'Right to delete personal information',
      'Right to opt-out of sale (we do not sell data)',
      'Right to non-discrimination'
    ]
  });
});

// Do Not Sell My Info
router.post('/privacy/do-not-sell', authenticate, async (req, res) => {
  await User.updateOne(
    { _id: req.user.id },
    { $set: { doNotSell: true } }
  );

  res.json({ success: true });
});
```

10.3.3 SOC 2 Compliance Preparation

Security Controls:

```
// Audit logging for all sensitive operations
async function auditLog(event: AuditEvent) {
  await AuditLog.create({
    userId: event.userId,
    action: event.action,
    resource: event.resource,
    resourceId: event.resourceId,
    changes: event.changes,
    ipAddress: event.ipAddress,
    userAgent: event.userAgent,
    timestamp: new Date(),
    result: event.result // 'success' | 'failure'
  });
}

// Critical operations that require audit logging
const auditableActions = [
  'user_login',
  'user_logout',
```

```
  'password_change',
  'two_factor_enable',
  'two_factor_disable',
  'payment_processed',
  'subscription_created',
  'subscription_cancelled',
  'data_export',
  'data_deletion',
  'admin_action',
  'permission_change'
];

// Middleware to auto-log
export function auditMiddleware(action: string) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const originalSend = res.send;

    res.send = function(data) {
      auditLog({
        userId: req.user?.id,
        action,
        resource: req.path,
        ipAddress: req.ip,
        userAgent: req.get('user-agent'),
        result: res.statusCode < 400 ? 'success' : 'failure'
      });

      return originalSend.call(this, data);
    };

    next();
  };
}
```

10.4 Testing Strategy

10.4.1 Testing Pyramid

```
        E2E          Manual/Exploratory: 5%

       Integ.        Integration Tests: 20%

        Unit         Unit Tests: 75%
```

10.4.2 Unit Testing

Backend Unit Tests:

```
// src/services/task/task.service.test.ts
import { TaskService } from './task.service';
import { Task } from '@/models/task.model';

describe('TaskService', () => {
  let taskService: TaskService;

  beforeEach(() => {
    taskService = new TaskService();
  });

  describe('createTask', () => {
    it('should create a task with valid data', async () => {
      const taskData = {
        userId: 'user123',
        title: 'Test Task',
        priority: 'high',
        dueDate: new Date('2026-12-31')
      };

      const task = await taskService.createTask(taskData);

      expect(task.title).toBe('Test Task');
      expect(task.priority).toBe('high');
      expect(task.status).toBe('not_started');
    });

    it('should throw error if title is missing', async () => {
      const taskData = {
        userId: 'user123',
        priority: 'high'
      };

      await expect(taskService.createTask(taskData))
        .rejects
        .toThrow('Title is required');
    });

    it('should set default priority if not provided', async () => {
      const taskData = {
        userId: 'user123',
        title: 'Test Task'
      };
```

```
      const task = await taskService.createTask(taskData);

      expect(task.priority).toBe('medium');
    });
  });

  describe('updateTask', () => {
    it('should update task status', async () => {
      const task = await taskService.createTask({
        userId: 'user123',
        title: 'Test'
      });

      const updated = await taskService.updateTask(task.id, {
        status: 'completed'
      });

      expect(updated.status).toBe('completed');
      expect(updated.completedAt).toBeDefined();
    });
  });
});
```

Frontend Unit Tests:

```
// src/components/TaskList/TaskList.test.tsx
import { render, screen, fireEvent } from '@testing-library/react';
import { TaskList } from './TaskList';

describe('TaskList', () => {
  const mockTasks = [
    { id: '1', title: 'Task 1', status: 'not_started' },
    { id: '2', title: 'Task 2', status: 'completed' }
  ];

  it('renders task list correctly', () => {
    render(<TaskList tasks={mockTasks} />);

    expect(screen.getByText('Task 1')).toBeInTheDocument();
    expect(screen.getByText('Task 2')).toBeInTheDocument();
  });

  it('calls onTaskClick when task is clicked', () => {
    const onTaskClick = jest.fn();
    render(<TaskList tasks={mockTasks} onTaskClick={onTaskClick} />);

    fireEvent.click(screen.getByText('Task 1'));

    expect(onTaskClick).toHaveBeenCalledWith('1');
```

```
  });

  it('filters completed tasks', () => {
    render(<TaskList tasks={mockTasks} filter="completed" />);

    expect(screen.getByText('Task 2')).toBeInTheDocument();
    expect(screen.queryByText('Task 1')).not.toBeInTheDocument();
  });
});
```

10.4.3 Integration Testing

```
// tests/integration/api/tasks.test.ts
import request from 'supertest';
import { app } from '@/app';
import { setupTestDB, cleanupTestDB } from '@/tests/helpers';

describe('Tasks API Integration', () => {
  let authToken: string;

  beforeAll(async () => {
    await setupTestDB();

    // Create test user and get token
    const response = await request(app)
      .post('/api/auth/register')
      .send({
        email: 'test@example.com',
        password: 'TestPass123!',
        firstName: 'Test',
        lastName: 'User'
      });

    authToken = response.body.data.tokens.accessToken;
  });

  afterAll(async () => {
    await cleanupTestDB();
  });

  describe('POST /api/tasks', () => {
    it('should create a new task', async () => {
      const response = await request(app)
        .post('/api/tasks')
        .set('Authorization', `Bearer ${authToken}`)
        .send({
          title: 'Integration Test Task',
          priority: 'high',
```

```
        dueDate: '2026-12-31T00:00:00Z'
      });

    expect(response.status).toBe(201);
    expect(response.body.success).toBe(true);
    expect(response.body.data.title).toBe('Integration Test Task');
  });

  it('should return 401 without auth token', async () => {
    const response = await request(app)
      .post('/api/tasks')
      .send({ title: 'Test' });

    expect(response.status).toBe(401);
  });

  it('should validate required fields', async () => {
    const response = await request(app)
      .post('/api/tasks')
      .set('Authorization', `Bearer ${authToken}`)
      .send({ priority: 'high' }); // Missing title

    expect(response.status).toBe(422);
    expect(response.body.error.code).toBe('VALIDATION_ERROR');
  });
 });
});
```

10.4.4 End-to-End Testing

```
// tests/e2e/user-journey.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Complete User Journey', () => {
  test('new user can sign up, create task, and log out', async ({ page }) => {
    // Sign up
    await page.goto('/signup');
    await page.fill('[name="email"]', 'newuser@example.com');
    await page.fill('[name="password"]', 'SecurePass123!');
    await page.fill('[name="firstName"]', 'John');
    await page.fill('[name="lastName"]', 'Doe');
    await page.click('button[type="submit"]');

    // Wait for redirect to dashboard
    await expect(page).toHaveURL('/dashboard');

    // Create a task
    await page.click('[data-testid="new-task-button"]');
```

```
    await page.fill('[name="title"]', 'My First Task');
    await page.selectOption('[name="priority"]', 'high');
    await page.click('[data-testid="save-task"]');

    // Verify task appears in list
    await expect(page.locator('text=My First Task')).toBeVisible();

    // Log out
    await page.click('[data-testid="user-menu"]');
    await page.click('text=Log Out');

    // Verify redirect to login
    await expect(page).toHaveURL('/login');
  });

  test('user can manage premium subscription', async ({ page }) => {
    // Login
    await page.goto('/login');
    await page.fill('[name="email"]', 'test@example.com');
    await page.fill('[name="password"]', 'TestPass123!');
    await page.click('button[type="submit"]');

    // Navigate to premium feature
    await page.click('text=Finance Manager');

    // Should see upgrade prompt
    await expect(page.locator('text=Upgrade to Premium')).toBeVisible();

    // Click upgrade
    await page.click('[data-testid="upgrade-button"]');

    // Fill payment info (test mode)
    await page.fill('[name="cardNumber"]', '4242424242424242');
    await page.fill('[name="expiry"]', '12/28');
    await page.fill('[name="cvc"]', '123');

    await page.click('[data-testid="subscribe-button"]');

    // Wait for success
    await expect(page.locator('text=Subscription successful')).toBeVisible();

    // Feature should now be accessible
    await page.goto('/finance');
    await expect(page.locator('[data-testid="finance-dashboard"]')).toBeVisible();
  });
});
```

10.4.5 Performance Testing

```
// tests/performance/load-test.ts
import autocannon from 'autocannon';

async function runLoadTest() {
  const result = await autocannon({
    url: 'http://localhost:3000',
    connections: 100,
    duration: 30,
    requests: [
      {
        method: 'GET',
        path: '/api/tasks',
        headers: {
          'Authorization': 'Bearer test-token'
        }
      }
    ]
  });

  console.log('Load Test Results:');
  console.log(`Requests/sec: ${result.requests.average}`);
  console.log(`Latency (avg): ${result.latency.mean}ms`);
  console.log(`Latency (p95): ${result.latency.p95}ms`);
  console.log(`Latency (p99): ${result.latency.p99}ms`);

  // Assert performance requirements
  if (result.latency.p95 > 200) {
    throw new Error('P95 latency exceeds 200ms requirement');
  }
}
```

10.4.6 Test Coverage Goals

Coverage Targets: - Overall: 80%+ - Critical paths: 95%+ - Business logic: 90%+ - UI components: 75%+

```
// jest.config.js
{
  "coverageThresholds": {
    "global": {
      "branches": 80,
      "functions": 80,
      "lines": 80,
      "statements": 80
    },
    "./src/services/**/*.ts": {
      "branches": 90,
```

```
      "functions": 90,
      "lines": 90,
      "statements": 90
    }
  }
}
```

10.5 Performance Optimization

10.5.1 Database Optimization

Query Optimization:

```
// Bad - N+1 query problem
async function getTasksWithProject() {
  const tasks = await Task.find({ userId });

  for (const task of tasks) {
    task.project = await Project.findById(task.projectId); // N queries
  }

  return tasks;
}

// Good - Use populate
async function getTasksWithProject() {
  return Task.find({ userId })
    .populate('projectId')
    .lean(); // Return plain objects for better performance
}

// Even better - Use aggregation for complex queries
async function getTasksWithStats() {
  return Task.aggregate([
    { $match: { userId: new ObjectId(userId) } },
    {
      $lookup: {
        from: 'projects',
        localField: 'projectId',
        foreignField: '_id',
        as: 'project'
      }
    },
    { $unwind: { path: '$project', preserveNullAndEmptyArrays: true } },
    {
      $group: {
        _id: '$status',
```

```
        count: { $sum: 1 },
        tasks: { $push: '$$ROOT' }
      }
    }
  ]);
}
```

Caching Strategy:

```
// src/services/cache/cache.service.ts
import Redis from 'ioredis';

export class CacheService {
  private redis: Redis;

  constructor() {
    this.redis = new Redis(process.env.REDIS_URL);
  }

  async get<T>(key: string): Promise<T | null> {
    const cached = await this.redis.get(key);
    return cached ? JSON.parse(cached) : null;
  }

  async set(key: string, value: any, ttl: number = 3600): Promise<void> {
    await this.redis.setex(key, ttl, JSON.stringify(value));
  }

  async del(key: string): Promise<void> {
    await this.redis.del(key);
  }

  async invalidatePattern(pattern: string): Promise<void> {
    const keys = await this.redis.keys(pattern);
    if (keys.length > 0) {
      await this.redis.del(...keys);
    }
  }
}

// Usage with decorator
function cached(ttl: number = 3600) {
  return function (
    target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor
  ) {
    const originalMethod = descriptor.value;
```

```
    descriptor.value = async function (...args: any[]) {
      const cacheKey = `${propertyKey}:${JSON.stringify(args)}`;
      const cache = new CacheService();

      const cached = await cache.get(cacheKey);
      if (cached) return cached;

      const result = await originalMethod.apply(this, args);
      await cache.set(cacheKey, result, ttl);

      return result;
    };
  };
}

// Apply caching
class TaskService {
  @cached(300) // 5 minutes
  async getTasks(userId: string) {
    return Task.find({ userId });
  }
}
```

10.5.2 API Optimization

Response Compression:

```
import compression from 'compression';

app.use(compression({
  filter: (req, res) => {
    if (req.headers['x-no-compression']) {
      return false;
    }
    return compression.filter(req, res);
  },
  level: 6
}));
```

Pagination & Cursor-based Loading:

```
// Cursor-based pagination for better performance on large datasets
router.get('/tasks', async (req, res) => {
  const { cursor, limit = 20 } = req.query;

  const query: any = { userId: req.user.id };

  if (cursor) {
    query._id = { $gt: new ObjectId(cursor) };
```

```
  }

  const tasks = await Task.find(query)
    .limit(Number(limit) + 1)
    .sort({ _id: 1 });

  const hasMore = tasks.length > Number(limit);
  const results = hasMore ? tasks.slice(0, -1) : tasks;
  const nextCursor = hasMore ? results[results.length - 1]._id : null;

  res.json({
    success: true,
    data: results,
    meta: {
      hasMore,
      nextCursor
    }
  });
});
```

10.5.3 Frontend Optimization

Code Splitting:

```
// src/App.tsx
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./pages/Dashboard'));
const Tasks = lazy(() => import('./pages/Tasks'));
const Calendar = lazy(() => import('./pages/Calendar'));
const Finance = lazy(() => import('./pages/Finance'));

function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/tasks" element={<Tasks />} />
        <Route path="/calendar" element={<Calendar />} />
        <Route path="/finance" element={<Finance />} />
      </Routes>
    </Suspense>
  );
}
```

Virtualization for Large Lists:

```
// src/components/TaskList/VirtualizedTaskList.tsx
import { FixedSizeList } from 'react-window';
```

```
export function VirtualizedTaskList({ tasks }: { tasks: Task[] }) {
  const Row = ({ index, style }: { index: number; style: React.CSSProperties }) => (
    <div style={style}>
      <TaskItem task={tasks[index]} />
    </div>
  );

  return (
    <FixedSizeList
      height={600}
      itemCount={tasks.length}
      itemSize={80}
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}
```

10.6 Monitoring & Logging

10.6.1 Application Monitoring

Health Check Endpoints:

```
// src/routes/health.ts
router.get('/health', async (req, res) => {
  const checks = await Promise.all([
    checkDatabase(),
    checkRedis(),
    checkS3(),
    checkStripe()
  ]);

  const allHealthy = checks.every(check => check.healthy);

  res.status(allHealthy ? 200 : 503).json({
    status: allHealthy ? 'healthy' : 'unhealthy',
    timestamp: new Date().toISOString(),
    checks
  });
});

async function checkDatabase() {
  try {
    await mongoose.connection.db.admin().ping();
```

```
    return { service: 'mongodb', healthy: true };
  } catch (error) {
    return { service: 'mongodb', healthy: false, error: error.message };
  }
}
```

Error Tracking with Sentry:

```
import * as Sentry from '@sentry/node';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 0.1,
  beforeSend(event, hint) {
    // Don't send errors for certain paths
    if (event.request?.url?.includes('/health')) {
      return null;
    }
    return event;
  }
});

// Error handling middleware
app.use(Sentry.Handlers.errorHandler());
```

Metrics with Prometheus:

```
import promClient from 'prom-client';

const register = new promClient.Registry();

// Default metrics
promClient.collectDefaultMetrics({ register });

// Custom metrics
const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.1, 0.5, 1, 2, 5]
});

register.registerMetric(httpRequestDuration);

// Metrics endpoint
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});
```

10.6.2 Logging Strategy

Structured Logging:

```
import winston from 'winston';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: {
    service: 'prms-api',
    environment: process.env.NODE_ENV
  },
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      )
    }),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error'
    }),
    new winston.transports.File({
      filename: 'logs/combined.log'
    })
  ]
});

// Usage
logger.info('User logged in', {
  userId: user.id,
  email: user.email,
  ipAddress: req.ip
});

logger.error('Database connection failed', {
  error: error.message,
  stack: error.stack
});
```

10.7 Disaster Recovery

10.7.1 Backup Strategy

Automated Backups:

```
// Daily database backups
import { exec } from 'child_process';
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';

async function createDatabaseBackup() {
  const timestamp = new Date().toISOString();
  const backupFile = `backup-${timestamp}.gz`;

  // Create MongoDB dump
  await new Promise((resolve, reject) => {
    exec(
      `mongodump --uri="${process.env.MONGODB_URI}" --archive=${backupFile} --gzip`,
      (error) => {
        if (error) reject(error);
        else resolve(null);
      }
    );
  });

  // Upload to S3
  const s3 = new S3Client({ region: 'us-east-1' });
  const fileStream = fs.createReadStream(backupFile);

  await s3.send(new PutObjectCommand({
    Bucket: 'prms-backups',
    Key: `daily/${backupFile}`,
    Body: fileStream,
    ServerSideEncryption: 'AES256'
  }));

  // Clean up local file
  fs.unlinkSync(backupFile);

  logger.info('Database backup completed', { file: backupFile });
}

// Schedule daily at 2 AM
cron.schedule('0 2 * * *', createDatabaseBackup);
```

10.7.2 Incident Response

Incident Response Plan:

Severity Levels: - P0 (Critical): Complete outage, data loss - P1 (High): Major feature broken,

security breach - P2 (Medium): Minor feature broken - P3 (Low): Cosmetic issues

Response Times: - P0: Immediate response, 1 hour resolution target - P1: 15 minute response, 4 hour resolution - P2: 1 hour response, 24 hour resolution - P3: Next business day

Communication Template:

```
// Incident notification
async function notifyIncident(incident: Incident) {
  const message = `
 INCIDENT ALERT
Severity: ${incident.severity}
Service: ${incident.service}
Description: ${incident.description}
Impact: ${incident.impact}
Status: ${incident.status}
Assigned to: ${incident.assignee}
  `;

  // Notify team on Slack
  await slackClient.postMessage('#incidents', message);

  // Send PagerDuty alert for P0/P1
  if (incident.severity <= 1) {
    await pagerDuty.trigger({
      incident_key: incident.id,
      description: incident.description
    });
  }
}
```

Document Control: - Version: 1.0 - Last Updated: January 19, 2026 - Owner: Security & Engineering Teams - Status: Draft - Final Phase

End of Documentation

Summary

This comprehensive Product Requirements & Analysis document for PrMS (Productivity Management System) provides complete specifications across 10 chapters:

1. Executive Summary - Vision, personas, value proposition
2. Individual Features - 12 features with schemas and pricing
3. Enterprise Features - 12 B2B features with workflows
4. Technical Architecture - Full system design and tech stack
5. Database Schemas - Complete MongoDB schemas with indexes
6. API Specifications - RESTful APIs and WebSocket events
7. UI/UX Design - Design system, components, user flows

8. Business Model - Pricing, revenue projections, competitive analysis
9. Development Roadmap - Phases, sprints, milestones
10. Security & Testing - Security architecture, compliance, testing strategy

Total Documentation: 10 comprehensive chapters ready for implementation.